

Chapter 7. Security

Preventing unauthorized access to sensitive data is essential in any environment in which multiple users have access to the same physical or network resources. An operating system, as well as individual users, must be able to protect files, memory, and configuration settings from unwanted viewing and modification. Operating system security includes obvious mechanisms such as accounts, passwords, and file protection. It also includes less obvious mechanisms, such as protecting the operating system from corruption, preventing less privileged users from performing actions (rebooting the computer, for example), and not allowing user programs to adversely affect the programs of other users or the operating system.

In this chapter, we explain how every aspect of the design and implementation of Microsoft Windows was influenced in some way by the stringent requirements of providing robust security.

Security ratings

Having software, including operating systems, rated against well-defined standards helps the government, corporations, and home users protect proprietary and personal data stored in computer systems. The current security rating standard used by the United States and many other countries is the Common Criteria (CC). To understand the security capabilities designed into Windows, however, it's useful to know the history of the security ratings system that influenced the design of Windows: the Trusted Computer System Evaluation Criteria (TCSEC).

Trusted Computer System Evaluation Criteria

The National Computer Security Center (NCSC) was established in 1981 as part of the U.S. Department of Defense's (DoD) National Security Agency (NSA). One goal of the NCSC was to create a range of security ratings, listed in [Table 7-1](#), to indicate the degree of protection commercial operating systems, network components, and trusted applications offer. These security ratings, which can be found at <http://csrc.nist.gov/publications/history/dod85.pdf>, were defined in 1983 and are commonly referred to as the *Orange Book*.

Rating	Description
A1	Verified design
B3	Security domains
B2	Structured protection
B1	Labeled security protection
C2	Controlled access protection
C1	Discretionary access protection (obsolete)
D	Minimal protection

TABLE 7-1 TCSEC rating levels

The TCSEC standard consists of levels-of-trust ratings, where higher levels build on lower levels by adding more rigorous protection and validation requirements. No operating system meets the A1 (verified design) rating. Although a few operating systems have earned one of the B-level ratings, C2 is considered sufficient and the highest rating practical for a general-purpose operating system.

The following were the key requirements for a C2 security rating, and they are still considered the core requirements for any secure operating system:

■ **A secure logon facility** This requires that users be able to be uniquely identified and that they must be granted access to the computer only after they have been authenticated in some way.

■ **Discretionary access control** This allows the owner of a resource (such as a file) to determine who can access the resource and what they can do with it. The owner grants rights that permit various kinds of access to a user or to a group of users.

■ **Security auditing** This affords the ability to detect and record security-related events or any attempts to create, access, or delete system resources. Logon identifiers record the identities of all users, making it easy to trace anyone who performs an unauthorized action.

■ **Object reuse protection** This prevents users from seeing data that another user has deleted or from accessing memory that another user previously used and then released. For example, in some operating systems, it's possible to create a new file of a certain length and then examine the contents of the file to see data that happens to have occupied the location on the disk where the file is allocated. This data might be sensitive information that was stored in another user's file but had been deleted. Object reuse protection prevents this potential security hole by initializing all objects, including files and memory, before they are allocated to a user.

Windows also meets two requirements of B-level security:

■ **Trusted path functionality** This prevents Trojan horse programs from being able to intercept users' names and passwords as they try to log on. The trusted path functionality in Windows comes in the form of its Ctrl+Alt+Delete logon-attention sequence, which cannot be intercepted by nonprivileged applications. This sequence of keystrokes, which is also known as the secure attention sequence (SAS), always displays a system-controlled Windows security screen (if a user is already logged on) or the logon screen so that would-be Trojan horses can easily be recognized. (The SAS can also be sent programmatically via the `SendSAS` API if Group Policy and other restrictions allow it.) A Trojan horse presenting a fake logon dialog box will be bypassed when the SAS is entered.

■ **Trusted facility management** This requires support for separate account roles for administrative functions. For example, separate accounts are provided for administration (Administrators), user accounts charged with backing up the computer, and standard users.

Windows meets all these requirements through its security subsystem and related components.

The Common Criteria

In January 1996, the United States, United Kingdom, Germany, France, Canada, and the Netherlands released the jointly developed Common Criteria for Information Technology Security Evaluation (CCITSE) specification. CCITSE, usually referred to as the Common Criteria (CC), is the recognized multinational standard for product security evaluation. The CC home page is at <http://www.niap-ccevs.org/cc-scheme>.

The CC is more flexible than the TCSEC trust ratings and has a structure closer to the ITSEC standard than to the TCSEC standard. The CC includes the concept of a Protection Profile (PP), used to collect security requirements into easily specified and compared sets, and the concept of a Security Target (ST), which contains a set of security requirements that can be made by reference to a PP. The CC also defines a range of seven Evaluation Assurance Levels (EALs), which indicate a level of confidence in the certification. In this way, the CC (like the ITSEC standard before it) removes the link between functionality and assurance level that was present in TCSEC and earlier certification schemes.

Windows 2000, Windows XP, Windows Server 2003, and Windows Vista Enterprise all achieved Common Criteria certification under the Controlled Access Protection Profile (CAPP). This is roughly equivalent to a TCSEC C2 rating. All received a rating of EAL 4+, the “plus” denoting “flaw remediation.” EAL 4 is the highest level recognized across national boundaries.

In March 2011, Windows 7 and Windows Server 2008 R2 were evaluated as meeting the requirements of the US Government Protection

Profile for General-Purpose Operating Systems in a Networked Environment, version 1.0 (GPOSPP)

(http://www.commoncriteriaportal.org/files/ppfiles/pp_gpospp_v1.0.pdf).

The certification includes the Hyper-V hypervisor. Again, Windows achieved Evaluation Assurance Level 4 with flaw remediation (EAL 4+).

The validation report can be found at

http://www.commoncriteriaportal.org/files/epfiles/st_vid10390-vr.pdf, and the description of the security target, giving details of the

requirements satisfied, can be found at

http://www.commoncriteriaportal.org/files/epfiles/st_vid10390-st.pdf. Similar certifications were achieved by Windows 10 and

Windows Server 2012 R2 in June 2016. The report can be found at

http://www.commoncriteriaportal.org/files/epfiles/cr_windows10.pdf.

Security system components

These are the core components and databases that implement Windows security. (All files mentioned are in the %SystemRoot%\System32 directory unless otherwise specified.)

■ **Security reference monitor (SRM)** This component in the Windows executive (Ntoskrnl.exe) is responsible for defining the access token data structure to represent a security context, performing security access checks on objects, manipulating privileges (user rights), and generating any resulting security audit messages.

■ **Local Security Authority Subsystem Service (Lsass)** This user-mode process runs the image Lsass.exe that is responsible for the local system security policy (such as which users are allowed to log on to the machine, password policies, privileges granted to users and groups, and the system security auditing settings), user authentication, and sending security audit messages to the event log. The Local Security Authority service (Lsasrv.dll), a library that Lsass loads, implements most of this functionality.

■ **LSAIso.exe** This is used by Lsass (if so configured on supported Windows 10 and Server 2016 systems), also known as Credential Guard

(see the upcoming “**Credential Guard**” section for more on Credential Guard), to store users’ token hashes instead of keeping them in Lsass’s memory. Because LsaIso.exe is a Trustlet (Isolated User Mode process) running in VTL 1, no normal process—not even the normal kernel—can access the address space of this process. Lsass itself stores an encrypted blob of the password hash needed when it communicates with LsaIso (via ALPC).

■ **Lsass policy database** This database contains the local system security policy settings. It is stored in the registry in an ACL-protected area under HKLM\SECURITY. It includes such information as what domains are entrusted to authenticate logon attempts, who has permission to access the system and how (interactive, network, and service logons), who is assigned which privileges, and what kind of security auditing is to be performed. The Lsass policy database also stores “secrets” that include logon information used for cached domain logons and Windows service user-account logons. (See Chapter 9, “Management mechanisms,” in *Windows Internals Part 2* for more information on Windows services.)

■ **Security Accounts Manager (SAM)** This service is responsible for managing the database that contains the user names and groups defined on the local machine. The SAM service, which is implemented in Samsrv.dll, is loaded into the Lsass process.

■ **SAM database** This database contains the defined local users and groups along with their passwords and other attributes. On domain controllers, the SAM does not store the domain-defined users, but stores the system’s administrator recovery account definition and password. This database is stored in the registry under HKLM\SAM.

■ **Active Directory** This is a directory service that contains a database that stores information about objects in a domain. A *domain* is a collection of computers and their associated security groups that are managed as a single entity. Active Directory stores information about the objects in the domain, including users, groups, and computers. Password information and privileges for domain users and groups are stored in

Active Directory, which is replicated across the computers that are designated as domain controllers of the domain. The Active Directory server, implemented as Ntdsa.dll, runs in the Lsass process. For more information on Active Directory, see Chapter 10, “Networking,” in Part 2.

■ **Authentication packages** These include dynamic link libraries (DLLs) that run in the context of both Lsass process and client processes and implement Windows authentication policy. An authentication DLL is responsible for authenticating a user by checking whether a given user name and password match (or whatever mechanism was used to provide credentials), and if so, returning to Lsass information detailing the user’s security identity, which Lsass uses to generate a token.

■ **Interactive logon manager (Winlogon)** This is a user-mode process running Winlogon.exe that is responsible for responding to the SAS and for managing interactive logon sessions. Winlogon creates a user’s first process when the user logs on, for example.

■ **Logon user interface (LogonUI)** This is a user-mode process running the image LogonUI.exe that presents users with the user interface they can use to authenticate themselves on the system. LogonUI uses credential providers to query user credentials through various methods.

■ **Credential providers (CPs)** These are in-process COM objects that run in the LogonUI process (started on demand by Winlogon when the SAS is performed) and used to obtain a user’s name and password, smartcard PIN, biometric data (such as a fingerprint), or other identification mechanism. The standard CPs are authui.dll, SmartcardCredentialProvider.dll, BioCredProv.Dll, and FaceCredentialProvider.dll, a face-detection provider added in Windows 10.

■ **Network logon service (Netlogon)** This is a Windows service (Netlogon.dll, hosted in a standard SvcHost) that sets up the secure channel to a domain controller, over which security requests—such as

an interactive logon (if the domain controller is running Windows NT 4) or LAN Manager and NT LAN Manager (v1 and v2) authentication validation—are sent. Netlogon is also used for Active Directory logons.

■ **Kernel Security Device Driver (KSecDD)** This is a kernel-mode library (%SystemRoot%\System32\Drivers\Ksecdd.sys) of functions that implement the advanced local procedure call (ALPC) interfaces that other kernel mode security components, including the Encrypting File System (EFS), use to communicate with Lsass in user mode.

■ **AppLocker** This mechanism allows administrators to specify which executable files, DLLs, and scripts can be used by specified users and groups. AppLocker consists of a driver (%SystemRoot%\System32\Drivers\AppId.sys) and a service (AppIdSvc.dll) running in a standard SvcHost process.

Figure 7-1 shows the relationships among some of these components and the databases they manage.

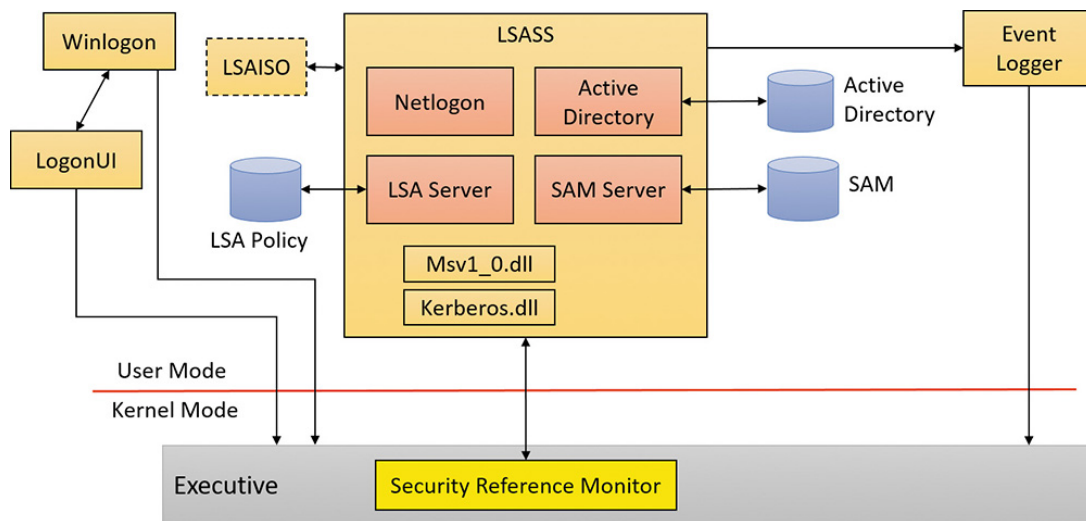


FIGURE 7-1 Windows security components.

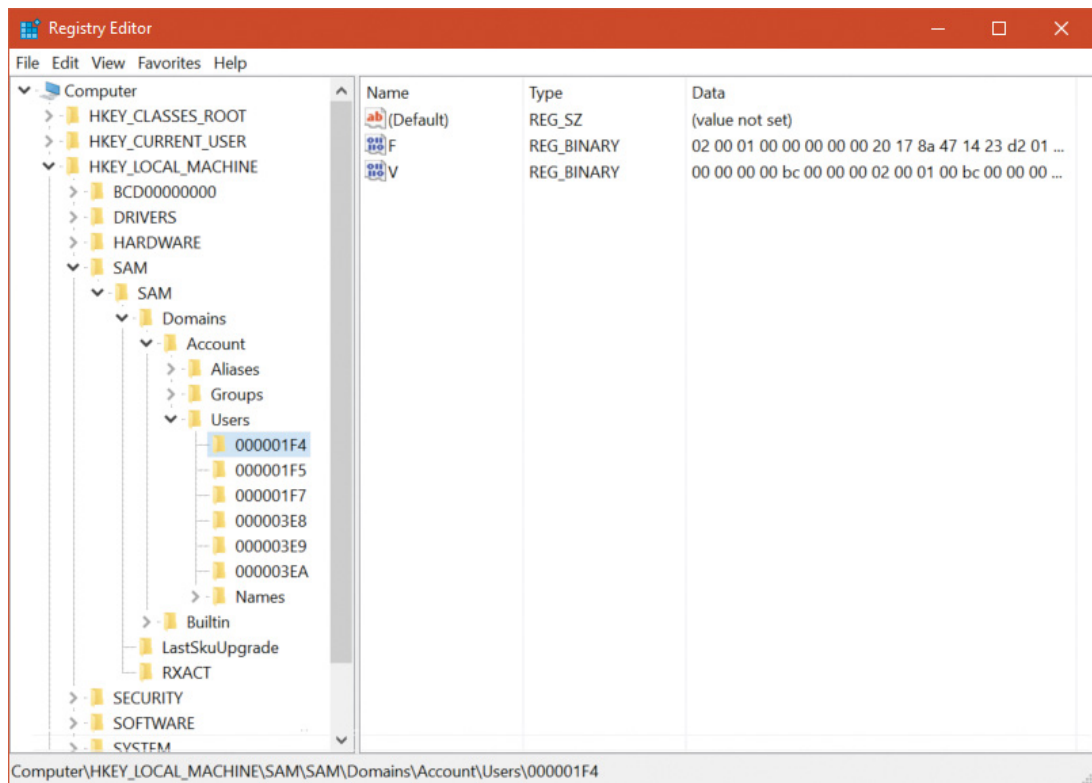
EXPERIMENT: LOOKING INSIDE HKLM\SAM AND HKLM\SECURITY

The security descriptors associated with the `SAM` and `Security` keys in the registry prevent access by any account other than the local system account. One way to gain access to these keys for exploration is to reset their security, but that can weaken the system's security. Another way is to execute `Regedit.exe` while running as the local system account. This can be done using the `Psexec` tool from Sysinternals with the `-s` option, as shown here:

[Click here to view code image](#)

```
C:\>psexec -s -i -d c:\windows\regedit.exe
```

The `-i` switch instructs `Psexec` to run the target executable under the interactive window station. Without it, the process would run in a non-interactive window station, on an invisible desktop. The `-d` switch just indicates `Psexec` should not wait until the target process exits.



The SRM, which runs in kernel mode, and Lsass, which runs in user mode, communicate using the ALPC facility described in Chapter 8, “System mechanisms,” in Part 2. During system initialization, the SRM

creates a port, named `SeRmCommandPort`, to which Lsass connects.

When the Lsass process starts, it creates an ALPC port named `SeLsaCommandPort`. The SRM connects to this port, resulting in the creation of private communication ports. The SRM creates a shared memory section for messages longer than 256 bytes, passing a handle in the connect call. Once the SRM and Lsass connect to each other during system initialization, they no longer listen on their respective connect ports. Therefore, a later user process has no way to connect successfully to either of these ports for malicious purposes. The connect request will never complete.

Virtualization-based security

It is common to refer to the kernel as *trusted*, due to its inherently higher level of privilege and isolation from user-mode applications. Yet, countless third-party drivers are written each month—Microsoft has stated that a million unique driver hashes are seen through telemetry, monthly! Each of these can contain any number of vulnerabilities, not to mention purposefully malicious kernel-mode code. In such a reality, the idea that the kernel is a small, protected component, and that user-mode applications are “safe” from attack, is clearly an unrealized ideal. This state of affairs leads to an inability to fully trust the kernel, and leaves key user-mode applications, which may contain highly private user data, open to compromise from other malicious user-mode applications (which exploit buggy kernel-mode components) or malicious kernel-mode programs.

As discussed in [Chapter 2, “System architecture,”](#) Windows 10 and Server 2016 include a virtualization-based security (VBS) architecture that enables an additional orthogonal level of trust: the virtual trust level (VTL). In this section, you will see how Credential Guard and Device Guard leverage VTLs to protect user data and provide an additional hardware-trust-based layer of security for digital code-signing purposes. At the end of this chapter, you will also see how Kernel Patch Protection (KPP) is provided through the PatchGuard component and enhanced by the VBS-powered HyperGuard technology.

As a reminder, normal user-mode and kernel code runs in VTL 0 and is unaware of the existence of VTL 1. This means anything placed at VTL 1 is hidden and inaccessible to VTL 0 code. If malware is able to penetrate the normal kernel, it still cannot gain access to anything stored in VTL 1, including even user-mode code running in VTL 1 (which is called Isolated User Mode). **Figure 7-2** shows the main VBS components we'll be looking at in this section:

- Hypervisor-Based Code Integrity (HVCI) and Kernel-Mode Code Integrity (KMCI), which power Device Guard
- LSA (Lsass.exe) and isolated LSA (LsaIso.exe), which power Credential Guard

Additionally, recall that the implementation of Trustlets, which run in IUM, was shown in **Chapter 3, “Process and jobs.”**

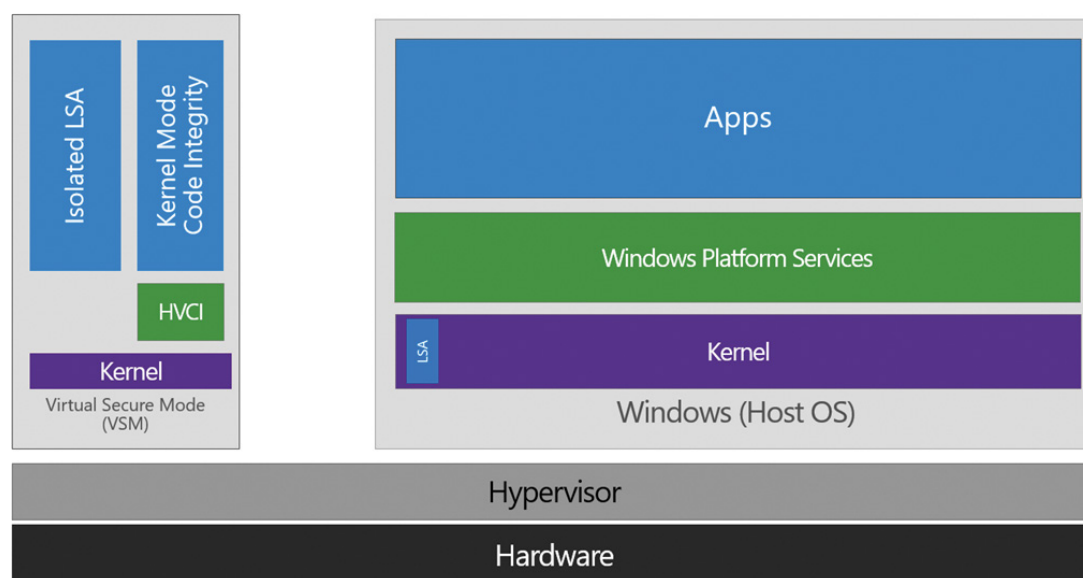


FIGURE 7-2 VBS components.

Of course, like any trusted component, VTL 1 also makes certain assumptions that the components it depends on can also be trusted. As such, VTL 1 requires Secure Boot (and thus, firmware) to function correctly, the hypervisor to not have been compromised, and hardware elements such as the IOMMU and Intel Management Engine to be free of VTL 0-accessible vulnerabilities. For more information on the hard-

ware chain of trust and boot-related security technologies, see Chapter 11, “Startup and shutdown,” in Part 2.

Credential Guard

To understand the security boundary and protection that Credential Guard provides, it is important to understand the various components that provide access to a user’s resources and data or login capabilities on a networked environment:

- **Password** This is the primary credential used by interactive users to identify themselves on the machine. This credential is used for authentication and to derive the other components of the credential model. It is the most highly sought after piece of a user’s identity.
- **NT one-way function (NT OWF)** This is a hash used by legacy components to identify the user (after a successful password logon) using the NT LAN Manager (NTLM) protocol. While modern networked systems no longer use NTLM to authenticate the user, many local components still do, as do some types of legacy network components (such as NTLM-based authenticating proxies). Because NTOWF is an MD4 hash, its algorithmic complexity in the face of today’s hardware, and its lack of anti-repeatability protection, means that intercepting the hash leads to instant compromise and even possible recovery of the password.
- **Ticket-granting ticket (TGT)** This is the equivalent of the NTOWF when a much more modern remote authentication mechanism is used: Kerberos. This is the default on Windows Active Directory-based domains and is enforced on Server 2016. The TGT and a corresponding key are provided to the local machine after a successful logon (just like the NTOWF on NTLM), and intercepting both components will result in instant compromise of the user’s credentials, although reuse and password recovery will not be possible.

Without Credential Guard enabled, some or all of these components of a user’s authentication credentials are present in the memory of Lsass.



To enable Credential Guard on Windows 10 Enterprise and Server 2016 editions, open the Group Policy editor (**gpedit.msc**), choose **Computer Configuration**, select **Administrative Templates**, choose **System**, choose **Device Guard**, and select **Turn on Virtualization Based Security**. In the top-left part of the dialog box that appears, select **Enabled**. Finally, select one of the **Enabled** options in the Credential Guard Configuration combo box.

Protecting the password

The password, encrypted with a local symmetric key, is stored to provide single sign-on (SSO) capabilities over protocols such as digest authentication (WDigest, used for HTTP-based authentication since Windows XP) or Terminal Services/RDP. As these protocols use plaintext authentication, the password must be kept in memory, which is then accessible through code injection, debugger, or other exploit techniques, and decrypted. Credential Guard cannot change the nature of these inherently unsafe protocols. Therefore, the only possible solution, which Credential Guard employs, is to disable SSO functionality for such protocols. This causes a loss of compatibility and forces the user to re-authenticate.

Obviously, a preferable solution is to remove the usage of a password completely, which Windows Hello, described in the “Windows Hello” section later in this chapter, allows. Authenticating with biometric credentials such as a user’s face or fingerprint removes the need to ever type a password, securing the interactive credential against hardware key loggers, kernel sniffing/hooks tools, and user mode-based spoofing applications. If the user never has a password to type, there is no password to steal. Another similar secure credential is the combination of a smart card and associated PIN. While a PIN may be stolen as its typed in, the smart card is a physical element whose key cannot be intercepted without a complex hardware-based attack. This is a type of

two-factor authentication (TFA), of which many other implementations exist.

Protecting the NTOWF/TGT key

Even with protected interactive credentials, a successful login results in a domain controller's key distribution center (KDC) returning the TGT and its key, as well as the NTOWF for legacy applications. Later, the user simply uses the NTOWF for accessing legacy resources and uses the TGT and its key to generate a service ticket. This can then be used to access remote resources (such as files on a share), as shown in **Figure 7-3**.

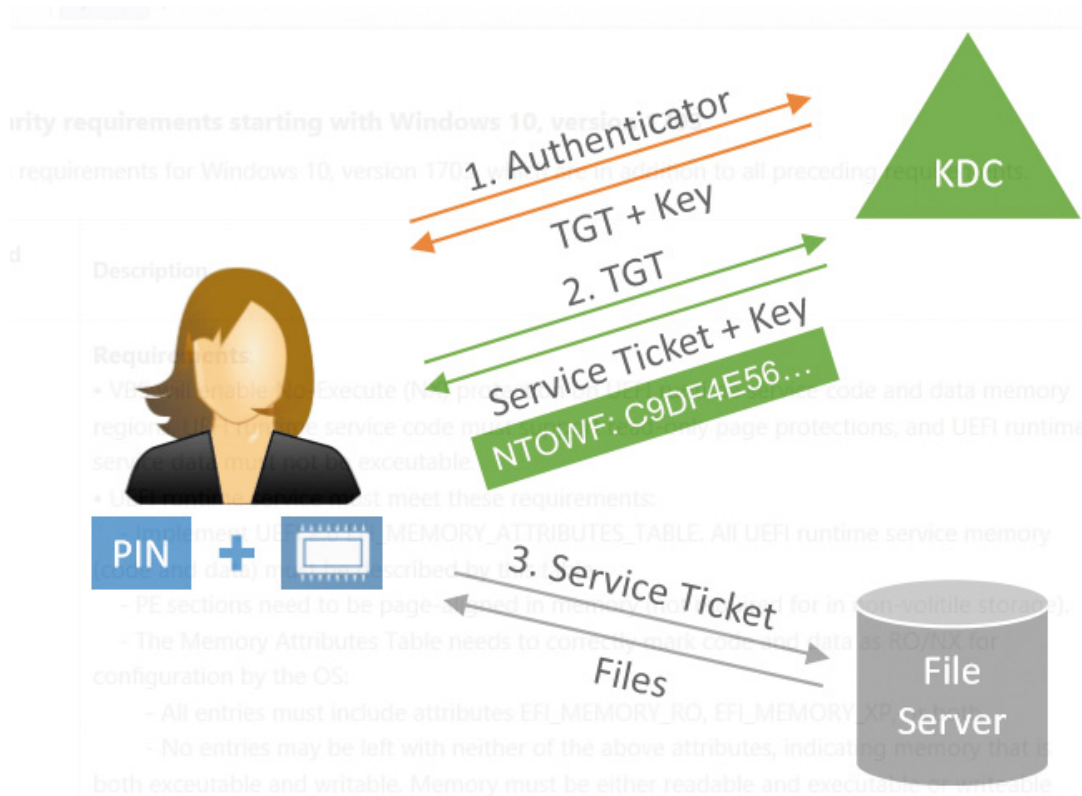


FIGURE 7-3 Accessing remote resources.

Thus, with either the NTOWF or the TGT and its key (stored in Lsass) in the attacker's hands, access to resources is possible even without the smart card, PIN, or user's face or fingerprint. Protecting Lsass from access by an attacker is thus one option that can be used, and which is possible using the Protected Process Light (PPL) architecture described in **Chapter 3**.

Lsass can be configured to run protected by setting the DWORD value `RunAsPPL` in the `HKLM\System\CurrentControlSet\Control\Lsa` registry key to `1`. (This is not a default option, as legitimate third-party authentication providers [DLLs] load and execute in the context of Lsass, which would not be possible if Lsass would run protected.)

Unfortunately, while this protection does guard the NTOWF and TGT key from user-mode attackers, it does not protect against kernel attackers or user-mode attackers that leverage vulnerabilities in any of the millions of drivers that are produced monthly. Credential Guard solves this problem by using another process, `Lsaiso.exe`, which runs as a Trustlet in VTL 1. This process therefore stores the user's secrets in its memory, not in Lsass.

Secure communication

As shown in [Chapter 2](#), VTL 1 has a minimal attack surface, as it does not have the full regular “NT” kernel, nor does it have any drivers or access to I/O of hardware of any kind. As such, isolated LSA, which is a VTL 1 Trustlet, cannot directly communicate with the KDC. This is still the responsibility of the Lsass process, which serves as a proxy and protocol implementer, communicating with the KDC to authenticate the user and to receive the TGT and the key and NTOWF, as well as communicating with the file server by using service ticket. This seemingly results in a problem: the TGT and its key/NTOWF transiently pass through Lsass during authentication, and the TGT and its key are somehow available to Lsass for the generation of service tickets. This leads to two questions: How does Lsass send and receive the secrets from isolated ISA, and how can we prevent an attacker from doing the same?

To answer the first question, recall that [Chapter 3](#), “[Processes and jobs](#),” described which services are available to Trustlets. One was the Advanced Local Procedure Call (ALPC), which the Secure Kernel supports by proxying the `NtAlpc` * calls to the Normal Kernel. Then, the Isolated User Mode environment implements support for the RPC runtime library (`Rpcrt4.dll`) over the ALPC protocol, which allows a VTL 0 and VTL 1 application to communicate using local RPC just like any other application and service. In [Figure 7-4](#), which shows Process

Explorer, you can see the LsaIso.exe process, which has a handle to the LSA_ISO_RPC_SERVER ALPC port. This is used to communicate with the Lsass.exe process. (See Chapter 8 in Part 2 for more information on ALPC.)

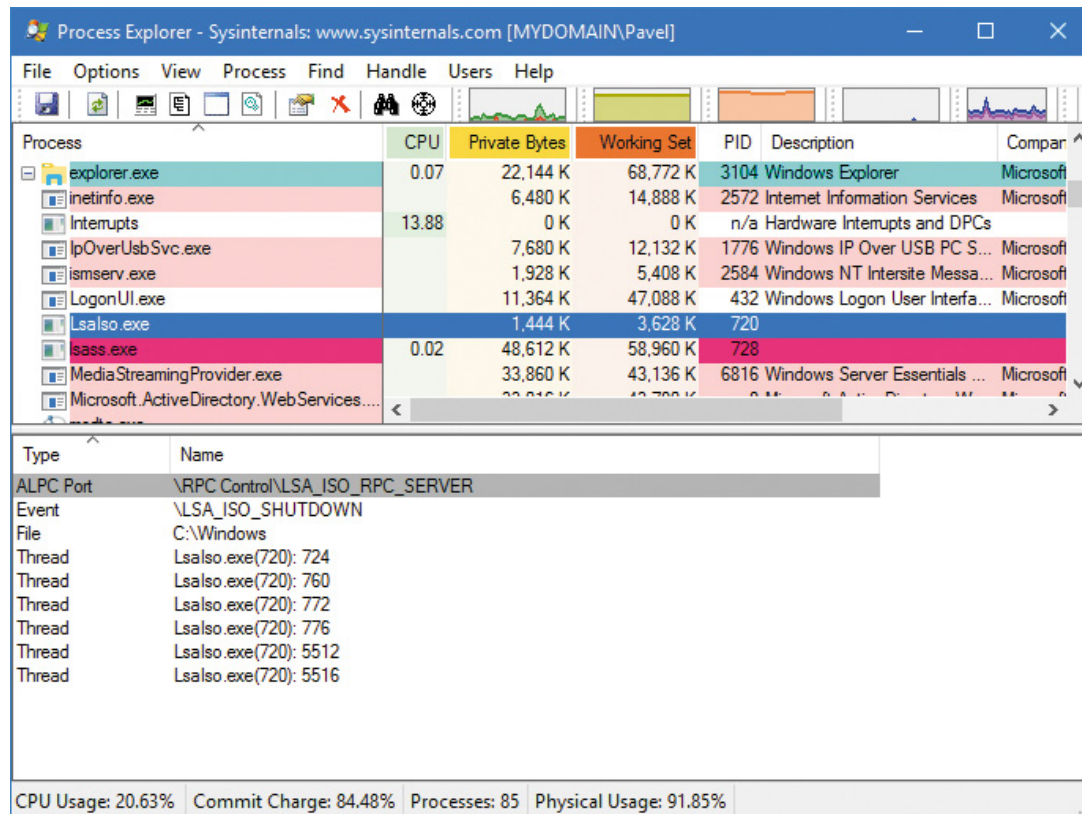


FIGURE 7-4 LsaIso.exe and its ALPC port.

To answer the second question, some understanding of cryptographic protocols and challenge/response models is required. If you're already familiar with some of the basic concepts of SSL/TLS technology and its use in Internet communications to prevent man-in-the-middle (MitM) attacks, you can think of the KDC and isolated LSA protocol in a similar way. Although Lsass sits in the middle as a proxy would, it only sees encrypted traffic between the KDC and isolated LSA, without the ability to understand its contents. Because isolated LSA establishes a local "session key," which only lives in VTL 1, and then uses a secure protocol to send this session key encrypted with yet another key, which only the KDC has, the KDC can then respond with the TGT and its key after encrypting it with the isolated LSA session key. Therefore, Lsass sees an encrypted message to the KDC (which it can't decrypt) and an encrypted message from the KDC (which it can't decrypt).

This model can even be used to protect legacy NTLM authentication, which is based on a challenge/response model. For example, when a user logs in with a plaintext credential, LSA sends it to isolated LSA, which then encrypts it with its session key and returns the encrypted credential to Lsass. Later, when an NTLM challenge/response is required, Lsass sends the NTLM challenge and the previously encrypted credentials to isolated LSA. At this point, only isolated LSA has the encryption key, so it decrypts the credentials and generates an NTLM response based on the challenge.

Note, however, that four possible attacks exist in this model:

- If the machine is already physically compromised, the plaintext password can be intercepted either as it is inputted or as it is sent to isolated LSA (if Lsass is already compromised). Using Windows Hello can mitigate against this.
- As mentioned, NTLM does not have anti-replay properties. Therefore, if the NTLM response is captured, it can be replayed for the same challenge. Alternatively, if the attacker can compromise Lsass after logon, it can capture the encrypted credential and force isolated LSA to generate new NTLM responses for arbitrary NTLM challenges. This attack, however, only works until reboot, because isolated LSA generates a new session key at that point.
- In the case of Kerberos logon, the NTOWF (which is not encrypted) can be intercepted and then reused, just like in a standard pass-the-hash attack. Again, however, this requires an already compromised machine (or physical network interception).
- The user, with physical access, may be able to disable Credential Guard. In this situation, the legacy authentication model is used (a so-called “downgrade attack”), and older attack models can now be employed.

UEFI lock

Because disabling Credential Guard (which is ultimately nothing more than a registry setting) is trivial for an attacker, Secure Boot and UEFI can be leveraged to prevent a non-physically present administrator (such as malware with admin rights) from disabling Credential Guard. This is done by enabling Credential Guard with UEFI Lock. In this mode, an EFI runtime variable is written to firmware memory and a reboot is required. At the reboot, the Windows boot loader, which still operates in EFI Boot Services mode, will write an EFI boot variable (which has the property of not being readable or writeable once EFI Boot Services mode is exited) to record the fact that Credential Guard is enabled. Additionally, a Boot Configuration Database (BCD) option will be recorded.

When the kernel boots, it will automatically rewrite the required Credential Guard registry key in the presence of the BCD option and/or UEFI runtime variable. If the BCD option is deleted by an attacker, BitLocker (if enabled) and TPM-based remote attestation (if enabled) will detect the change and require physical input of the admin's recovery key before booting, which will then restore the BCD option based on the UEFI runtime variable. If the UEFI runtime variable is deleted, the Windows boot loader will restore it based on the UEFI boot variable. As such, without special code to delete the UEFI boot variable—which can only be done in EFI Boot Services mode—there is no way to disable Credential Guard in UEFI lock mode.

The only such code that exists is in a special Microsoft binary called SecComp.efi. This must be downloaded by the administrator, who must then either boot the computer from an alternate EFI-based device and manually execute it (which will require the BitLocker recovery key as well as physical access) or modify the BCD (which will require the BitLocker recovery key). At the reboot, SecComp.efi will require user confirmation while in UEFI mode (which can only be done by a physical user).

Authentication policies and armored Kerberos

Using a security model of “secure, unless already compromised before logon or by a physical administrator” is definitely an improvement over the traditional non-Credential Guard–based security model. However, some enterprises and organizations may want an even stronger security guarantee: that even a compromised machine cannot be used to fake or replay a user’s credentials, and that if a user’s credentials *have* been compromised, they cannot be used outside of specific systems. By leveraging a Server 2016 feature called Authentication Policies, and armored Kerberos, Credential Guard can operate in this heightened security mode.

In this mode, the VTL 1 Secure Kernel will collect, using the TPM (a file on disk can also be used, but makes the security moot), a special machine ID key. This key is then used to generate a machine TGT key during the initial domain join operation as the machine is provisioned (obviously, it is important to ensure the machine is in a trusted state during provisioning), and this TGT key is sent to the KDC. Once configured, when the user logs in with his or her credential, it is combined with the machine’s credential (which only isolated LSA has access to), which forms a proof-of-origin key. The KDC will then reply with the NTOWF and user TGT and its key after encrypting it with the proof-of-origin key. In this mode, two security guarantees are provided:

- **The user is authenticating from a known machine** If the user, or an attacker, has the original credentials, and attempts to use them on a different machine, its TPM-based machine credential will be different.
- **The NTLM response/user ticket is coming from isolated LSA and has not been manually generated from Lsass** This guarantees that Credential Guard is enabled on the machine, even if the physical user can disable it in some way.

Unfortunately, once again, if the machine is compromised in such a way that the proof-of-origin-encrypted KDC response that contains the user TGT and its key is intercepted, it can be stored and used to request ses-

sion key-encrypted service tickets from isolated LSA. This can then be sent to a file server (for example) to access it until a reboot is issued to wipe the session key. As such, on a system with Credential Guard, it is recommended to reboot each time a user logs off. Otherwise, an attacker may be able to issue valid tickets even after the user is no longer present.

Future improvements

As discussed in [Chapter 2](#) and [Chapter 3](#), the Secure Kernel in VTL 1 is currently undergoing improvements to add support for specialized classes of PCI and USB hardware, which can exclusively be communicated with only through the hypervisor and VTL 1 code using the Secure Device Framework (SDF). Combined with BioIso.exe and FsIso.exe, which are new Trustlets to securely obtain biometric data and video frames (from a webcam), a VTL 0 kernel mode-based component cannot intercept the contents of a Windows Hello authentication attempt (which we've classified as safe compared to a user's plaintext password, but still technically capturable through custom driver-based interception). Once released, Windows Hello credentials will be guaranteed at the hardware level to not ever be available to VTL 0. In this mode, Lsass will not need to be involved in a Windows Hello authentication. Isolated LSA will obtain the credentials directly from the isolated biometrics or isolated frame service.



The Secure Driver Framework (SDF) is the WDF-equivalent for VTL 1 drivers. This framework is not currently public, but is shared with Microsoft partners only for creating VTL 1 drivers.

Device Guard

While Credential Guard is concerned with safeguarding the user's credentials, Device Guard has a completely different goal: protecting the user's machine itself from different kinds of software- and hardware-based attacks. Device Guard leverages the Windows Code Integrity services, such as Kernel-Mode Code Signing (KMCS) and User-Mode Code Integrity (UMCI), and strengthens them through HyperVisor Code Integrity (HVCI). (See Chapter 8 in Part 2 for more information on Code Integrity.)

Additionally, Device Guard is fully configurable, thanks to Custom Code Integrity (CCI) and signing policies that are protected by Secure Boot and defined by the enterprise administrator. These policies, which are explained in Chapter 8, allow the enforcement of inclusion/exclusion lists that are based on cryptographically sound information (such as certificate signers or SHA-2 hashes) instead of file paths or file names as with AppLocker's policies. (See the section “**AppLocker**” later in this chapter for more on AppLocker.)

Therefore, while we won't describe here the different ways in which Code Integrity policies can be defined and customized, we will show how Device Guard enforces whatever these policies may be set to, through the following guarantees:

- **If kernel-mode code signing is enforced, only signed code can load, regardless of the kernel itself being compromised** This is because the kernel-loading process will notify the Secure Kernel in VTL 1

whenever it loads a driver, and only successfully load it once HVCI has validated its signature.

■ **If kernel-mode code signing is enforced, signed code cannot be modified once loaded, even by the kernel itself** This is because the executable code pages will be marked as read-only through the hypervisor's Second Level Address Translation (SLAT) mechanism, which is further explained in Chapter 8 in Part 2.

■ **If kernel-mode code signing is enforced, dynamically allocated code is prohibited (a tautology of the first two bullets)** This is because the kernel does not have the ability to allocate executable entries in the SLAT page table entries, even though the kernel's page tables themselves may mark such code as executable.

■ **If kernel-mode code signing is enforced, UEFI runtime code cannot be modified, even by other UEFI runtime code or by the kernel itself** Additionally, Secure Boot should already have validated that this code was signed at the time it was loaded. (Device Guard relies on this assumption.) Furthermore, UEFI runtime data cannot be made executable. This is done by reading all the UEFI runtime code and data, enforcing the correct permissions, and duplicating them in the SLAT page table entries, which are protected in VTL 1.

■ **If kernel-mode code signing is enforced, only kernel-mode (ring 0) signed code can execute** This may once again sound like a tautology of the first three bullets, but consider signed ring 3 code. Such code is valid from UMCI's perspective and has been authorized as executable code in the SLAT page table entries. The Secure Kernel relies on the Mode-Based Execution Control (MBEC) feature, if present in hardware, which enhances the SLAT with a user/kernel executable bit, or the hypervisor's software emulation of this feature, called Restricted User Mode (RUM).

■ **If user-mode code signing is enforced, only signed user-mode images can be loaded** This means all executable processes must be signed

(.exe) files as well as the libraries they load (.dll).

■ **If user-mode code signing is enforced, the kernel does not allow user-mode applications to make existing executable code pages writable** Obviously, it is impossible for user-mode code to allocate executable memory or to modify existing memory without asking the kernel permission. As such, the kernel can apply its usual enforcement rules. But even in the case of a compromised kernel, the SLAT ensures that no user-mode pages will be executable without the Secure Kernel's knowledge and approval, and that such executable pages can never be writable.

■ **If user-mode code signing is enforced, and hard code guarantees are requested by the signing policy, dynamically allocated code is prohibited** This is an important distinction from the kernel scenarios. By default, signed user-mode code is allowed to allocate additional executable memory to support JIT scenarios unless a special enhanced key usage (EKU) is present in the application's certificate, which serves as a dynamic code generation entitlement. At present, NGEN.EXE (.NET Native Image Generation) has this EKU, which allows IL-only .NET executables to function even in this mode.

■ **If user-mode PowerShell constrained language mode is enforced, all PowerShell scripts that use dynamic types, reflection, or other language features that allow the execution of arbitrary code and/or marshalling to Windows/.NET API functions must also be signed** This prevents possibly malicious PowerShell scripts from escaping constrained mode.

SLAT page table entries are protected in VTL 1 and contain the "ground truth" for what permissions a given page of memory can have. By withholding the executable bit as needed, and/or withholding the writable bit from existing executable pages (a security model known as W^X, pronounced *double-you xor ex*), Device Guard moves all code-signing enforcement into VTL 1 (in a library called SKCI.DLL, or Secure Kernel Code Integrity).

Additionally, even if not configured explicitly on the machine, Device Guard operates in a third mode if Credential Guard is enabled by enforcing that all Trustlets have a specific Microsoft signature with a certificate that includes the Isolated User Mode EKU. Otherwise, an attacker with ring 0 privileges could attack the regular KMCS mechanism and load a malicious Trustlet to attack the isolated LSA component. Furthermore, all user-mode code-signing enforcements are active for the Trustlet, which executes in hard code guarantees mode.

Finally, as a performance optimization, it is important to understand that the HVCI mechanism will not reauthenticate every single page when the system resumes from hibernation (S4 sleep state). In some cases, the certificate data may not even be available. Even if this were the case, the SLAT data must be reconstructed, which means that the SLAT page table entries are stored in the hibernation file itself. As such, the hypervisor needs to trust the hibernation file has not been modified in any way. This is done by encrypting the hibernation file with a local machine key that is stored in the TPM. Unfortunately, without a TPM present, this key must be stored in a UEFI runtime variable, which allows a local attacker to decrypt the hibernation file, modify it, and re-encrypt it.

Protecting objects

Object protection and access logging are the essence of discretionary access control and auditing. The objects that can be protected on Windows include files, devices, mailslots, pipes (named and anonymous), jobs, processes, threads, events, keyed events, event pairs, mutexes, semaphores, shared memory sections, I/O completion ports, LPC ports, waitable timers, access tokens, volumes, window stations, desktops, network shares, services, registry keys, printers, Active Directory objects, and so on—theoretically, anything managed by the executive object manager. In practice, objects that are not exposed to user mode (such as driver objects) are usually not protected. Kernel-mode code is trusted and usually uses interfaces to the object manager that do not perform access checking. Because system resources that are exported to user mode (and hence require security validation) are implemented as objects in kernel mode, the Windows object manager plays a key role in enforcing object security.

You can view object protection with the WinObj Sysinternals tool (for named objects), shown in [Figure 7-5](#). [Figure 7-6](#) shows the Security property page of a section object in the user's session. Although files are the resources most commonly associated with object protection, Windows uses the same security model and mechanism for executive objects as it does for files in the file system. As far as access controls are concerned, executive objects differ from files only in the access methods supported by each type of object.

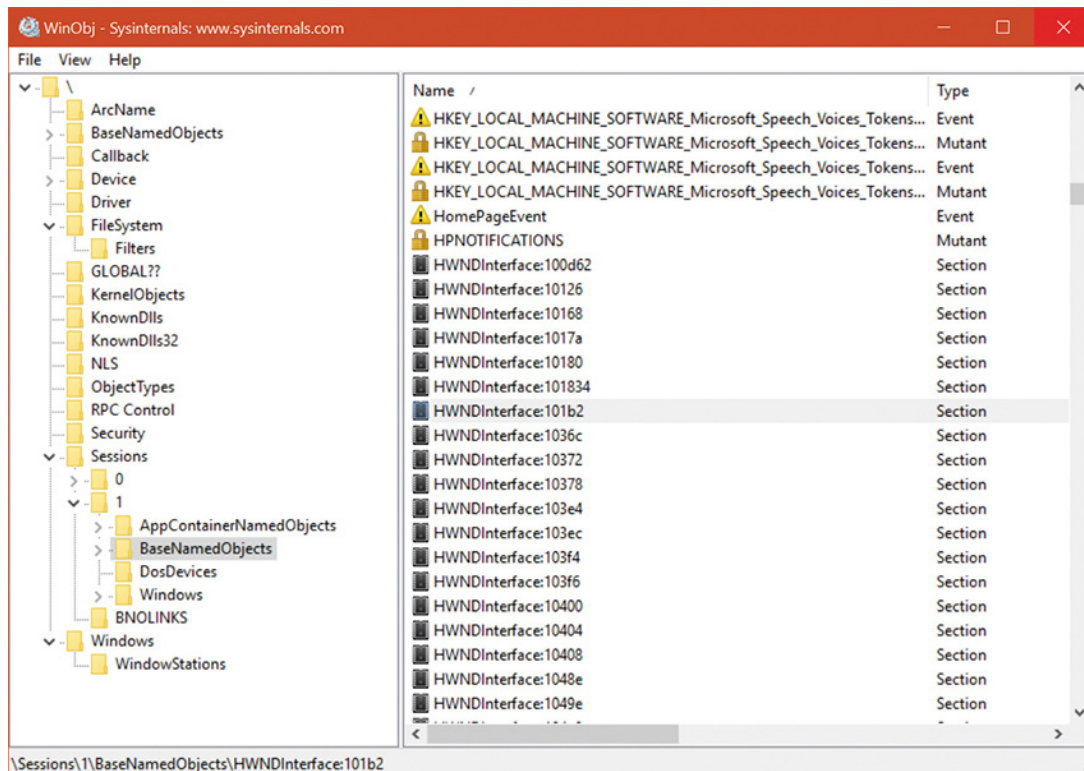


FIGURE 7-5 WinObj with a section object selected.

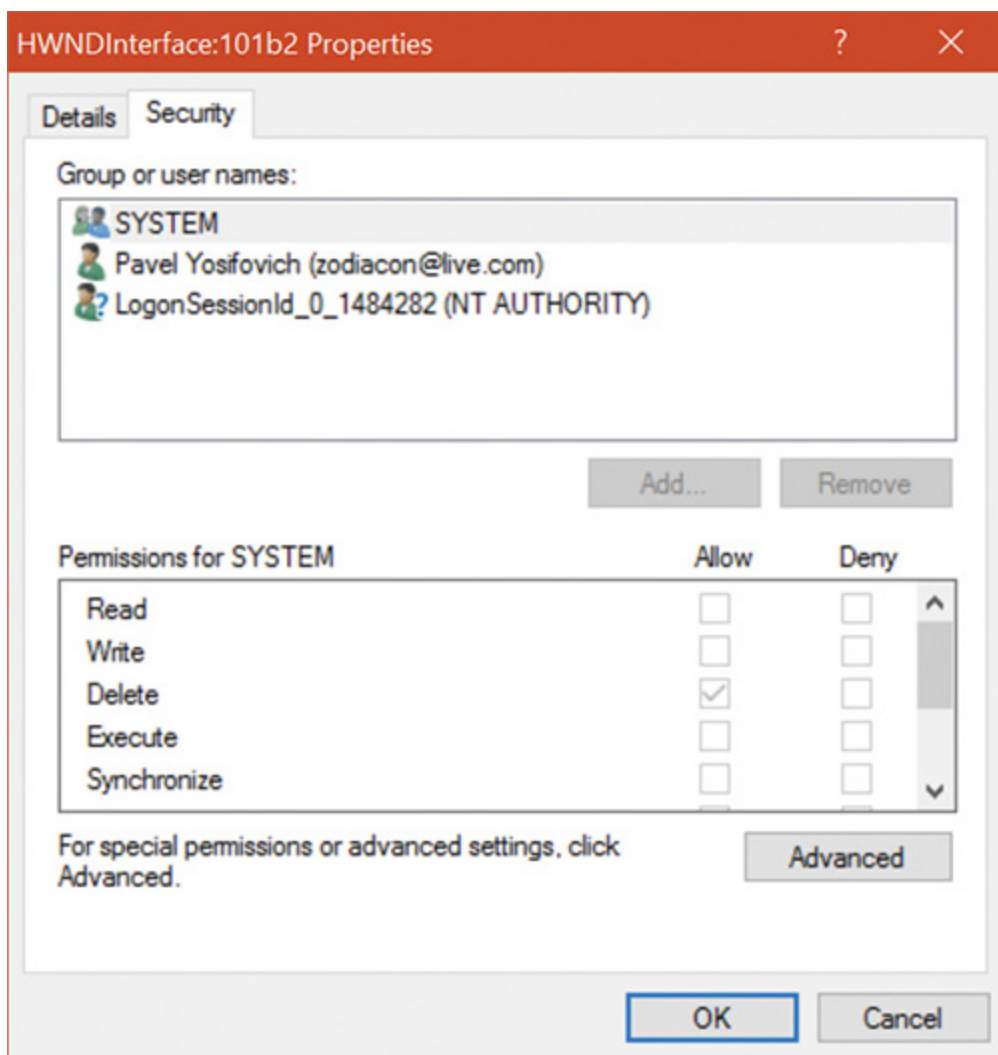


FIGURE 7-6 An executive object and its security descriptor, viewed by WinObj.

What is shown in **Figure 7-6** is actually the object's discretionary access control list (DACL). We will describe DACLs in detail in the section **"Security descriptors and access control."**

You can use Process Explorer to view the security properties of objects by double-clicking a handle in the lower pane view (when configured to show handles). This has the added benefit of displaying objects that are unnamed. The Property page shown is the same in both tools, as the page itself is provided by Windows.

To control who can manipulate an object, the security system must first be sure of each user's identity. This need to guarantee the user's identity is the reason that Windows requires authenticated logon before accessing any system resources. When a process requests a handle to an object, the object manager and the security system use the caller's security identification and the object's security descriptor to determine whether the caller should be assigned a handle that grants the process access to the object it desires.

As discussed later in this chapter, a thread can assume a different security context than that of its process. This mechanism is called *impersonation*. When a thread is impersonating, security validation mechanisms use the thread's security context instead of that of the thread's process. When a thread isn't impersonating, security validation falls back on using the security context of the thread's owning process. It's important to keep in mind that all the threads in a process share the same handle table, so when a thread opens an object—even if it's impersonating—all the threads of the process have access to the object.

Sometimes, validating the identity of a user isn't enough for the system to grant access to a resource that should be accessible by the account. Logically, one can think of a clear distinction between a service running under the Alice account and an unknown application that Alice downloaded while browsing the Internet. Windows achieves this kind of intra-user isolation with the Windows integrity mechanism, which implements integrity levels. The Windows integrity mechanism is used by

User Account Control (UAC) elevations, User Interface Privilege Isolation (UIPI) and AppContainers, all described later in this chapter.

Access checks

The Windows security model requires that a thread specify up front, at the time that it opens an object, what types of actions it wants to perform on the object. The object manager calls the SRM to perform access checks based on a thread's desired access. If the access is granted, a handle is assigned to the thread's process with which the thread (or other threads in the process) can perform further operations on the object.

One event that causes the object manager to perform security access validation is when a thread opens an existing object using a name. When an object is opened by name, the object manager performs a lookup of the specified object in the object manager namespace. If the object isn't located in a secondary namespace, such as the configuration manager's registry namespace or a file system driver's file system namespace, the object manager calls the internal function

`ObpCreateHandle` once it locates the object. As its name implies, `ObpCreateHandle` creates an entry in the process handle table that becomes associated with the object. `ObpCreateHandle` first calls `ObpGrantAccess` to see if the thread has permission to access the object. If so, `ObpCreateHandle` calls the executive function `ExCreateHandle` to create the entry in the process handle table. `ObpGrantAccess` calls `ObCheckObjectAccess` to initiate the security access check.

`ObpGrantAccess` passes to `ObCheckObjectAccess` the security credentials of the thread opening the object, the types of access to the object that the thread is requesting (read, write, delete, and so forth, including object-specific operations), and a pointer to the object.

`ObCheckObjectAccess` first locks the object's security descriptor and the security context of the thread. The object security lock prevents another thread in the system from changing the object's security while the access check is in progress. The lock on the thread's security context

prevents another thread (from that process or a different process) from altering the security identity of the thread while security validation is in progress. `ObCheckObjectAccess` then calls the object's security method to obtain the security settings of the object. (See Chapter 8 in Part 2 for a description of object methods.) The call to the security method might invoke a function in a different executive component. However, many executive objects rely on the system's default security management support.

When an executive component defining an object doesn't want to override the SRM's default security policy, it marks the object type as having default security. Whenever the SRM calls an object's security method, it first checks to see whether the object has default security. An object with default security stores its security information in its header, and its security method is `SeDefaultObjectMethod`. An object that doesn't rely on default security must manage its own security information and supply a specific security method. Objects that rely on default security include mutexes, events, and semaphores. A file object is an example of an object that overrides default security. The I/O manager, which defines the file object type, has the file system driver on which a file resides manage (or choose not to implement) the security for its files. Thus, when the system queries the security on a file object that represents a file on an NTFS volume, the I/O manager file object security method retrieves the file's security using the NTFS file system driver. Note, however, that `ObCheckObjectAccess` isn't executed when files are opened because they reside in secondary namespaces. The system invokes a file object's security method only when a thread explicitly queries or sets the security on a file (with the Windows `SetFileSecurity` or `GetFileSecurity` functions, for example).

After obtaining an object's security information, `ObCheckObjectAccess` invokes the SRM function `SeAccessCheck`. `SeAccessCheck` is one of the functions at the heart of the Windows security model. Among the input parameters `SeAccessCheck` accepts are the object's security information, the security identity of the thread as captured by `ObCheckObjectAccess`, and the access that the thread is requesting.

`SeAccessCheck` returns `true` or `false`, depending on whether the thread is granted the access it requested to the object.

Here is an example: Suppose a thread wants to know when a specific process exits (or terminates in some way). It needs to get a handle to the target process by calling the `OpenProcess` API, passing in two important arguments: the unique process ID (let's assume it's known or has been obtained in some way) and an access mask indicating the operations that the thread wants to perform using the returned handle. Lazy developers may just pass `PROCESS_ALL_ACCESS` for the access mask, specifying they want all possible access rights for the process. One of the following two results would occur:

- If the calling thread can be granted all the permissions, it would get back a valid handle and then could call `WaitForSingleObject` to wait for the process to exit. However, another thread in the process, perhaps with fewer privileges, can use the same handle to do other operations with the process, such as terminate it prematurely with `TerminateProcess`, because the handle allows all possible operations on the process.
- The call can fail if the calling thread does not have sufficient privileges to be granted all possible access and the result is an invalid handle, meaning no access to the process. This is unfortunate, because the thread just needed to ask for the `SYNCHRONIZE` access mask. That has a much better chance of succeeding than asking for `PROCESS_ALL_ACCESS`.

The simple conclusion here is that a thread should request the exact access it requires—no more, no less.

Another event that causes the object manager to execute access validation is when a process references an object using an existing handle. Such references often occur indirectly, as when a process calls on a Windows API to manipulate an object and passes an object handle. For example, a thread opening a file can request read permission to the file. If the thread has permission to access the object in this way, as dictated

by its security context and the security settings of the file, the object manager creates a handle—representing the file—in the handle table of the thread’s process. The types of accesses the threads in the process are granted through the handle are stored with the handle by the object manager.

Subsequently, the thread could attempt to write to the file using the `WriteFile` Windows function, passing the file’s handle as a parameter. The system service `NtWriteFile`, which `WriteFile` calls via `Ntdll.dll`, uses the object manager function `ObReferenceObjectByHandle` (documented in the WDK) to obtain a pointer to the file object from the handle. `ObReferenceObjectByHandle` accepts the access that the caller wants from the object as a parameter. After finding the handle entry in the process handle table, `ObReferenceObjectByHandle` compares the access being requested with the access granted at the time the file was opened. In this example, `ObReferenceObjectByHandle` will indicate that the write operation should fail because the caller didn’t obtain write access when the file was opened.

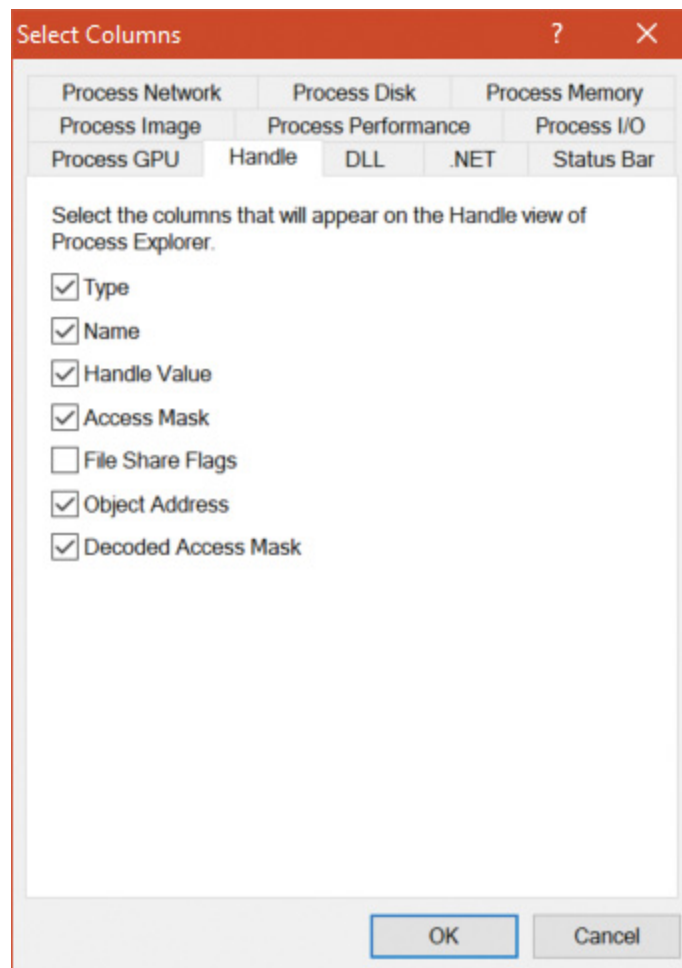
The Windows security functions also enable Windows applications to define their own private objects and to call on the services of the SRM (through the AuthZ user-mode APIs, described later) to enforce the Windows security model on those objects. Many kernel-mode functions that the object manager and other executive components use to protect their own objects are exported as Windows user-mode APIs. The user-mode equivalent of `SeAccessCheck` is the AuthZ API `AccessCheck`. Windows applications can therefore leverage the flexibility of the security model and transparently integrate with the authentication and administrative interfaces that are present in Windows.

The essence of the SRM’s security model is an equation that takes three inputs: the security identity of a thread, the access that the thread wants to an object, and the security settings of the object. The output is either yes or no and indicates whether the security model grants the thread the access it desires. The following sections describe the inputs in more detail and then document the model’s access-validation algorithm.

EXPERIMENT: VIEWING HANDLE ACCESS MASKS

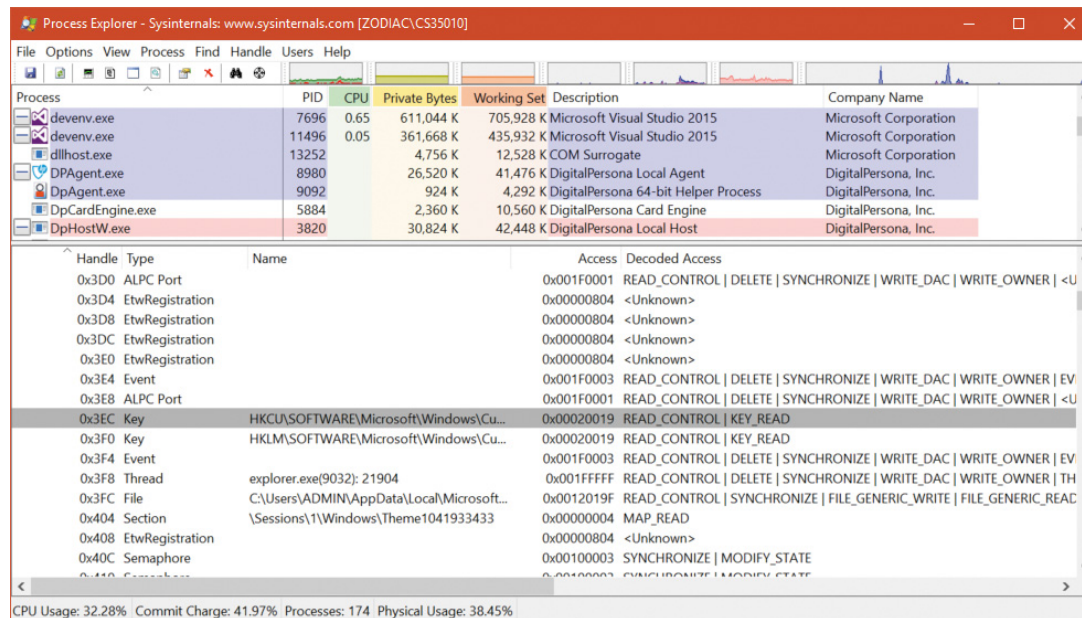
Process Explorer can show the access mask associated with open handles. Follow these steps:

1. Open Process Explorer.
2. Open the **View** menu, choose **Lower Pane View**, and select **Handles** to configure the lower pane to show handles.
3. Right-click the column headers of the lower pane and choose **Select Columns** to open the dialog box shown here:



4. Select the **Access Mask** and **Decoded Access Mask** check boxes (the latter is available in version 16.10 and later) and click **OK**.
5. Select **Explorer.exe** from the process list and look at the lower pane handles. Each handle has an access mask, indicating the access granted using this handle. To help with interpreting the bits of the access mask,

the decoded access mask column shows a textual representation of the access masks for many types of objects:



Notice there are generic access rights (for example, `READ_CONTROL` and `SYNCHRONIZE`) and specific ones (for example, `KEY_READ` and `MODIFY_STATE`). Most of the specific ones are shortened versions of the actual defines in the windows headers (for example, `MODIFY_STATE` instead of `EVENT_MODIFY_STATE`, `TERMINATE` instead of `PROCESS_TERMINATE`).

Security identifiers

Instead of using names (which might or might not be unique) to identify entities that perform actions in a system, Windows uses security identifiers (SIDs). Users have SIDs, as do local and domain groups, local computers, domains, domain members, and services. A SID is a variable-length numeric value that consists of a SID structure revision number, a 48-bit identifier authority value, and a variable number of 32-bit subauthority or relative identifier (RID) values. The authority value identifies the agent that issued the SID, and this agent is typically a Windows local system or a domain. Subauthority values identify trustees relative to the issuing authority, and RIDs are simply a way for Windows to create unique SIDs based on a common base SID. Because SIDs are long and Windows takes care to generate truly random values

within each SID, it is virtually impossible for Windows to issue the same SID twice on machines or domains anywhere in the world. When displayed textually, each SID carries an S prefix, and its various components are separated with hyphens like so:

S-1-5-21-1463437245-1224812800-863842198-1128

In this SID, the revision number is 1, the identifier authority value is 5 (the Windows security authority), and four subauthority values plus one RID (1128) make up the remainder of the SID. This SID is a domain SID, but a local computer on the domain would have a SID with the same revision number, identifier authority value, and number of subauthority values.

When you install Windows, the Windows Setup program issues the computer a machine SID. Windows assigns SIDs to local accounts on the computer. Each local-account SID is based on the source computer's SID and has a RID at the end. RIDs for user accounts and groups start at 1000 and increase in increments of 1 for each new user or group. Similarly, Domain Controller Promote (Dcpromo.exe), the utility used to create a new Windows domain, reuses the computer SID of the computer being promoted to domain controller as the domain SID and re-creates a new SID for the computer if it is ever demoted. Windows issues to new domain accounts SIDs that are based on the domain SID and have an appended RID (again starting at 1000 and increasing in increments of 1 for each new user or group). A RID of 1028 indicates that the SID is the twenty-ninth SID the domain issued.

Windows issues SIDs that consist of a computer or domain SID with a predefined RID to many predefined accounts and groups. For example, the RID for the Administrator account is 500, and the RID for the guest account is 501. A computer's local Administrator account, for example, has the computer SID as its base with the RID of 500 appended to it:

S-1-5-21-13124455-12541255-61235125-500

Windows also defines a number of built-in local and domain SIDs to represent well-known groups. For example, a SID that identifies any and all accounts (except anonymous users) is the Everyone SID: S-1-1-0. Another example of a group that a SID can represent is the Network group, which is the group that represents users who have logged on to a machine from the network. The Network group SID is S-1-5-2. **Table 7-2**, reproduced here from the Windows SDK documentation, shows some basic well-known SIDs, their numeric values, and their use. Unlike users' SIDs, these SIDs are predefined constants, and have the same values on every Windows system and domain in the world. Thus, a file that is accessible by members of the Everyone group on the system where it was created is also accessible to Everyone on any other system or domain to which the hard drive where it resides happens to be moved. Users on those systems must, of course, authenticate to an account on those systems before becoming members of the Everyone group.

SID	Name	Use
S-1-0-0	Nobody	Used when the SID is unknown
S-1-1-0	Everyone	A group that includes all users except anonymous users
S-1-2-0	Local	Users who log on to terminals locally (physically) connected to the system
S-1-3-0	Creator Owner ID	A security identifier to be replaced by the security identifier of the user who created a new object (used in inheritable ACEs)
S-1-3-1	Creator Group ID	Identifies a security identifier to be replaced by the Primary group SID of the user who created a new object (used in inheritable ACEs)
S-1-5-18	Local System account	Used by services
S-1-5-19	Local Service account	Used by services
S-1-5-20	Network Service account	Used by services

TABLE 7-2 A few well-known SIDs



NOTE

See Microsoft Knowledge Base article 243330 for a list of defined SIDs at <http://support.microsoft.com/kb/243330>.

Finally, Winlogon creates a unique logon SID for each interactive logon session. A typical use of a logon SID is in an access control entry (ACE) that allows access for the duration of a client's logon session. For example, a Windows service can use the `LogonUser` function to start a new

logon session. The `LogonUser` function returns an access token from which the service can extract the logon SID. The service can then use the SID in an ACE (described in the section “[**Security descriptors and access control**](#)” later in this chapter) that allows the client’s logon session to access the interactive window station and desktop. The SID for a logon session is S-1-5-5-X-Y, where the X and Y are randomly generated.

You can easily see the SID representation for any account you're using by running the `PsGetSid` utility from Sysinternals. `PsGetSid`'s options allow you to translate machine and user account names to their corresponding SIDs and vice versa.

If you run `PsGetSid` with no options, it prints the SID assigned to the local computer. Because the Administrator account always has a RID of 500, you can determine the name assigned to the account (in cases where a system administrator has renamed the account for security reasons) simply by passing the machine SID appended with -500 as `PsGetSid`'s command-line argument.

To obtain the SID of a domain account, enter the user name with the domain as a prefix:

```
c:\>psgetsid redmond\johndoe
```

You can determine the SID of a domain by specifying the domain's name as the argument to `PsGetSid`:

```
c:\>psgetsid Redmond
```

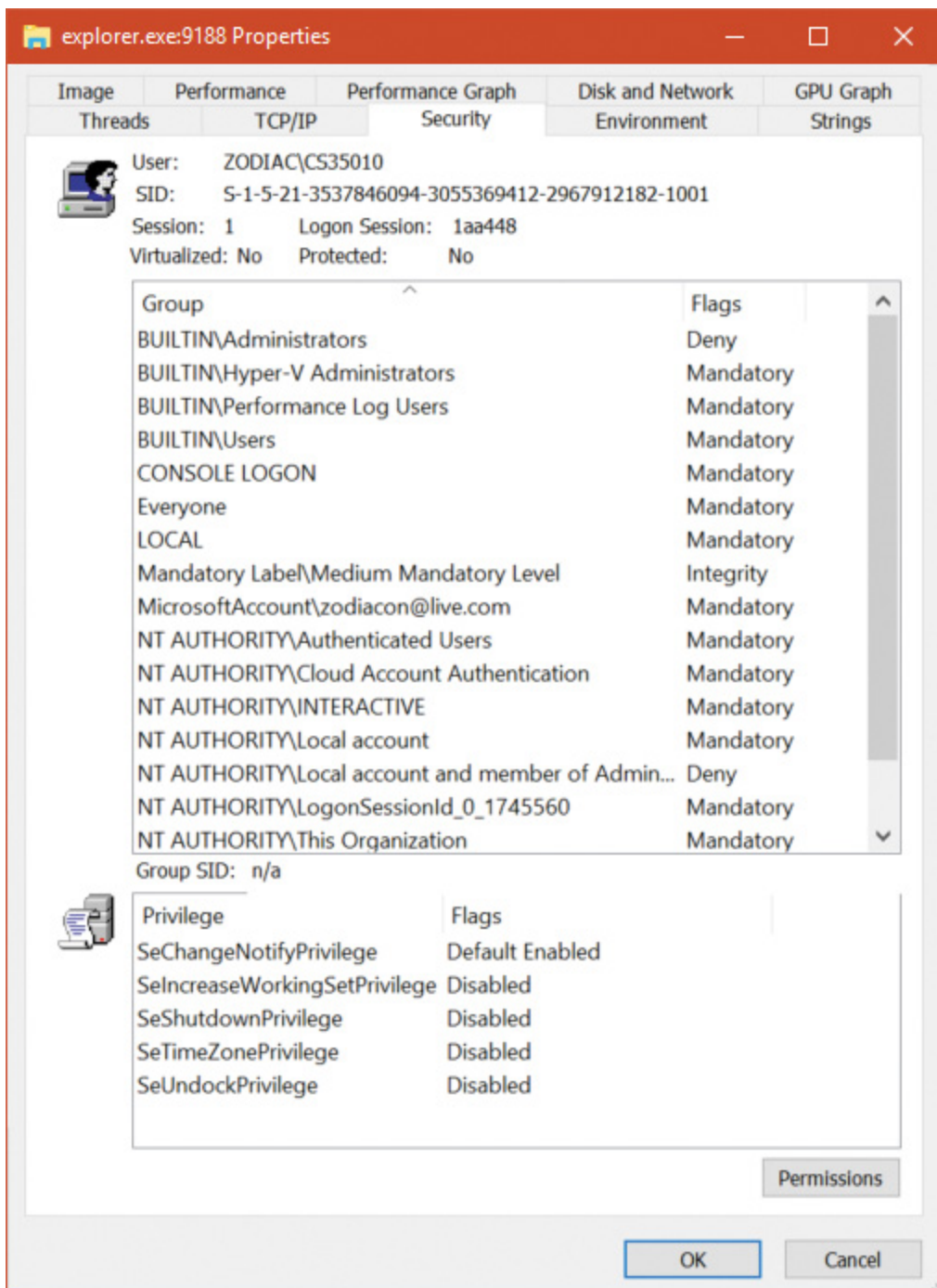
Finally, by examining the RID of your own account, you know at least a number of security accounts (equal to the number resulting from subtracting 999 from your RID) have been created in your domain or on your local machine (depending on whether you are using a domain or local machine account). You can determine what accounts have been assigned RIDs by passing a SID with the RID you want to query to `PsGetSid`. If `PsGetSid` reports that no mapping between the SID and an account name was possible and the RID is lower than that of your account, you know that the account assigned the RID has been deleted.

For example, to find out the name of the account assigned the 28th RID, pass the domain SID appended with -1027 to `PsGetSid`:

[Click here to view code image](#)

```
c:\>psgetsid S-1-5-21-1787744166-3910675280-2727264193-1027
Account for S-1-5-21-1787744166-3910675280-2727264193-1027:
User: redmond\johndoe
```

Process Explorer can also show you information on account and group SIDs on your system through its Security tab. This tab shows you information such as who owns this process and which groups the account is a member of. To view this information, simply double-click any process (for example, Explorer.exe) in the Process list and then click the **Security** tab. You should see something similar to the following:



The information displayed in the User field contains the friendly name of the account owning this process, while the SID field contains the actual SID value. The Group list includes information on all the groups that this account is a member of (groups are described later in this chapter).

Integrity levels

As mentioned, integrity levels can override discretionary access to differentiate a process and objects running as and owned by the same

user, offering the ability to isolate code and data within a user account. The mechanism of Mandatory Integrity Control (MIC) allows the SRM to have more detailed information about the nature of the caller by associating it with an integrity level. It also provides information on the trust required to access the object by defining an integrity level for it.

The integrity level of a token can be obtained with the `GetTokenInformation` API with the `Token-IntegrityLevel` enumeration value. These integrity levels are specified by a SID. Although integrity levels can be arbitrary values, the system uses six primary levels to separate privilege levels, as described in [Table 7-3](#).

SID	Name (Level)	Use
S-1-16-0x0	Untrusted (0)	Used by processes started by the Anonymous group. It blocks most write access.
S-1-16-0x1000	Low (1)	Used by AppContainer processes (UWP) and Protected Mode Internet Explorer. It blocks write access to most objects (such as files and registry keys) on the system.
S-1-16-0x2000	Medium (2)	Used by normal applications being launched while UAC is enabled.
S-1-16-0x3000	High (3)	Used by administrative applications launched through elevation when UAC is enabled, or normal applications if UAC is disabled and the user is an administrator.
S-1-16-0x4000	System (4)	Used by services and other system-level processes (such as Wininit, Winlogon, Smss, and so on).
S-1-16-0x5000	Protected (5)	Currently unused by default. Can be set by kernel-mode caller only.

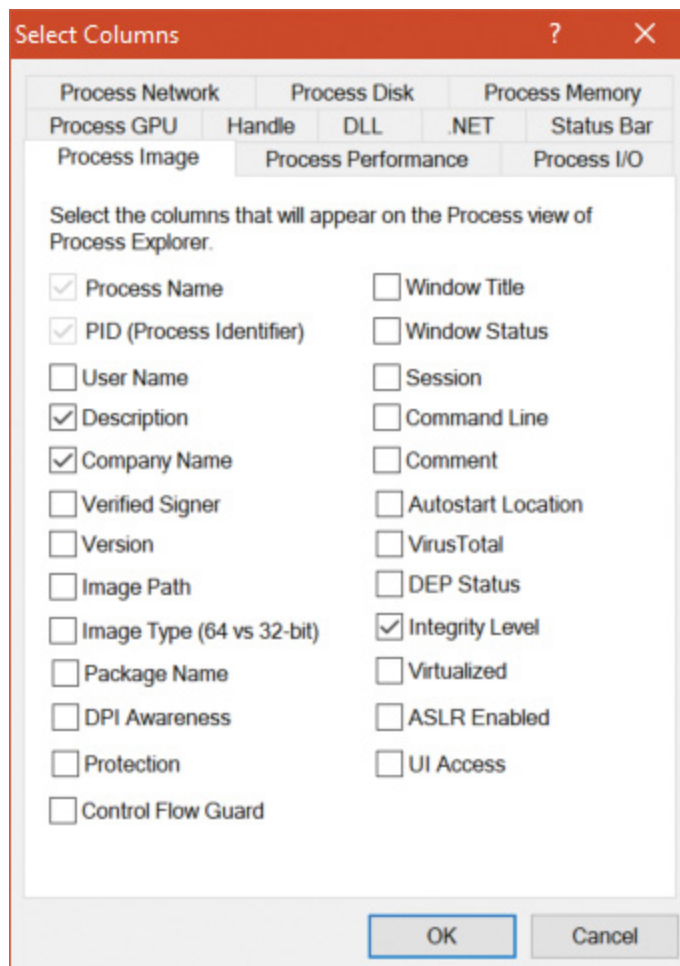
TABLE 7-3 Integrity level SIDs

Another, seemingly additional, integrity level is called *AppContainer*, used by UWP apps. Although seemingly another level, it's in fact equal to Low. UWP process tokens have another attribute that indicates they are running inside an AppContainer (described in the “[AppContainers](#)” section). This information is available with the `GetTokenInformation` API with the `TokenIsAppContainer` enumeration value.

EXPERIMENT: LOOKING AT THE INTEGRITY LEVEL OF PROCESSES

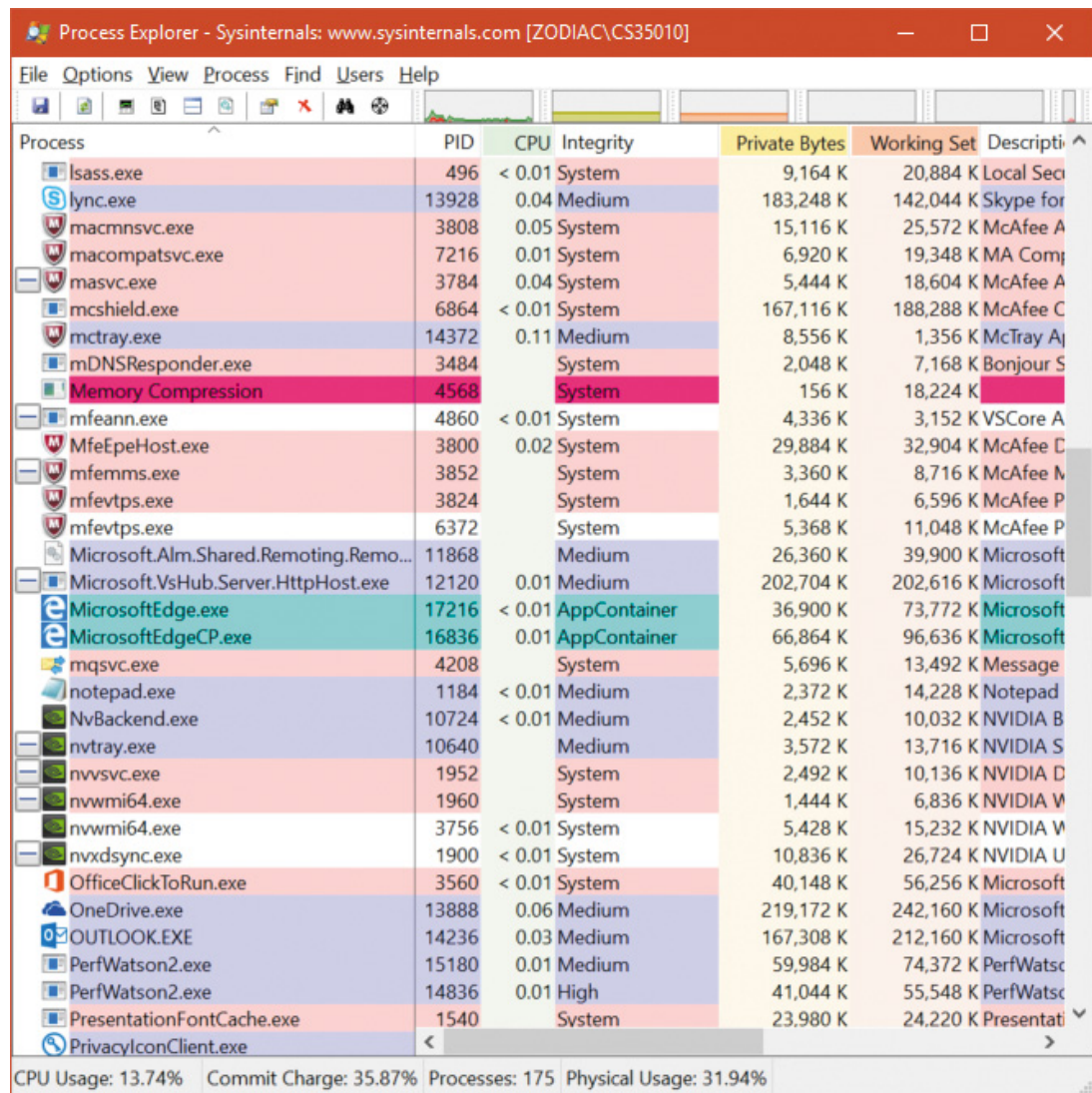
You can use Process Explorer to quickly display the integrity level for the processes on your system. The following steps demonstrate this functionality.

1. Launch Microsoft Edge browser and Calc.exe (Windows 10).
2. Open an elevated command prompt window.
3. Open Notepad normally (without elevating it).
4. Open an elevated Process Explorer, right-click any column header in the Process list, and then click **Select Columns**.
5. Select the **Process Image** tab and select the **Integrity Level** check box. The dialog box should look similar to the one shown here:



6. Process Explorer shows you the integrity level of the processes on your system. You should see the Notepad process at medium, the Edge

(MicrosoftEdge.exe) process at AppContainer, and the elevated command prompt at High. Also note that the services and system processes are running at an even higher integrity level, System.



Process	PID	CPU	Integrity	Private Bytes	Working Set	Description
lsass.exe	496	< 0.01	System	9,164 K	20,884 K	Local Security Authority
lync.exe	13928	0.04	Medium	183,248 K	142,044 K	Skype for Business
macmnsvc.exe	3808	0.05	System	15,116 K	25,572 K	McAfee A
maccompatsvc.exe	7216	0.01	System	6,920 K	19,348 K	MA Comp
masvc.exe	3784	0.04	System	5,444 K	18,604 K	McAfee A
mcshield.exe	6864	< 0.01	System	167,116 K	188,288 K	McAfee C
mctray.exe	14372	0.11	Medium	8,556 K	1,356 K	McTray A
mDNSResponder.exe	3484		System	2,048 K	7,168 K	Bonjour S
Memory Compression	4568		System	156 K	18,224 K	
mfeann.exe	4860	< 0.01	System	4,336 K	3,152 K	VSCore A
MfeEpeHost.exe	3800	0.02	System	29,884 K	32,904 K	McAfee D
mfemms.exe	3852		System	3,360 K	8,716 K	McAfee M
mfefvps.exe	3824		System	1,644 K	6,596 K	McAfee P
mfefvps.exe	6372		System	5,368 K	11,048 K	McAfee P
Microsoft.Alm.Shared.Remoting.Rem...	11868		Medium	26,360 K	39,900 K	Microsoft
Microsoft.VsHub.Server.HttpHost.exe	12120	0.01	Medium	202,704 K	202,616 K	Microsoft
MicrosoftEdge.exe	17216	< 0.01	AppContainer	36,900 K	73,772 K	Microsoft
MicrosoftEdgeCP.exe	16836	0.01	AppContainer	66,864 K	96,636 K	Microsoft
mqsvc.exe	4208		System	5,696 K	13,492 K	Message
notepad.exe	1184	< 0.01	Medium	2,372 K	14,228 K	Notepad
NvBackend.exe	10724	< 0.01	Medium	2,452 K	10,032 K	NVIDIA B
nvtray.exe	10640		Medium	3,572 K	13,716 K	NVIDIA S
nvsvs.exe	1952		System	2,492 K	10,136 K	NVIDIA D
nvwmi64.exe	1960		System	1,444 K	6,836 K	NVIDIA V
nvwmi64.exe	3756	< 0.01	System	5,428 K	15,232 K	NVIDIA V
nvxdsync.exe	1900	< 0.01	System	10,836 K	26,724 K	NVIDIA U
OfficeClickToRun.exe	3560	< 0.01	System	40,148 K	56,256 K	Microsoft
OneDrive.exe	13888	0.06	Medium	219,172 K	242,160 K	Microsoft
OUTLOOK.EXE	14236	0.03	Medium	167,308 K	212,160 K	Microsoft
PerfWatson2.exe	15180	0.01	Medium	59,984 K	74,372 K	PerfWatsc
PerfWatson2.exe	14836	0.01	High	41,044 K	55,548 K	PerfWatsc
PresentationFontCache.exe	1540		System	23,980 K	24,220 K	Presentati
PrivacyIconClient.exe	<					

CPU Usage: 13.74% Commit Charge: 35.87% Processes: 175 Physical Usage: 31.94%

Every process has an integrity level that is represented in its token and propagated according to the following rules:

- A process normally inherits the integrity level of its parent (which means an elevated command prompt will spawn other elevated processes).
- If the file object for the executable image to which the child process belongs has an integrity level and the parent process's integrity level is medium or higher, the child process will inherit the lower of the two.

■ A parent process can create a child process with an explicit integrity level lower than its own. To do this, it uses `DuplicateTokenEx` to duplicate its own access token, it uses `SetToken-Information` to change the integrity level in the new token to the desired level, and then it calls `CreateProcessAsUser` with that new token.

Table 7-3 lists the integrity level associated with processes, but what about objects? Objects also have an integrity level stored as part of their security descriptor, in a structure that is called the *mandatory label*.

To support migrating from previous versions of Windows (whose registry keys and files would not include integrity-level information), and to make it simpler for application developers, all objects have an implicit integrity level to avoid having to manually specify one. This implicit integrity level is medium, meaning that the mandatory policy (described shortly) on the object will be performed on tokens accessing this object with an integrity level lower than medium.

When a process creates an object without specifying an integrity level, the system checks the integrity level in the token. For tokens with a level of medium or higher, the implicit integrity level of the object remains medium. However, when a token contains an integrity level lower than medium, the object is created with an explicit integrity level that matches the level in the token.

Objects that are created by high- or system-integrity-level processes have a medium integrity level themselves so that users can disable and enable UAC. If object integrity levels always inherited their creator's integrity level, the applications of an administrator who disables UAC and subsequently re-enables it could fail because the administrator would not be able to modify any registry settings or files created when running at the high integrity level. Objects can also have an explicit integrity level that is set by the system or by the creator of the object. For example, processes, threads, tokens, and jobs are given an explicit integrity level by the kernel when it creates them. The reason for assigning an integrity level to these objects is to prevent a process for the same user, but one running at a lower integrity level, from accessing these

objects and modifying their content or behavior (for example, DLL injection or code modification).

Apart from an integrity level, objects also have a mandatory policy, which defines the actual level of protection that's applied based on the integrity-level check. Three types are possible, shown in [Table 7-4](#). The integrity level and the mandatory policy are stored together in the same ACE.

Policy	Present on, by Default	Description
No-Write-Up	Implicit on all objects	Used to restrict write access coming from a lower integrity level process to the object.
No-Read-Up	Only on process objects	Used to restrict read access coming from a lower integrity level process to the object. Specific use on process objects protects against information leakage by blocking address space reads from an external process.
No-Execute-Up	Only on binaries implementing COM classes	Used to restrict execute access coming from a lower integrity level process to the object. Specific use on COM classes is to restrict launch-activation permissions on a COM class.

TABLE 7-4 Object mandatory policies

EXPERIMENT: LOOKING AT THE INTEGRITY LEVEL OF OBJECTS

You can use the `AccessChk` tool from Sysinternals to display the integrity level of objects on the system, such as files, processes, and registry keys. Here's an experiment showing the purpose of the LocalLow directory in Windows:

1. Browse to `C:\Users\<UserName>\` in a command prompt window, where `<username>` is your user name.
2. Try running `AccessChk` on the AppData folder, as follows:

[Click here to view code image](#)

```
C:\Users\UserName> accesschk -v appdata
```

3. Note the differences between the Local and LocalLow subfolders in your output, similar to that shown here:

[Click here to view code image](#)

```
C:\Users\UserName\AppData\Local
Medium Mandatory Level (Default) [No-Write-Up]
[...]
C:\Users\UserName\AppData\LocalLow
Low Mandatory Level [No-Write-Up]
[...]
C:\Users\UserName\AppData\Roaming
Medium Mandatory Level (Default) [No-Write-Up]
[...]
```

4. Notice that the LocalLow directory has an integrity level set to Low, while the Local and Roaming directories have integrity levels of Medium (default). The default means the system is using an implicit integrity level.

5. Pass the `-e` flag to `AccessChk` so it displays only explicit integrity levels. If you run the tool on the AppData folder again, you'll notice only the LocalLow information is displayed.

The `-o` (object), `-k` (registry key), and `-p` (process) flags allow you to specify something other than a file or directory.

Tokens

The SRM uses an object called a *token* (or *access token*) to identify the security context of a process or thread. A security context consists of information that describes the account, groups, and privileges associated with the process or thread. Tokens also include information such as the session ID, the integrity level, and the UAC virtualization state. (We'll describe both privileges and UAC's virtualization mechanism later in this chapter.)

During the logon process (described later in this chapter), Lsass creates an initial token to represent the user logging on. It then determines whether the user logging on is a member of a powerful group or possesses a powerful privilege. The groups checked for in this step are as follows:

- Built-In Administrators
- Certificate Administrators
- Domain Administrators
- Enterprise Administrators
- Policy Administrators
- Schema Administrators
- Domain Controllers
- Enterprise Read-Only Domain Controllers
- Read-Only Domain Controllers

- Account Operators
- Backup Operators
- Cryptographic Operators
- Network Configuration Operators
- Print Operators
- System Operators
- RAS Servers
- Power Users
- Pre-Windows 2000 Compatible Access

Many of the groups listed are used only on domain-joined systems and don't give users local administrative rights directly. Instead, they allow users to modify domain-wide settings.

The privileges checked for are as follows:

- SeBackupPrivilege
 - SeCreateTokenPrivilege
 - SeDebugPrivilege
 - SeImpersonatePrivilege
 - SeLabelPrivilege
 - SeLoadDriverPrivilege
 - SeRestorePrivilege
-

- SeTakeOwnershipPrivilege

- SeTcbPrivilege

These privileges are described in detail in the section “Privileges,” later in this chapter.

If one or more of these groups or privileges are present, Lsass creates a restricted token for the user (also called a *filtered admin token*) and creates a logon session for both. The standard user token is attached to the initial process or processes that Winlogon starts (by default, Userinit.exe).



If UAC has been disabled, administrators run with a token that includes their administrator group memberships and privileges.

Because child processes inherit a copy of the token of their creators by default, all processes in the user’s session run under the same token. You can also generate a token by using the Windows `LogonUser` function. You can then use this token to create a process that runs within the security context of the user logged on through the `LogonUser` function by passing the token to the Windows `CreateProcessAsUser` function. The `CreateProcessWithLogonW` function combines these into a single call, which is how the `Runas` command launches processes under alternative tokens.

Tokens vary in size because different user accounts have different sets of privileges and associated group accounts. However, all tokens contain the same types of information. The most important contents of a token are represented in **Figure 7-7**.

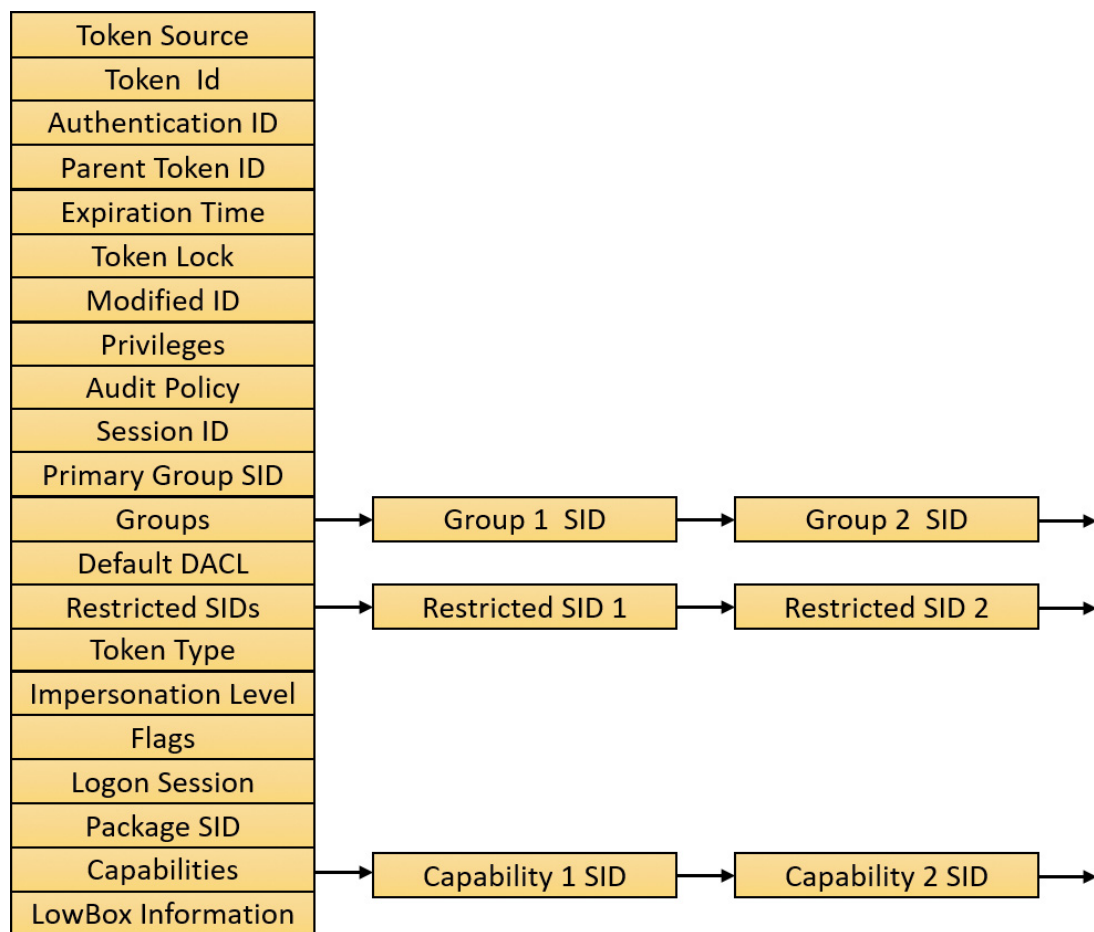


FIGURE 7-7 Access tokens.

The security mechanisms in Windows use two components to determine what objects can be accessed and what secure operations can be performed. One component comprises the token's user account SID and group SID fields. The SRM uses SIDs to determine whether a process or thread can obtain requested access to a securable object, such as an NTFS file.

The group SIDs in a token indicate which groups a user's account is a member of. For example, a server application can disable specific groups to restrict a token's credentials when the server application is performing actions requested by a client. Disabling a group produces nearly the same effect as if the group wasn't present in the token. (It results in a deny-only group, described in the section "**Restricted to-
kens.**" Disabled SIDs are used as part of security access checks, described in the section "**Determining access**" later in the chapter.) Group SIDs can also include a special SID that contains the integrity level of the process or thread. The SRM uses another field in the token,

which describes the mandatory integrity policy, to perform the mandatory integrity check described later in the chapter.

The second component in a token that determines what the token's thread or process can do is the privilege array. A token's privilege array is a list of rights associated with the token. An example of a privilege is the right of the process or thread associated with the token to shut down the computer. Privileges are described in more detail later in this chapter.

A token's default primary group field and default discretionary access control list (DACL) field are security attributes that Windows applies to objects that a process or thread creates when it uses the token. By including security information in tokens, Windows makes it convenient for a process or thread to create objects with standard security attributes because the process or thread doesn't need to request discrete security information for every object it creates.

Each token's type distinguishes a primary token (a token that identifies the security context of a process) from an impersonation token (a type of token that threads use to temporarily adopt a different security context, usually of another user). Impersonation tokens carry an impersonation level that signifies what type of impersonation is active in the token. (Impersonation is described later in this chapter.)

A token also includes the mandatory policy for the process or thread, which defines how MIC will behave when processing this token. There are two policies:

- **TOKEN_MANDATORY_NO_WRITE_UP** Enabled by default, this sets the No-Write-Up policy on this token, specifying that the process or thread will not be able to access objects with a higher integrity level for write access.

- **TOKEN_MANDATORY_NEW_PROCESS_MIN** Also enabled by default, this specifies that the SRM should look at the integrity level of the executable image when launching a child process and compute the mini-

imum integrity level of the parent process and the file object's integrity level as the child's integrity level.

Token flags include parameters that determine the behavior of certain UAC and UIPI mechanisms, such as virtualization and user interface access. Those mechanisms will be described later in this chapter.

Each token can also contain attributes that are assigned by the Application Identification service (part of AppLocker) when AppLocker rules have been defined. AppLocker and its use of attributes in the access token are described later in this chapter.

A token for a UWP process includes information on the AppContainer hosting the process. First, it stores a package SID, identifying the UWP package the process originated from. The significance of this SID will be described in the “[AppContainers](#)” section later in this chapter. Second, UWP processes need to request capabilities for operations that require the user's consent. Examples of capabilities include network access, using the phone capabilities of the device (if any), accessing the camera on the device, and more. Each such capability is represented with a SID, stored as part of the token. (Capabilities will be discussed further in the “[AppContainers](#)” section.)

The remaining fields in a token serve informational purposes. The token source field contains a short textual description of the entity that created the token. Programs that want to know where a token originated use the token source to distinguish among sources such as the Windows Session Manager, a network file server, or the remote procedure call (RPC) server. The token identifier is a locally unique identifier (LUID) that the SRM assigns to the token when it creates the token. The Windows executive maintains the executive LUID, a monotonically increasing counter it uses to assign a unique numeric identifier to each token. A LUID is guaranteed to be unique only until the system is shut down.

The token authentication ID is another kind of LUID. A token's creator assigns the token's authentication ID when calling the `LsaLogonUser`

function. If the creator doesn't specify a LUID, Lsass obtains the LUID from the executive LUID. Lsass copies the authentication ID for all tokens descended from an initial logon token. A program can obtain a token's authentication ID to see whether the token belongs to the same logon session as other tokens the program has examined.

The executive LUID refreshes the modified ID every time a token's characteristics are modified. An application can test the modified ID to discover changes in a security context since the context's last use.

Tokens contain an expiration time field that can be used by applications performing their own security to reject a token after a specified amount of time. However, Windows itself does not enforce the expiration time of tokens.



NOTE

To guarantee system security, the fields in a token are immutable (because they are located in kernel memory).

Except for fields that can be modified through a specific system call designed to modify certain token attributes (assuming the caller has the appropriate access rights to the token object), data such as privileges and SIDs in a token can never be modified from user mode.

EXPERIMENT: VIEWING ACCESS TOKENS

The kernel debugger `dt _TOKEN` command displays the format of an internal token object. Although this structure differs from the user-mode token structure returned by Windows API security functions, the fields are similar. For further information on tokens, see the description in the Windows SDK documentation.

Here's a token structure on Windows 10:

[Click here to view code image](#)

```
lkd> dt nt!_token
+0x000 TokenSource   : _TOKEN_SOURCE
+0x010 TokenId       : _LUID
+0x018 AuthenticationId : _LUID
+0x020 ParentTokenId  : _LUID
+0x028 ExpirationTime : _LARGE_INTEGER
+0x030 TokenLock      : Ptr64 _ERESOURCE
+0x038 ModifiedId     : _LUID
+0x040 Privileges     : _SEP_TOKEN_PRIVILEGES
+0x058 AuditPolicy    : _SEP_AUDIT_POLICY
+0x078 SessionId      : UInt4B
+0x07c UserAndGroupCount : UInt4B
+0x080 RestrictedSidCount : UInt4B
+0x084 VariableLength : UInt4B
```

- +0x088 DynamicCharged : Uint4B
- +0x08c DynamicAvailable : Uint4B
- +0x090 DefaultOwnerIndex : Uint4B
- +0x098 UserAndGroups : Ptr64 _SID_AND_ATTRIBUTES
- +0x0a0 RestrictedSids : Ptr64 _SID_AND_ATTRIBUTES
- +0x0a8 PrimaryGroup : Ptr64 Void
- +0x0b0 DynamicPart : Ptr64 Uint4B
- +0x0b8 DefaultDacl : Ptr64 _ACL
- +0x0c0 TokenType : _TOKEN_TYPE
- +0x0c4 ImpersonationLevel : _SECURITY_IMPERSONATION_LEVEL
- +0x0c8 TokenFlags : Uint4B
- +0x0cc TokenInUse : UChar
- +0x0d0 IntegrityLevelIndex : Uint4B
- +0x0d4 MandatoryPolicy : Uint4B
- +0x0d8 LogonSession : Ptr64 _SEP_LOGON_SESSION_REFERENCES
- +0x0e0 OriginatingLogonSession : _LUID
- +0x0e8 SidHash : _SID_AND_ATTRIBUTES_HASH
- +0x1f8 RestrictedSidHash : _SID_AND_ATTRIBUTES_HASH
- +0x308 pSecurityAttributes : Ptr64
- _AUTHZBASEP_SECURITY_ATTRIBUTES_INFORMATION
- +0x310 Package : Ptr64 Void
- +0x318 Capabilities : Ptr64 _SID_AND_ATTRIBUTES
- +0x320 CapabilityCount : Uint4B
- +0x328 CapabilitiesHash : _SID_AND_ATTRIBUTES_HASH
- +0x438 LowboxNumberEntry : Ptr64 _SEP_LOWBOX_NUMBER_ENTRY
- +0x440 LowboxHandlesEntry : Ptr64
- _SEP_LOWBOX_HANDLES_ENTRY
- +0x448 pClaimAttributes : Ptr64
- _AUTHZBASEP_CLAIM_ATTRIBUTES_COLLECTION
- +0x450 TrustLevelSid : Ptr64 Void
- +0x458 TrustLinkedToken : Ptr64 _TOKEN
- +0x460 IntegrityLevelSidValue : Ptr64 Void
- +0x468 TokenSidValues : Ptr64 _SEP_SID_VALUES_BLOCK
- +0x470 IndexEntry : Ptr64 _SEP_LUID_TO_INDEX_MAP_ENTRY
- +0x478 DiagnosticInfo : Ptr64 _SEP_TOKEN_DIAG_TRACK_ENTRY

+0x480 SessionObject : Ptr64 Void

+0x488 VariablePart : Uint8B

You can examine the token for a process with the `!token` command. You'll find the address of the token in the output of the `!process` command. Here's an example for an explorer.exe process:

[Click here to view code image](#)

```
lkd> !process 0 1 explorer.exe
```

```
PROCESS fffff18304dfd780
```

```
SessionId: 1 Cid: 23e4 Peb: 00c2a000 ParentCid: 2264
```

```
DirBase: 2aa0f6000 ObjectTable: fffffd82c72fcd80 HandleCount:
```

```
<Data Not
```

```
Accessible>
```

```
Image: explorer.exe
```

```
VadRoot fffff18303655840 Vads 705 Clone 0 Private 12264. Modified  
376410. Locked 18.
```

```
DeviceMap fffffd82c39bc0d0
```

```
Token fffffd82c72fc060
```

```
...
```

```
PROCESS fffff1830670a080
```

```
SessionId: 1 Cid: 27b8 Peb: 00950000 ParentCid: 035c
```

```
DirBase: 2cba97000 ObjectTable: fffffd82c7ccc500 HandleCount:
```

```
<Data Not
```

```
Accessible>
```

```
Image: explorer.exe
```

```
VadRoot fffff183064e9f60 Vads 1991 Clone 0 Private 19576. Modified  
87095. Locked 0.
```

```
DeviceMap fffffd82c39bc0d0
```

```
Token fffffd82c7cd9060
```

```
...
```

```
lkd> !token fffffd82c72fc060
```

```
_TOKEN 0xffffd82c72fc060
```

```
TS Session ID: 0x1
```

User: S-1-5-21-3537846094-3055369412-2967912182-1001

User Groups:

00 S-1-16-8192

Attributes - GroupIntegrity GroupIntegrityEnabled

01 S-1-1-0

Attributes - Mandatory Default Enabled

02 S-1-5-114

Attributes - DenyOnly

03 S-1-5-21-3537846094-3055369412-2967912182-1004

Attributes - Mandatory Default Enabled

04 S-1-5-32-544

Attributes - DenyOnly

05 S-1-5-32-578

Attributes - Mandatory Default Enabled

06 S-1-5-32-559

Attributes - Mandatory Default Enabled

07 S-1-5-32-545

Attributes - Mandatory Default Enabled

08 S-1-5-4

Attributes - Mandatory Default Enabled

09 S-1-2-1

Attributes - Mandatory Default Enabled

10 S-1-5-11

Attributes - Mandatory Default Enabled

11 S-1-5-15

Attributes - Mandatory Default Enabled

12 S-1-11-96-3623454863-58364-18864-2661722203-1597581903-

1225312835-2511459453-

1556397606-2735945305-1404291241

Attributes - Mandatory Default Enabled

13 S-1-5-113

Attributes - Mandatory Default Enabled

14 S-1-5-5-0-1745560

Attributes - Mandatory Default Enabled LogonId

15 S-1-2-0

Attributes - Mandatory Default Enabled

16 S-1-5-64-36

Attributes - Mandatory Default Enabled

Primary Group: S-1-5-21-3537846094-3055369412-2967912182-1001

Privs:

19 0x0000000013 SeShutdownPrivilege Attributes -

23 0x0000000017 SeChangeNotifyPrivilege Attributes - Enabled

Default

25 0x0000000019 SeUndockPrivilege Attributes -

33 0x0000000021 SeIncreaseWorkingSetPrivilege Attributes -

34 0x0000000022 SeTimeZonePrivilege Attributes -

Authentication ID: (0,1aa448)

Impersonation Level: Anonymous

TokenType: Primary

Source: User32 TokenFlags: 0x2a00 (Token in use)

Token ID: 1be803 ParentToken ID: 1aa44b

Modified ID: (0, 43d9289)

RestrictedSidCount: 0 RestrictedSids: 0x0000000000000000

OriginatingLogonSession: 3e7

PackageSid: (null)

CapabilityCount: 0 Capabilities: 0x0000000000000000

LowboxNumberEntry: 0x0000000000000000

Security Attributes:

Unable to get the offset of

nt!_AUTHZBASEP_SECURITY_ATTRIBUTE.ListLink

Process Token TrustLevelSid: (null)

Notice that there is no package SID for Explorer, since it's not running inside an AppContainer.

Run calc.exe under Windows 10, which spawns calculator.exe (now a UWP app), and examine its token:

[Click here to view code image](#)

lkd> !process 0 1 calculator.exe

PROCESS fffff18309e874c0

SessionId: 1 Cid: 3c18 Peb: cd0182c000 ParentCid: 035c

DirBase: 7a15e4000 ObjectTable: fffcd82ec9a37c0 HandleCount:
<Data Not
Accessible>
Image: Calculator.exe
VadRoot ffffe1831cf197c0 Vads 181 Clone 0 Private 3800. Modified
3746. Locked 503.
DeviceMap fffcd82c39bc0d0
Token **ffffcd82e26168f0**
...

lkd> !token **ffffcd82e26168f0**
_TOKEN 0xffffcd82e26168f0
TS Session ID: 0x1
User: S-1-5-21-3537846094-3055369412-2967912182-1001
User Groups:
00 S-1-16-4096
Attributes - GroupIntegrity GroupIntegrityEnabled
01 S-1-1-0
Attributes - Mandatory Default Enabled
02 S-1-5-114
Attributes - DenyOnly
03 S-1-5-21-3537846094-3055369412-2967912182-1004
Attributes - Mandatory Default Enabled
04 S-1-5-32-544
Attributes - DenyOnly
05 S-1-5-32-578
Attributes - Mandatory Default Enabled
06 S-1-5-32-559
Attributes - Mandatory Default Enabled
07 S-1-5-32-545
Attributes - Mandatory Default Enabled
08 S-1-5-4
Attributes - Mandatory Default Enabled
09 S-1-2-1
Attributes - Mandatory Default Enabled
10 S-1-5-11

Attributes - Mandatory Default Enabled

11 S-1-5-15

Attributes - Mandatory Default Enabled

12 S-1-11-96-3623454863-58364-18864-2661722203-1597581903-

1225312835-2511459453-

1556397606-2735945305-1404291241

Attributes - Mandatory Default Enabled

13 S-1-5-113

Attributes - Mandatory Default Enabled

14 S-1-5-5-0-1745560

Attributes - Mandatory Default Enabled LogonId

15 S-1-2-0

Attributes - Mandatory Default Enabled

16 S-1-5-64-36

Attributes - Mandatory Default Enabled

Primary Group: S-1-5-21-3537846094-3055369412-2967912182-1001

Privs:

19 0x000000013 SeShutdownPrivilege Attributes -

23 0x000000017 SeChangeNotifyPrivilege Attributes - Enabled

Default

25 0x000000019 SeUndockPrivilege Attributes -

33 0x000000021 SeIncreaseWorkingSetPrivilege Attributes -

34 0x000000022 SeTimeZonePrivilege Attributes -

Authentication ID: (0,1aa448)

Impersonation Level: Anonymous

TokenType: Primary

Source: User32 TokenFlags: 0x4a00 (Token in use)

Token ID: 4ddb8c0 ParentToken ID: 1aa44b

Modified ID: (0, 4ddb8b2)

RestrictedSidCount: 0 RestrictedSids: 0x0000000000000000

OriginatingLogonSession: 3e7

PackageSid: S-1-15-2-466767348-3739614953-2700836392-1801644223-

4227750657-

1087833535-2488631167

CapabilityCount: 1 Capabilities: 0xffffcd82e1bfccd0

Capabilities:

00 S-1-15-3-466767348-3739614953-2700836392-1801644223-
4227750657-1087833535-
2488631167

Attributes - Enabled

LowboxNumberEntry: 0xffffcd82fa2c1670

LowboxNumber: 5

Security Attributes:

Unable to get the offset of

nt!_AUTHZBASEP_SECURITY_ATTRIBUTE.ListLink

Process Token TrustLevelSid: (null)

You can see there is one capability required by Calculator (which is in fact equal to its AppContainer SID RID, as described in the section “**AppContainers**” later in this chapter). Looking at the token of the Cortana process (searchui.exe) shows the following capabilities:

[Click here to view code image](#)

```
lkd> !process 0 1 searchui.exe
```

```
PROCESS fffffe1831307d080
```

```
    SessionId: 1  Cid: 29d8  Peb: fb407ec000  ParentCid: 035c
```

```
DeepFreeze
```

```
    DirBase: 38b635000  ObjectTable: fffffcd830059e580  HandleCount:
```

```
<Data Not
```

```
Accessible>
```

```
    Image: SearchUI.exe
```

```
    VadRoot fffffe1831fe89130  Vads 420  Clone 0  Private 11029. Modified  
2031. Locked 0.
```

```
    DeviceMap fffffcd82c39bc0d0
```

```
    Token                fffffcd82d97d18f0
```

```
    ...
```

```
lkd> !token fffffcd82d97d18f0
```

```
_TOKEN 0xffffcd82d97d18f0
```

```
TS Session ID: 0x1
```

```
User: S-1-5-21-3537846094-3055369412-2967912182-1001
```

```
User Groups:
```

...

Primary Group: S-1-5-21-3537846094-3055369412-2967912182-1001

Privs:

19 0x000000013 SeShutdownPrivilege Attributes -
23 0x000000017 SeChangeNotifyPrivilege Attributes - Enabled

Default

25 0x000000019 SeUndockPrivilege Attributes -
33 0x000000021 SeIncreaseWorkingSetPrivilege Attributes -
34 0x000000022 SeTimeZonePrivilege Attributes -

Authentication ID: (0,1aa448)

Impersonation Level: Anonymous

TokenType: Primary

Source: User32 TokenFlags: 0x4a00 (Token in use)

Token ID: 4483430 ParentToken ID: 1aa44b

Modified ID: (0, 4481b11)

RestrictedSidCount: 0 RestrictedSids: 0x0000000000000000

OriginatingLogonSession: 3e7

PackageSid: S-1-15-2-1861897761-1695161497-2927542615-642690995-
327840285-

2659745135-2630312742

CapabilityCount: 32 Capabilities: 0xffffcd82f78149b0

Capabilities:

00 S-1-15-3-1024-1216833578-114521899-3977640588-1343180512-
2505059295-473916851-
3379430393-3088591068

Attributes - Enabled

01 S-1-15-3-1024-3299255270-1847605585-2201808924-710406709-
3613095291-873286183-
3101090833-2655911836

Attributes - Enabled

02 S-1-15-3-1024-34359262-2669769421-2130994847-3068338639-
3284271446-2009814230-
2411358368-814686995

Attributes - Enabled

03 S-1-15-3-1

Attributes - Enabled

...

29 S-1-15-3-3633849274-1266774400-1199443125-2736873758

Attributes - Enabled

30 S-1-15-3-2569730672-1095266119-53537203-1209375796

Attributes - Enabled

31 S-1-15-3-2452736844-1257488215-2818397580-3305426111

Attributes - Enabled

LowboxNumberEntry: 0xffffcd82c7539110

LowboxNumber: 2

Security Attributes:

Unable to get the offset of

nt!_AUTHZBASEP_SECURITY_ATTRIBUTE.ListLink

Process Token TrustLevelSid: (null)

There are 32 capabilities required by Cortana. This simply indicates the process is richer in features that need to be accepted by end users and validated by the system.

You can indirectly view token contents with Process Explorer's Security tab in the process Properties dialog box. The dialog box shows the groups and privileges included in the token of the process you examine.

EXPERIMENT: LAUNCHING A PROGRAM AT LOW INTEGRITY LEVEL

When you elevate a program, either by using the Run as Administrator option or because the program is requesting it, the program is explicitly launched at high integrity level. However, it is also possible to launch a program at low integrity level by using **PsExec** from Sysinternals:

1. Launch Notepad at low integrity level by using the following command:

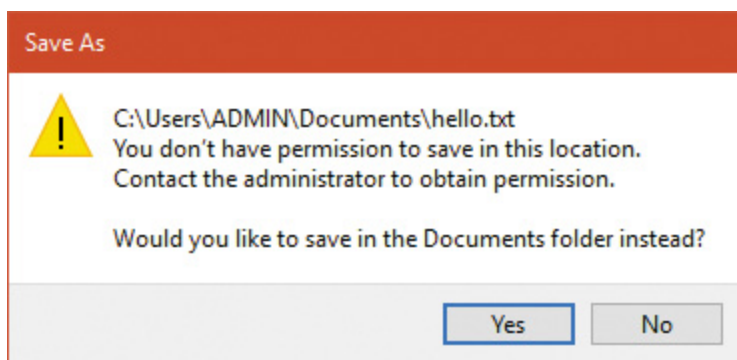
```
c:\psexec -l notepad.exe
```

2. Try opening a file (such as one of the XML files) in the %SystemRoot%\System32 directory. Notice that you can browse the di-

rectory and open any file contained within it.

3. In Notepad, open the **File** menu and choose **New**.

4. Enter some text in the window and try saving it in the %SystemRoot%\System32 directory. Notepad displays a dialog box indicating a lack of permissions and suggests saving the file in the Documents folder.



5. Accept Notepad's suggestion. You will get the same message box again, and repeatedly for each attempt.

6. Now try saving the file in the LocalLow directory of your user profile, shown in an experiment earlier in the chapter.

In the previous experiment, saving a file in the LocalLow directory worked because Notepad was running with low integrity level, and only the LocalLow directory also had low integrity level. All the other directories where you tried to save the file had an implicit medium integrity level. (You can verify this with `AccessChk`.) However, reading from the %SystemRoot%\System32 directory, as well as opening files within it, did work, even though the directory and its file also have an implicit medium integrity level.

Impersonation

Impersonation is a powerful feature Windows uses frequently in its security model. Windows also uses impersonation in its client/server programming model. For example, a server application can provide access

to resources such as files, printers, and databases. Clients wanting to access a resource send a request to the server. When the server receives the request, it must ensure that the client has permission to perform the desired operations on the resource. For example, if a user on a remote machine tries to delete a file on an NTFS share, the server exporting the share must determine whether the user is allowed to delete the file. The obvious way to determine whether a user has permission is for the server to query the user's account and group SIDs and scan the security attributes on the file. This approach is tedious to program, prone to errors, and wouldn't permit new security features to be supported transparently. Thus, Windows provides impersonation services to simplify the server's job.

Impersonation lets a server notify the SRM that the server is temporarily adopting the security profile of a client making a resource request. The server can then access resources on behalf of the client, and the SRM carries out the access validation, but it does so based on the impersonated client security context. Usually, a server has access to more resources than a client does and loses some of its security credentials during impersonation. However, the reverse can be true: The server can gain security credentials during impersonation.

A server impersonates a client only within the thread that makes the impersonation request. Thread-control data structures contain an optional entry for an impersonation token. However, a thread's primary token, which represents the thread's real security credentials, is always accessible in the process's control structure.

Windows makes impersonation available through several mechanisms. For example, if a server communicates with a client through a named pipe, the server can use the `ImpersonateNamedPipeClient` Windows API function to tell the SRM that it wants to impersonate the user on the other end of the pipe. If the server is communicating with the client through Dynamic Data Exchange (DDE) or RPC, it can make similar impersonation requests using `DdeImpersonateClient` and `RpcImpersonateClient`. A thread can create an impersonation token that's simply a copy of its process token with the `ImpersonateSelf`

function. The thread can then alter its impersonation token, perhaps to disable SIDs or privileges. A Security Support Provider Interface (SSPI) package can impersonate its clients with

`ImpersonateSecurityContext`. SSPIs implement a network authentication protocol such as LAN Manager version 2 or Kerberos. Other interfaces such as COM expose impersonation through APIs of their own, such as `CoImpersonateClient`.

After the server thread finishes its task, it reverts to its primary security context. These forms of impersonation are convenient for carrying out specific actions at the request of a client and for ensuring that object accesses are audited correctly. (For example, the audit that is generated gives the identity of the impersonated client rather than that of the server process.) The disadvantage to these forms of impersonation is that they can't execute an entire program in the context of a client. In addition, an impersonation token can't access files or printers on network shares unless it is a delegation-level impersonation (described shortly) and has sufficient credentials to authenticate to the remote machine, or the file or printer share supports null sessions. (A null session is one that results from an anonymous logon.)

If an entire application must execute in a client's security context or must access network resources without using impersonation, the client must be logged on to the system. The `LogonUser` Windows API function enables this action. `LogonUser` takes an account name, a password, a domain or computer name, a logon type (such as interactive, batch, or service), and a logon provider as input, and it returns a primary token. A server thread can adopt the token as an impersonation token, or the server can start a program that has the client's credentials as its primary token. From a security standpoint, a process created using the token returned from an interactive logon via `LogonUser`, such as with the `CreateProcessAsUser` API, looks like a program a user starts by logging on to the machine interactively. The disadvantage to this approach is that a server must obtain the user's account name and password. If the server transmits this information across the network, the

server must encrypt it securely so that a malicious user snooping network traffic can't capture it.

To prevent the misuse of impersonation, Windows doesn't let servers perform impersonation without a client's consent. A client process can limit the level of impersonation that a server process can perform by specifying a security quality of service (SQOS) when connecting to the server. For instance, when opening a named pipe, a process can specify `SECURITY_ANONYMOUS`, `SECURITY_IDENTIFICATION`, `SECURITY_IMPERSONATION`, or `SECURITY_DELEGATION` as flags for the Windows `CreateFile` function. These same options apply to other impersonation-related functions listed earlier. Each level lets a server perform different types of operations with respect to the client's security context:

- **SecurityAnonymous** This is the most restrictive level of impersonation. The server can't impersonate or identify the client.
- **SecurityIdentification** This lets the server obtain the identity (the SIDs) of the client and the client's privileges, but the server can't impersonate the client.
- **SecurityImpersonation** This lets the server identify and impersonate the client on the local system.
- **SecurityDelegation** This is the most permissive level of impersonation. It lets the server impersonate the client on local and remote systems.

Other interfaces such as RPC use different constants with similar meanings (for example, `RPC_C_IMP_LEVEL_IMPERSONATE`).

If the client doesn't set an impersonation level, Windows chooses the `SecurityImpersonation` level by default. The `CreateFile` function also accepts `SECURITY_EFFECTIVE_ONLY` and `SECURITY_CONTEXT_TRACKING` as modifiers for the impersonation setting:

- **SECURITY_EFFECTIVE_ONLY** This prevents a server from enabling or disabling a client's privileges or groups while the server is impersonating.

- **SECURITY_CONTEXT_TRACKING** This specifies that any changes a client makes to its security context are reflected in a server that is impersonating it. If this option isn't specified, the server adopts the context of the client at the time of the impersonation and doesn't receive any changes. This option is honored only when the client and server processes are on the same system.

To prevent spoofing scenarios in which a low-integrity process could create a user interface that captured user credentials and then used `LogonUser` to obtain that user's token, a special integrity policy applies to impersonation scenarios: a thread cannot impersonate a token of higher integrity than its own. For example, a low-integrity application cannot spoof a dialog box that queries administrative credentials and then attempt to launch a process at a higher privilege level. The integrity-mechanism policy for impersonation access tokens is that the integrity level of the access token that is returned by `LsaLogonUser` must be no higher than the integrity level of the calling process.

Restricted tokens

A restricted token is created from a primary or impersonation token using the `CreateRestrictedToken` function. The restricted token is a copy of the token it's derived from, with the following possible modifications:

- Privileges can be removed from the token's privilege array.
- SIDs in the token can be marked as deny-only. These SIDs remove access to any resources for which the SID's access is denied by using a matching access-denied ACE that would otherwise be overridden by an ACE granting access to a group containing the SID earlier in the security descriptor.

- SIDs in the token can be marked as restricted. These SIDs are subject to a second pass of the access-check algorithm, which will parse only the restricted SIDs in the token. The results of both the first pass and the second pass must grant access to the resource or no access is granted to the object.

Restricted tokens are useful when an application wants to impersonate a client at a reduced security level, primarily for safety reasons when running untrusted code. For example, the restricted token can have the shutdown-system privilege removed from it to prevent code executed in the restricted token's security context from rebooting the system.

Filtered admin token

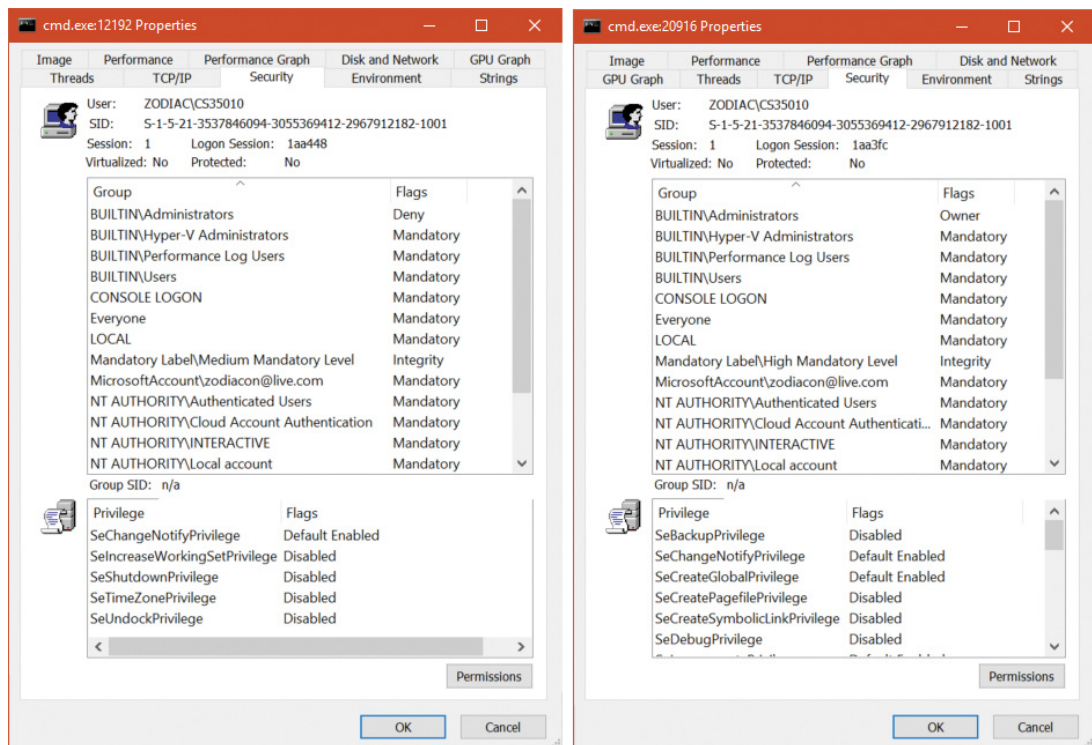
As you saw earlier, restricted tokens are also used by UAC to create the filtered admin token that all user applications will inherit. A filtered admin token has the following characteristics:

- The integrity level is set to medium.
- The administrator and administrator-like SIDs mentioned previously are marked as deny-only to prevent a security hole if the group were to be removed altogether. For example, if a file had an access control list (ACL) that denied the Administrators group all access but granted some access to another group the user belongs to, the user would be granted access if the Administrators group was absent from the token, which would give the standard user version of the user's identity more access than the user's administrator identity.
- All privileges are stripped except Change Notify, Shutdown, Undock, Increase Working Set, and Time Zone.

EXPERIMENT: LOOKING AT FILTERED ADMIN TOKENS

You can make Explorer launch a process with either the standard user token or the administrator token by following these steps on a machine with UAC enabled:

1. Log on to an account that's a member of the Administrators group.
2. Open the **Start** menu, type **command**, right-click the **Command Prompt** option that appears, and choose **Run as Administrator** to run an elevated command prompt.
3. Run a new instance of cmd.exe, but this time do it normally (that is, not elevated).
4. Run Process Explorer elevated, open the Properties dialog boxes for the two command prompt processes, and click the **Security** tabs. Note that the standard user token contains a deny-only SID and a medium mandatory label, and that it has only a couple of privileges. The properties on the right in the following screenshot are from a command prompt running with an administrator token, and the properties on the left are from one running with the filtered administrator token:



Virtual service accounts

Windows provides a specialized type of account known as a *virtual service account* (or simply *virtual account*) to improve the security isolation and access control of Windows services with minimal administrative effort. (See Chapter 9 in Part 2 for more information on Windows services.) Without this mechanism, Windows services must run under one of the accounts defined by Windows for its built-in services (such as Local Service or Network Service) or under a regular domain account. The accounts such as Local Service are shared by many existing services and so offer limited granularity for privilege and access control; furthermore, they cannot be managed across the domain. Domain accounts require periodic password changes for security, and the availability of services during a password-change cycle might be affected. Furthermore, for best isolation, each service should run under its own account, but with ordinary accounts this multiplies the management effort.

With virtual service accounts, each service runs under its own account with its own security ID. The name of the account is always NT SERVICE\ followed by the internal name of the service. Virtual service accounts can appear in access control lists and can be associated with privileges via Group Policy like any other account name. They cannot, however, be created or deleted through the usual account-management tools, nor assigned to groups.

Windows automatically sets and periodically changes the password of the virtual service account. Similar to the Local System and Other Service Accounts, there is a password, but the password is unknown to the system administrators.

EXPERIMENT: USING VIRTUAL SERVICE ACCOUNTS

You can create a service that runs under a virtual service account by using the Service Control (Sc.exe) tool. Follow these steps:

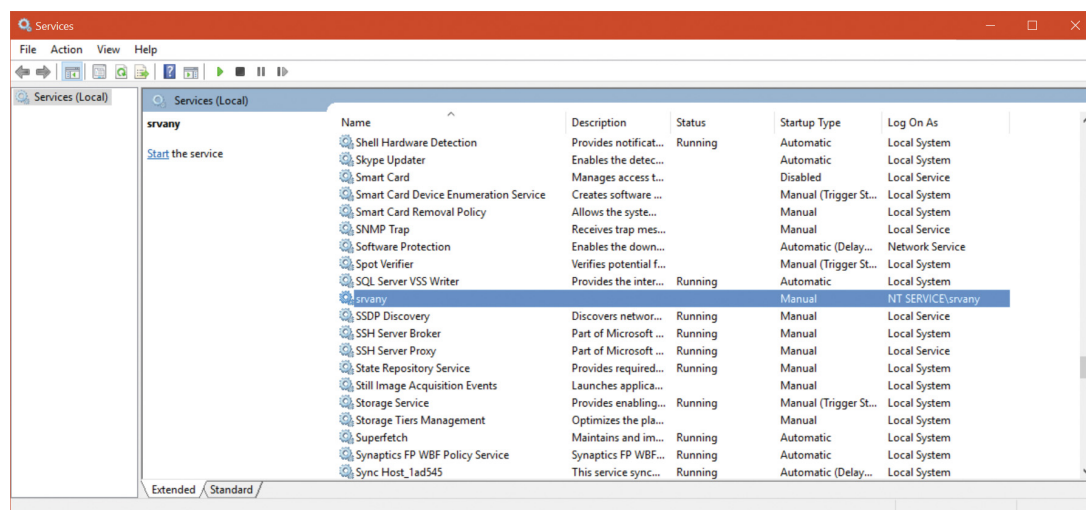
1. In an Administrator command prompt, type the **create** command in the Sc.exe command-line tool to create a service and a virtual account in which it will run. This example uses the `srvany` service from the Windows 2003 resource kit, which you can download here:

<https://www.microsoft.com/en-us/download/details.aspx?id=17657>.

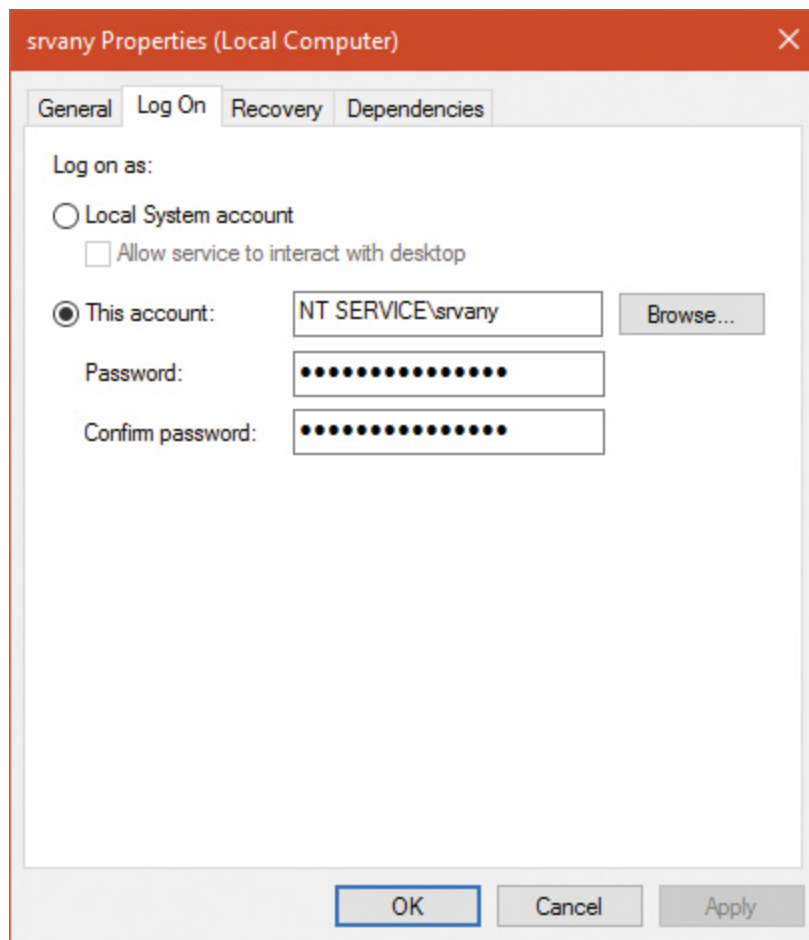
[Click here to view code image](#)

```
C:\Windows\system32>sc create srvany obj= "NT SERVICE\srvany"
binPath=
"c:\temp\srvany.exe"
[SC] CreateService SUCCESS
```

2. The previous command created the service (in the registry and in the service controller manager's internal list) and created the virtual service account. Now run the Services MMC snap-in (services.msc), select the new service, and open its Properties dialog box.

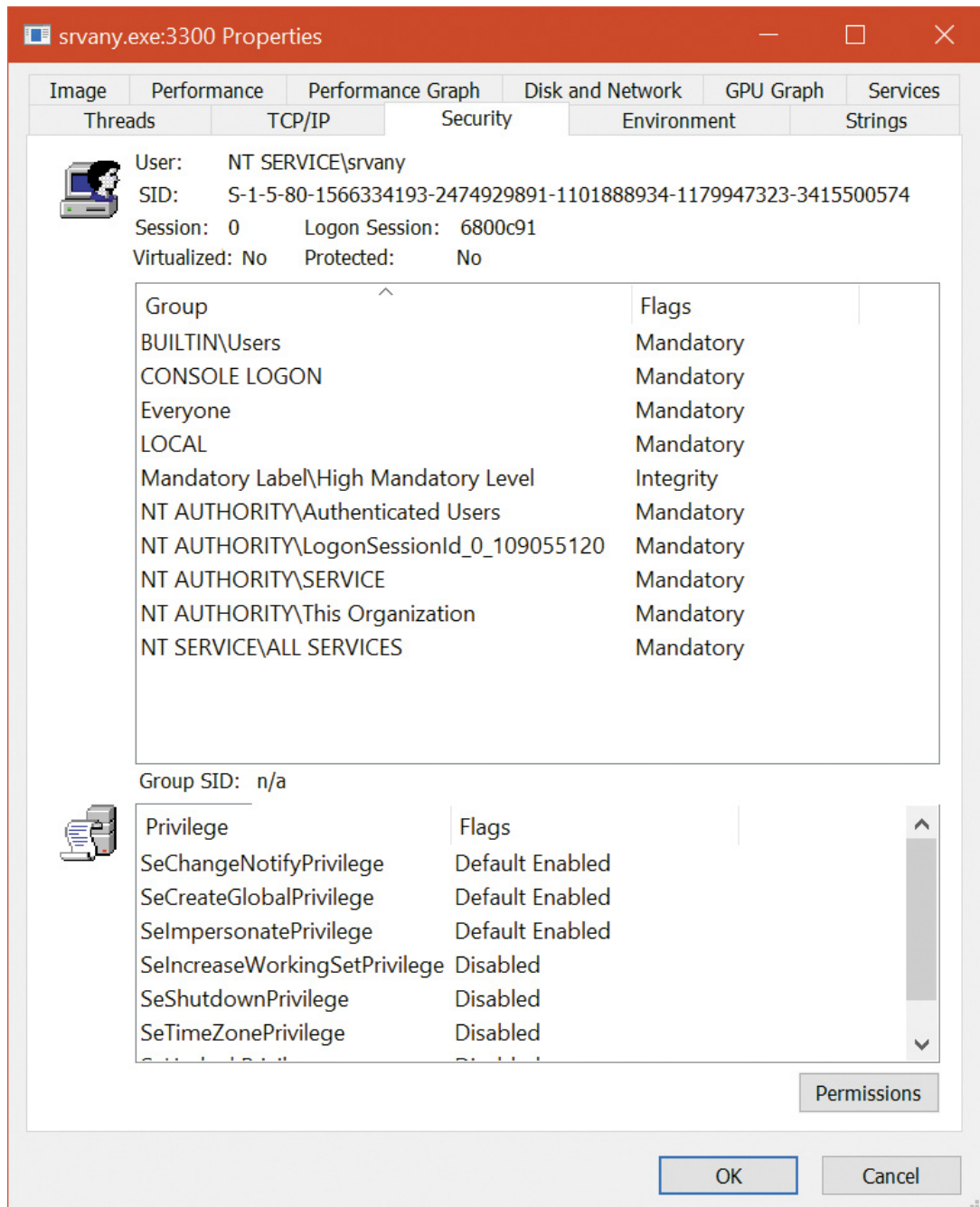


3. Click the **Log On** tab.

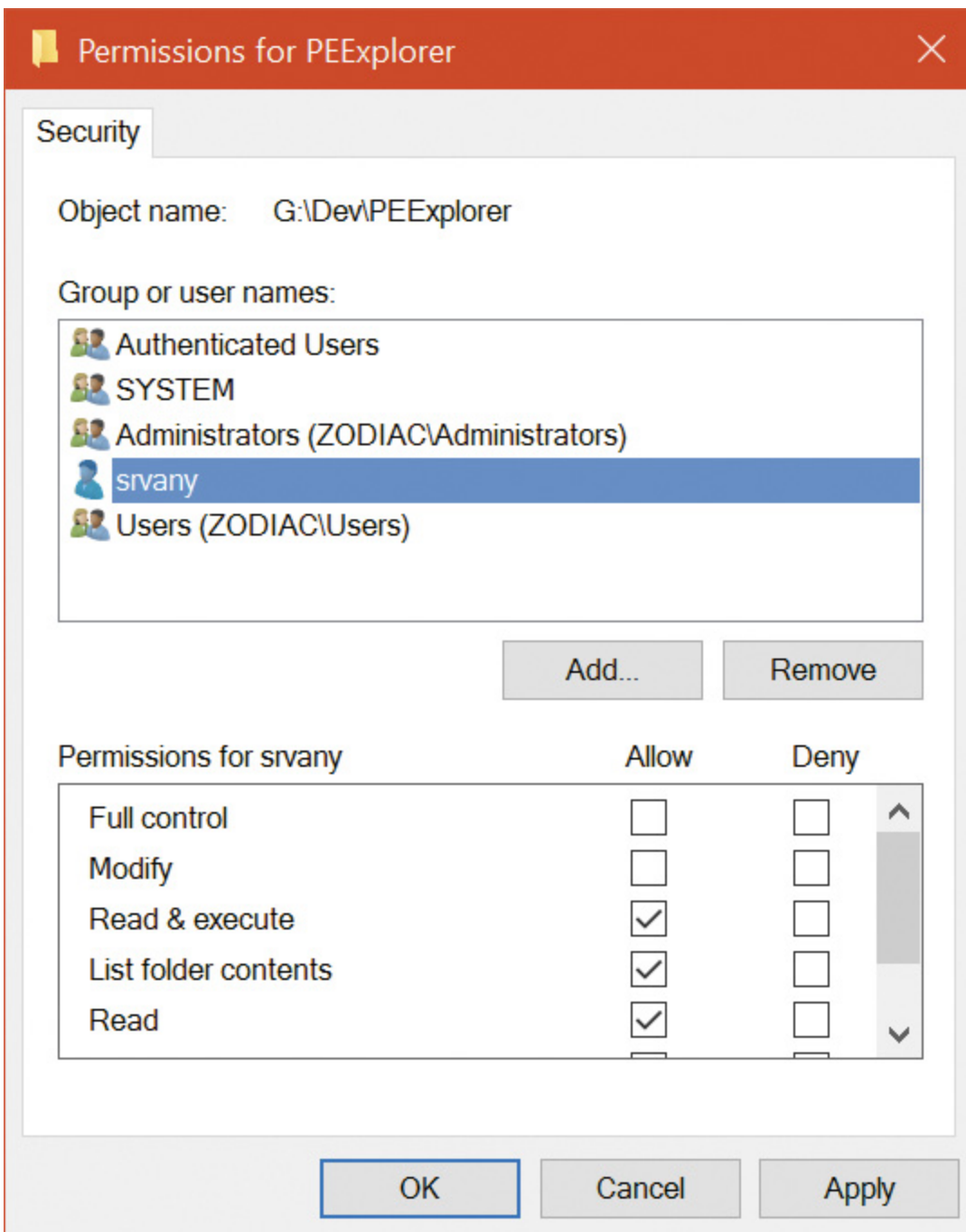


4. You can use the service's Properties dialog box to create a virtual service account for an existing service. To do so, change the account name to **NT SERVICE\servicename** in the This Account field and clear both password fields. Note, however, that existing services might not run correctly under a virtual service account because the account might not have access to files or other resources needed by the service.

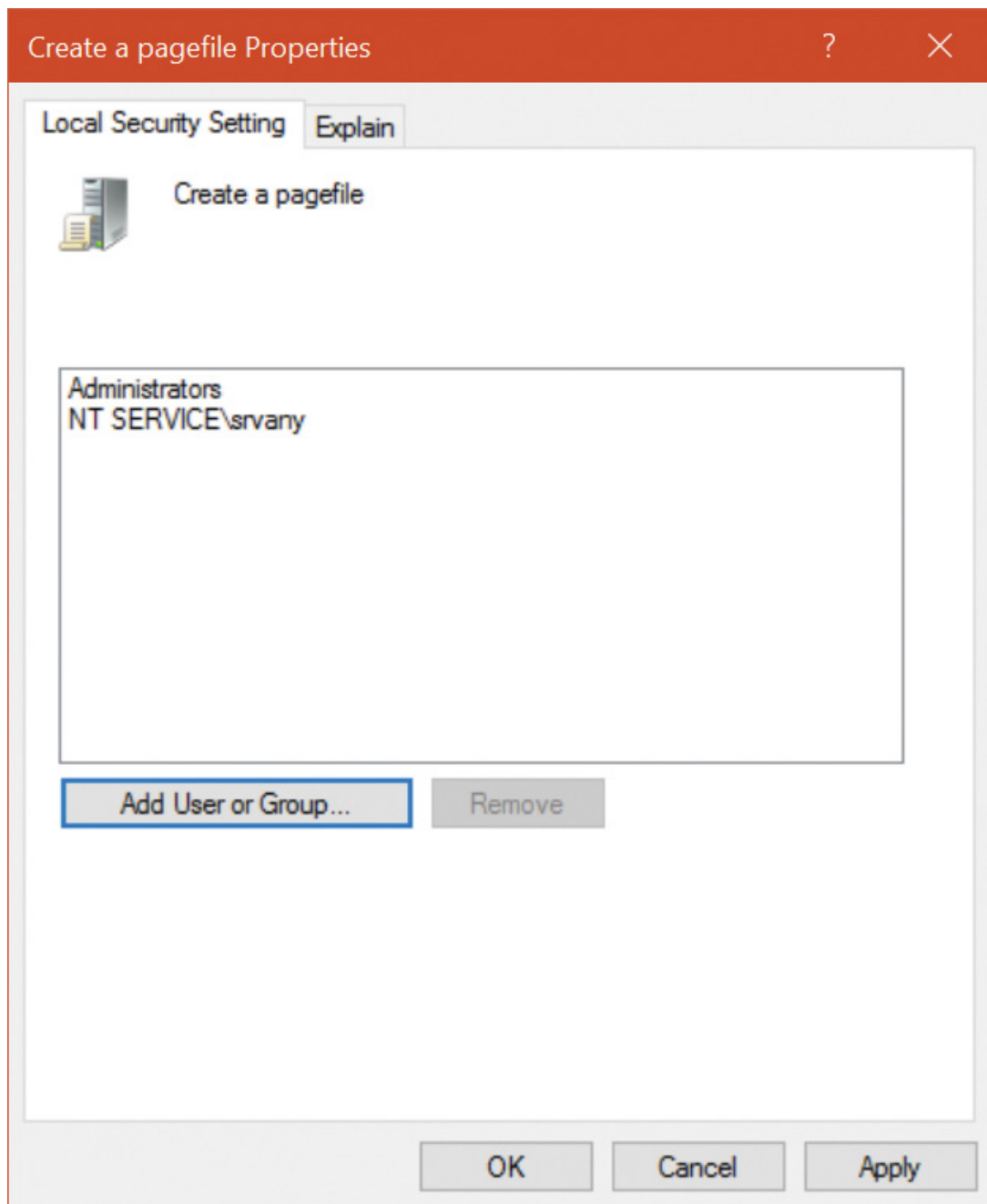
5. If you run Process Explorer and view the Security tab in the Properties dialog box for a service that uses a virtual account, you can observe the virtual account name and its security ID (SID). To try this, in the Properties dialog box of the `srvany` service, enter the command-line arguments `notepad.exe. (` `srvany` can be used to turn normal executables into services, so it must accept some executable on the command line.) Then click the **Start** button to start the service.



6. The virtual service account can appear in an access control entry for any object (such as a file) the service needs to access. If you click the **Security** tab in a file's Properties dialog box and create an ACL that references the virtual service account, you will find that the account name you typed (for example, NT SERVICE\srvany) is changed to simply the service name (srvany) by the Check Names function, and it appears in the access control list in this shortened form.

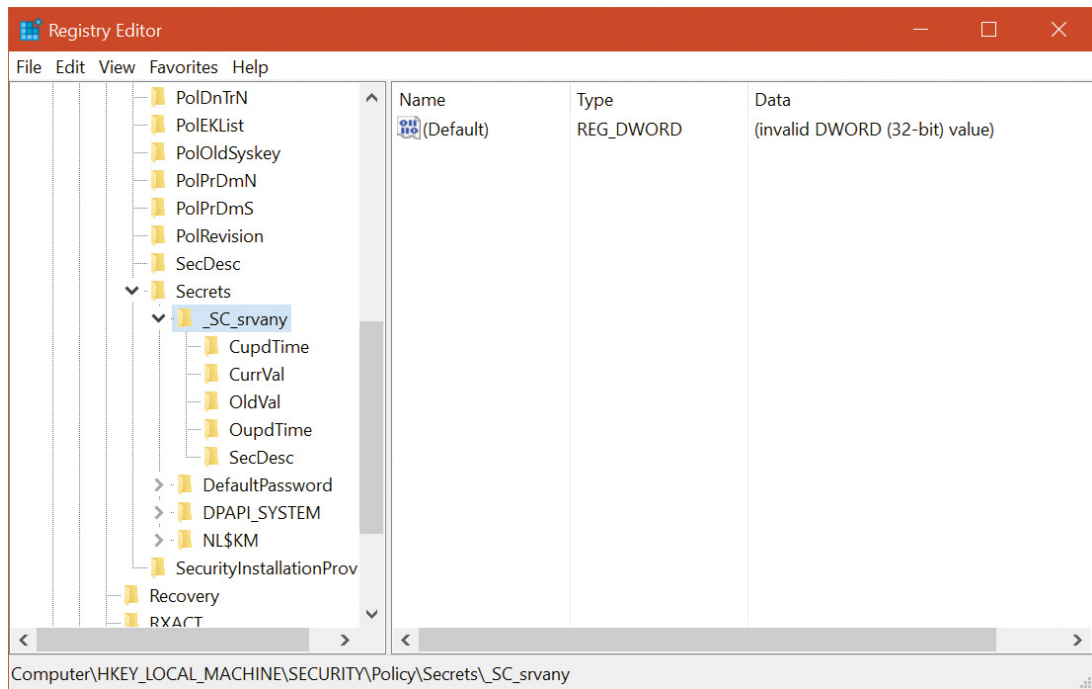


7. The virtual service account can be granted permissions (or user rights) via Group Policy. In this example, the virtual account for the **svany** service has been granted the right to create a pagefile (using the Local Security Policy editor, secpol.msc).



8. You won't see the virtual service account in user-administration tools like `lusrmgr.msc` because it is not stored in the SAM registry hive. However, if you examine the registry within the context of the built-in System account (as described previously), you will see evidence of the account in the `HKLM\Security\Policy\Secrets` key:

```
C:\>psexec -s -i -d regedit.exe
```



Security descriptors and access control

Tokens, which identify a user's credentials, are only part of the object security equation. Another part of the equation is the security information associated with an object, which specifies who can perform what actions on the object. The data structure for this information is called a *security descriptor*. A security descriptor consists of the following attributes:

- **Revision number** This is the version of the SRM security model used to create the descriptor.
- **Flags** These are optional modifiers that define the behavior or characteristics of the descriptor. These flags are listed in [Table 7-5](#) (most are documented in the Windows SDK).
- **Owner SID** This is the owner's SID.
- **Group SID** This is the SID of the primary group for the object (used only by the POSIX subsystem, now unused since POSIX is no longer supported).

■ **Discretionary access control list (DACL)** This specifies who has what access to the object.

■ **System access control list (SACL)** This specifies which operations by which users should be logged in the security audit log and the explicit integrity level of an object.

Flag	Meaning
SE_OWNER_DEFAULTED	This indicates a security descriptor with a default owner security identifier (SID). Use this bit to find all the objects that have default owner permissions set.
SE_GROUP_DEFAULTED	This indicates a security descriptor with a default group SID. Use this bit to find all the objects that have default group permissions set.
SE_DACL_PRESENT	This indicates a security descriptor that has a DACL. If this flag is not set, or if this flag is set and the DACL is NULL, the security descriptor allows full access to everyone.
SE_DACL_DEFAULTED	This indicates a security descriptor with a default DACL. For example, if an object creator does not specify a DACL, the object receives the default DACL from the access token of the creator. This flag can affect how the system treats the DACL with respect to access control entry (ACE) inheritance. The system ignores this flag if the SE_DACL_PRESENT flag is not set.
SE_SACL_PRESENT	This indicates a security descriptor that has a system access control list (SACL).
SE_SACL_DEFAULTED	This indicates a security descriptor with a default SACL. For example, if an object creator does not specify a SACL, the object receives the default SACL from the access token of the creator. This flag can affect how the system treats the SACL with respect to ACE inheritance. The system ignores this flag if the SE_SACL_PRESENT flag is not set.
SE_DACL_UNTRUSTED	This indicates that the ACL pointed to by the DACL of the security descriptor was provided by an untrusted source. If this flag is set and a compound ACE is encountered, the system will substitute known valid SIDs for the server SIDs in the ACEs.
SE_SERVER_SECURITY	This requests that the provider for the object protected by the security descriptor be a server ACL based on the input ACL, regardless of its source (explicit or defaulting). This is done by replacing all the GRANT ACEs with compound ACEs granting the current server access. This flag is meaningful only if the subject is impersonating.
SE_DACL_AUTO_INHERIT_REQ	This requests that the provider for the object protected by the security descriptor automatically propagate the DACL to existing child objects. If the provider supports automatic inheritance, the DACL is propagated to any existing child objects, and the SE_DACL_AUTO_INHERITED bit in the security descriptor of the parent and child objects is set.
SE_SACL_AUTO_INHERIT_REQ	This requests that the provider for the object protected by the security descriptor automatically propagate the SACL to existing child objects. If the provider supports automatic inheritance, the SACL is propagated to any existing child objects, and the SE_SACL_AUTO_INHERITED bit in the security descriptors of the parent object and child objects is set.
SE_DACL_AUTO_INHERITED	This indicates a security descriptor in which the DACL is set up to support automatic propagation of inheritable ACEs to existing child objects. The system sets this bit when it performs the automatic inheritance algorithm for the object and its existing child objects.
SE_SACL_AUTO_INHERITED	This indicates a security descriptor in which the SACL is set up to support automatic propagation of inheritable ACEs to existing child objects. The system sets this bit when it performs the automatic inheritance algorithm for the object and its existing child objects.
SE_DACL_PROTECTED	This prevents the DACL of a security descriptor from being modified by inheritable ACEs.
SE_SACL_PROTECTED	This prevents the SACL of a security descriptor from being modified by inheritable ACEs.
SE_RM_CONTROL_VALID	This indicates that the resource control manager bits in the security descriptor are valid. The resource control manager bits are 8 bits in the security descriptor structure that contains information specific to the resource manager accessing the structure.
SE_SELF_RELATIVE	This indicates a security descriptor in self-relative format, with all the security information in a contiguous block of memory. If this flag is not set, the security descriptor is in absolute format.

TABLE 7-5 Security descriptor flags

Security descriptors (SDs) can be retrieved programmatically by using various functions, such as `GetSecurityInfo`,

`GetKernelObjectSecurity`, `GetFileSecurity`, `GetNamedSecurityInfo`, and other more esoteric functions. After retrieval, the SD can be manipulated and then the relevant `Set` function called to make the change. Furthermore, a security descriptor can be constructed using a string in a language called Security Descriptor Definition Language (SDDL), which is capable of representing a security descriptor using a compact string. This string can be converted to a true SD by calling `ConvertStringSecurityDescriptorToSecurityDescriptor`. As you might expect, the converse function exists as well (`ConvertSecurityDescriptorToStringSecurityDescriptor`). See the Windows SDK for a detailed description of the SDDL.

An access control list (ACL) is made up of a header and zero or more access control entry (ACE) structures. There are two types of ACLs: DACLs and SACLs. In a DACL, each ACE contains a SID and an access mask (and a set of flags, explained shortly), which typically specifies the access rights (read, write, delete, and so forth) that are granted or denied to the holder of the SID. There are nine types of ACEs that can appear in a DACL: access allowed, access denied, allowed object, denied object, allowed callback, denied callback, allowed object callback, denied-object callback, and conditional claims. As you would expect, the access-allowed ACE grants access to a user, and the access-denied ACE denies the access rights specified in the access mask. The callback ACEs are used by applications that make use of the AuthZ API (described later) to register a callback that AuthZ will call when it performs an access check involving this ACE.

The difference between allowed object and access allowed, and between denied object and access denied, is that the object types are used only within Active Directory. ACEs of these types have a globally unique identifier (GUID) field that indicates that the ACE applies only to particular objects or subobjects (those that have GUID identifiers). (A GUID is a 128-bit identifier guaranteed to be universally unique.) In addition, another optional GUID indicates what type of child object will inherit the ACE when a child is created within an Active Directory container

that has the ACE applied to it. The conditional claims ACE is stored in a `*-callback` type ACE structure and is described in the section on the AuthZ APIs.

The accumulation of access rights granted by individual ACEs forms the set of access rights granted by an ACL. If no DACL is present (a null DACL) in a security descriptor, everyone has full access to the object. If the DACL is empty (that is, it has zero ACEs), no user has access to the object.

The ACEs used in DACLs also have a set of flags that control and specify characteristics of the ACE related to inheritance. Some object namespaces have containers and objects. A container can hold other container objects and leaf objects, which are its child objects. Examples of containers are directories in the file system namespace and keys in the registry namespace. Certain flags in an ACE control how the ACE propagates to child objects of the container associated with the ACE. [Table 7-6](#), reproduced in part from the Windows SDK, lists the inheritance rules for ACE flags.

Flag	Inheritance Rule
CONTAINER_INHERIT_ACE	Child objects that are containers, such as directories, inherit the ACE as an effective ACE. The inherited ACE is inheritable unless the NO_PROPAGATE_INHERIT_ACE bit flag is also set.
INHERIT_ONLY_ACE	This flag indicates an inherit-only ACE that doesn't control access to the object it's attached to. If this flag is not set, the ACE controls access to the object to which it is attached.
INHERITED_ACE	This flag indicates that the ACE was inherited. The system sets this bit when it propagates an inheritable ACE to a child object.
NO_PROPAGATE_INHERIT_ACE	If the ACE is inherited by a child object, the system clears the OBJECT_INHERIT_ACE and CONTAINER_INHERIT_ACE flags in the inherited ACE. This action prevents the ACE from being inherited by subsequent generations of objects.
OBJECT_INHERIT_ACE	Non-container child objects inherit the ACE as an effective ACE. For child objects that are containers, the ACE is inherited as an inherit-only ACE unless the NO_PROPAGATE_INHERIT_ACE bit flag is also set.

TABLE 7-6 Inheritance rules for ACE flags

A SACL contains two types of ACEs: system audit ACEs and system audit-object ACEs. These ACEs specify which operations performed on the object by specific users or groups should be audited. Audit information is stored in the system audit log. Both successful and unsuccessful attempts can be audited. Like their DACL object-specific ACE cousins, system audit-object ACEs specify a GUID indicating the types of objects or

sub-objects that the ACE applies to and an optional GUID that controls propagation of the ACE to particular child object types. If a SACL is null, no auditing takes place on the object. (Security auditing is described later in this chapter.) The inheritance flags that apply to DACL ACEs also apply to system audit and system audit-object ACEs.

Figure 7-8 is a simplified picture of a file object and its DACL. As shown, the first ACE allows USER1 to read the file. The second ACE denies members of the group TEAM1 write access to the file. The third ACE grants all other users (Everyone) execute access.

FIGURE 7-8 Discretionary access control list (DACL).

EXPERIMENT: VIEWING A SECURITY DESCRIPTOR

Most executive subsystems rely on the object manager's default security functionality to manage security descriptors for their objects. The object manager's default security functions use the security descriptor pointer to store security descriptors for such objects. For example, the process manager uses default security, so the object manager stores process and thread security descriptors in the object headers of process and thread objects, respectively. The security descriptor pointer of events, mutexes, and semaphores also store their security descriptors. You can use live kernel debugging to view the security descriptors of these objects once you locate their object header, as outlined in the following steps. (Note that both Process Explorer and `AccessChk` can also show security descriptors for processes.)

1. Start local kernel debugging.
2. Type **!process 0 0 explorer.exe** to obtain process information about Explorer:

[Click here to view code image](#)

```
lkd> !process 0 0 explorer.exe
PROCESS fffffe18304dfd780
    SessionId: 1 Cid: 23e4 Peb: 00c2a000 ParentCid: 2264
    DirBase: 2aa0f6000 ObjectTable: fffcd82c72fcd80 HandleCount:
<Data Not Accessible>
    Image: explorer.exe

PROCESS fffffe1830670a080
    SessionId: 1 Cid: 27b8 Peb: 00950000 ParentCid: 035c
    DirBase: 2cba97000 ObjectTable: fffcd82c7ccc500 HandleCount:
<Data Not Accessible>
    Image: explorer.exe
```

3. If more than one instance of explorer is listed, choose one. (It doesn't matter which.) Type **!object** with the address of the `EPROCESS` in the

output of the previous command as the argument to show the object data structure:

[Click here to view code image](#)

```
lkd> !object fffffe18304dfd780
```

```
Object: fffffe18304dfd780 Type: (ffffe182f7496690) Process
```

```
ObjectHeader: fffffe18304dfd750 (new version)
```

```
HandleCount: 15 PointerCount: 504639
```

4. Type **dt _OBJECT_HEADER** and the address of the object header field from the previous command's output to show the object header data structure, including the security descriptor pointer value:

[Click here to view code image](#)

```
lkd> dt nt!_object_header fffffe18304dfd750
```

```
+0x000 PointerCount : 0n504448
```

```
+0x008 HandleCount : 0n15
```

```
+0x008 NextToFree : 0x00000000'0000000f Void
```

```
+0x010 Lock : _EX_PUSH_LOCK
```

```
+0x018 TypeIndex : 0xe5 "
```

```
+0x019 TraceFlags : 0 "
```

```
+0x019 DbgRefTrace : 0y0
```

```
+0x019 DbgTracePermanent : 0y0
```

```
+0x01a InfoMask : 0x88 "
```

```
+0x01b Flags : 0 "
```

```
+0x01b NewObject : 0y0
```

```
+0x01b KernelObject : 0y0
```

```
+0x01b KernelOnlyAccess : 0y0
```

```
+0x01b ExclusiveObject : 0y0
```

```
+0x01b PermanentObject : 0y0
```

```
+0x01b DefaultSecurityQuota : 0y0
```

```
+0x01b SingleHandleEntry : 0y0
```

```
+0x01b DeletedInline : 0y0
```

```
+0x01c Reserved : 0x30003100
```

```
+0x020 ObjectCreateInfo : 0xfffffe183'09e84ac0
```

`_OBJECT_CREATE_INFORMATION`

`+0x020 QuotaBlockCharged : 0xffffe183'09e84ac0 Void`

`+0x028 SecurityDescriptor : 0xffffcd82'cd0e97ed Void`

`+0x030 Body : _QUAD`

5. Finally, use the debugger's `!sd` command to dump the security descriptor. The security descriptor pointer in the object header uses some of the low-order bits as flags, and these must be zeroed before following the pointer. On 32-bit systems there are three flag bits, so use `& -8` with the security descriptor address displayed in the object header structure, as follows. On 64-bit systems there are four flag bits, so you use `& -10` instead.

[Click here to view code image](#)

```
lkd> !sd 0xffffcd82'cd0e97ed & -10
```

```
->Revision: 0x1
```

```
->Sbz1 : 0x0
```

```
->Control : 0x8814
```

```
    SE_DACL_PRESENT
```

```
    SE_SACL_PRESENT
```

```
    SE_SACL_AUTO_INHERITED
```

```
    SE_SELF_RELATIVE
```

```
->Owner : S-1-5-21-3537846094-3055369412-2967912182-1001
```

```
->Group : S-1-5-21-3537846094-3055369412-2967912182-1001
```

```
->Dacl :
```

```
->Dacl : ->AclRevision: 0x2
```

```
->Dacl : ->Sbz1 : 0x0
```

```
->Dacl : ->AclSize : 0x5c
```

```
->Dacl : ->AceCount : 0x3
```

```
->Dacl : ->Sbz2 : 0x0
```

```
->Dacl : ->Ace[0]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
```

```
->Dacl : ->Ace[0]: ->AceFlags: 0x0
```

```
->Dacl : ->Ace[0]: ->AceSize: 0x24
```

```
->Dacl : ->Ace[0]: ->Mask : 0x001fffff
```

```
->Dacl : ->Ace[0]: ->SID: S-1-5-21-3537846094-3055369412-2967912182-1001
```

```
->Dacl : ->Ace[1]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl : ->Ace[1]: ->AceFlags: 0x0
->Dacl : ->Ace[1]: ->AceSize: 0x14
->Dacl : ->Ace[1]: ->Mask : 0x001fffff
->Dacl : ->Ace[1]: ->SID: S-1-5-18
```

```
->Dacl : ->Ace[2]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl : ->Ace[2]: ->AceFlags: 0x0
->Dacl : ->Ace[2]: ->AceSize: 0x1c
->Dacl : ->Ace[2]: ->Mask : 0x00121411
->Dacl : ->Ace[2]: ->SID: S-1-5-5-0-1745560
```

```
->Sacl :
->Sacl : ->AclRevision: 0x2
->Sacl : ->Sbz1 : 0x0
->Sacl : ->AclSize : 0x1c
->Sacl : ->AceCount : 0x1
->Sacl : ->Sbz2 : 0x0
->Sacl : ->Ace[0]: ->AceType: SYSTEM_MANDATORY_LABEL_ACE_TYPE
->Sacl : ->Ace[0]: ->AceFlags: 0x0
->Sacl : ->Ace[0]: ->AceSize: 0x14
->Sacl : ->Ace[0]: ->Mask : 0x00000003
->Sacl : ->Ace[0]: ->SID: S-1-16-8192
```

The security descriptor contains three access-allowed ACEs: one for the current user (S-1-5-21-3537846094-3055369412-2967912182-1001), one for the System account (S-1-5-18), and the last for the Logon SID (S-1-5-5-0-1745560). The system access control list has one entry (S-1-16-8192) labeling the process as medium integrity level.

ACL assignment

To determine which DACL to assign to a new object, the security system uses the first applicable rule of the following four assignment rules:

1. If a caller explicitly provides a security descriptor when creating the object, the security system applies it to the object. If the object has a name and resides in a container object (for example, a named event object in the \BaseNamedObjects object manager namespace directory), the system merges any inheritable ACEs (ACEs that might propagate from the object's container) into the DACL unless the security descriptor has the `SE_DACL_PROTECTED` flag set, which prevents inheritance.

2. If a caller doesn't supply a security descriptor and the object has a name, the security system looks at the security descriptor in the container in which the new object name is stored. Some of the object directory's ACEs might be marked as inheritable, meaning they should be applied to new objects created in the object directory. If any of these inheritable ACEs are present, the security system forms them into an ACL, which it attaches to the new object. (Separate flags indicate ACEs that should be inherited only by container objects rather than by objects that aren't containers.)

3. If no security descriptor is specified and the object doesn't inherit any ACEs, the security system retrieves the default DACL from the caller's access token and applies it to the new object. Several subsystems on Windows have hard-coded DACLs that they assign on object creation (for example, services, LSA, and SAM objects).

4. If there is no specified descriptor, no inherited ACEs, and no default DACL, the system creates the object with no DACL, which allows everyone (all users and groups) full access to the object. This rule is the same as the third rule, in which a token contains a null default DACL.

The rules the system uses when assigning a SACL to a new object are similar to those used for DACL assignment, with some exceptions:

- Inherited system audit ACEs don't propagate to objects with security descriptors marked with the `SE_SACL_PROTECTED` flag (similar to the `SE_DACL_PROTECTED` flag, which protects DACLs).

- If there are no specified security audit ACEs and there is no inherited SACL, no SACL is applied to the object. This behavior is different from that used to apply default DACLs because tokens don't have a default SACL.

When a new security descriptor containing inheritable ACEs is applied to a container, the system automatically propagates the inheritable ACEs to the security descriptors of child objects. (Note that a security descriptor's DACL doesn't accept inherited DACL ACEs if its `SE_DACL_PROTECTED` flag is enabled, and its SACL doesn't inherit SACL ACEs if the descriptor has the `SE_SACL_PROTECTED` flag set.) The order in which inheritable ACEs are merged with an existing child object's security descriptor is such that any ACEs that were explicitly applied to the ACL are kept ahead of ACEs that the object inherits. The system uses the following rules for propagating inheritable ACEs:

- If a child object with no DACL inherits an ACE, the result is a child object with a DACL containing only the inherited ACE.
- If a child object with an empty DACL inherits an ACE, the result is a child object with a DACL containing only the inherited ACE.
- For objects in Active Directory only, if an inheritable ACE is removed from a parent object, automatic inheritance removes any copies of the ACE inherited by child objects.
- For objects in Active Directory only, if automatic inheritance results in the removal of all ACEs from a child object's DACL, the child object has an empty DACL rather than no DACL.

As you'll soon discover, the order of ACEs in an ACL is an important aspect of the Windows security model.



Inheritance is generally not directly supported by the object stores, such as file systems, the registry, or Active Directory. Windows APIs that support inheritance, including `SetEntriesInAcl`, do so by invoking appropriate functions within the security inheritance support DLL (`%SystemRoot%\System32\Ntmaria.dll`) that know how to traverse those object stores.

Trust ACEs

The advent of protected processes and Protected Processes Light (PPL, discussed in [Chapter 3](#)) created a need for such a process to make objects as accessible by protected processes only. This is important to protect certain resources such as the `KnownDlls` registry key from tampering, even by admin-level code. Such ACEs are specified with well-known SIDs that provide the protection level and signer that is required to obtain access. [Table 7-7](#) shows the SIDs and their level and meaning.

TABLE 7-7 Trust SIDs

A *trust SID* is part of a token object that exists for tokens attached to protected or PPL processes. The higher the SID number, the more powerful the token is. (remember that Protected is higher than Protected Light).

EXPERIMENT: VIEWING TRUST SIDS

In this experiment, you'll look at trust SIDs in tokens of protected processes.

Start local kernel debugging.

List Csrss.exe processes with basic information:

[Click here to view code image](#)

```
lkd> !process 0 1 csrss.exe
PROCESS ffff8188e50b5780
  SessionId: 0 Cid: 0358 Peb: b3a9f5e000 ParentCid: 02ec
  DirBase: 1273a3000 ObjectTable: ffffbe0d829e2040 HandleCount:
<Data Not Accessible>
  Image: csrss.exe
  VadRoot ffff8188e6ccc8e0 Vads 159 Clone 0 Private 324. Modified
4470.
  Locked 0.
  DeviceMap ffffbe0d70c15620
  Token                ffffbe0d829e7060
  ...
PROCESS ffff8188e7a92080
  SessionId: 1 Cid: 03d4 Peb: d5b0de4000 ParentCid: 03bc
  DirBase: 162d93000 ObjectTable: ffffbe0d8362d7c0 HandleCount:
<Data Not Accessible>Modified 462372. Locked 0.
  DeviceMap ffffbe0d70c15620
  Token                ffffbe0d8362d060
  ...
```

Select one of the tokens and show its details:

[Click here to view code image](#)

```
lkd> !token ffffbe0d829e7060
_TOKEN 0xffffbe0d829e7060
TS Session ID: 0
```

User: S-1-5-18

...

Process Token TrustLevelSid: **S-1-19-512-8192**

That's a PPL with a WinTcb signer.

Determining access

Two methods are used for determining access to an object:

- The mandatory integrity check, which determines whether the integrity level of the caller is high enough to access the resource, based on the resource's own integrity level and its mandatory policy.
- The discretionary access check, which determines the access that a specific user account has to an object.

When a process tries to open an object, the integrity check takes place before the standard Windows DACL check in the kernel's `SeAccessCheck` function because it is faster to execute and can quickly eliminate the need to perform the full discretionary access check. Given the default integrity policies in its access token (`TOKEN_MANDATORY_NO_WRITE_UP` and `TOKEN_MANDATORY_NEW_PROCESS_MIN`, described previously), a process can open an object for write access if its integrity level is equal to or higher than the object's integrity level and the DACL also grants the process the accesses it desires. For example, a low-integrity-level process cannot open a medium-integrity-level process for write access, even if the DACL grants the process write access.

With the default integrity policies, processes can open any object—with the exception of process, thread, and token objects—for read access as long as the object's DACL grants them read access. That means a process running at low integrity level can open any files accessible to the user account in which it's running. Protected Mode Internet Explorer uses integrity levels to help prevent malware that infects it from modifying

user account settings, but it does not stop malware from reading the user’s documents.

Recall that process, thread, and token objects are exceptions because their integrity policy also includes No-Read-Up. That means a process integrity level must be equal to or higher than the integrity level of the process or thread it wants to open, and the DACL must grant it the access it wants for an attempt to open it to succeed. Assuming the DACLs allow the desired access, **Table 7-8** shows the types of access that processes running at various integrity levels have to other processes and objects.

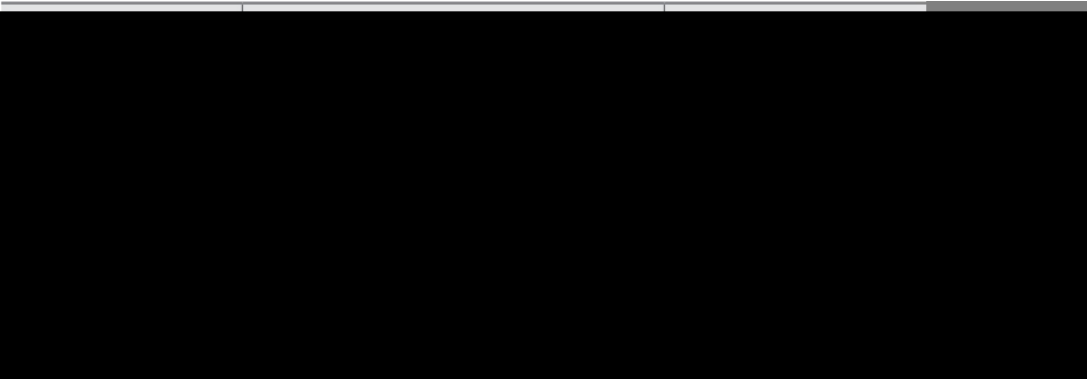


TABLE 7-8 Accessing objects and processes based on integrity level



NOTE

The read access to a process described in this section means full read access, such as reading the contents of the process address space. No-Read-Up does not prevent opening a higher-integrity-level process from a lower one for a more limited access, such as `PROCESS_QUERY_LIMITED_INFORMATION`, which provides only basic information about the process.

USER INTERFACE PRIVILEGE ISOLATION

The Windows messaging subsystem also honors integrity levels to implement User Interface Privilege Isolation (UIPI). The subsystem does this by preventing a process from sending window messages to the windows owned by a process having a higher integrity level, with the following informational messages being exceptions:

- `WM_NULL`
 - `WM_MOVE`
 - `WM_SIZE`
 - `WM_GETTEXT`
 - `WM_GETTEXTLENGTH`
 - `WM_GETHOTKEY`
 - `WM_GETICON`
 - `WM_RENDERFORMAT`
 - `WM_DRAWCLIPBOARD`
-

■ WM_CHANGECHAIN

■ WM_THEMECHANGED

This use of integrity levels prevents standard user processes from driving input into the windows of elevated processes or from performing a shatter attack (such as sending the process malformed messages that trigger internal buffer overflows, which can lead to the execution of code at the elevated process's privilege level). UIPI also blocks window hooks (`SetWindowsHookEx` API) from affecting the windows of higher-integrity-level processes so that a standard user process can't log the keystrokes the user types into an administrative application, for example. Journal hooks are also blocked in the same way to prevent lower-integrity-level processes from monitoring the behavior of higher-integrity-level processes.

Processes (running with medium or higher integrity level only) can choose to allow additional messages to pass the guard by calling the `ChangeWindowMessageFilterEx` API. This function is typically used to add messages required by custom controls to communicate outside native common controls in Windows. An older API, `ChangeWindowMessageFilter`, performs a similar function, but it is per-process rather than per-window. With `ChangeWindowMessageFilter`, it is possible for two custom controls inside the same process to be using the same internal window messages, which could lead to one control's potentially malicious window message to be allowed through, simply because it happens to be a query-only message for the other custom control.

Because accessibility applications such as the On-Screen Keyboard (`Osk.exe`) are subject to UIPI's restrictions (which would require the accessibility application to be executed for each kind of visible integrity-level process on the desktop), these processes can enable UI access. This flag can be present in the manifest file of the image and will run the process at a slightly higher integrity level than medium (between `0x2000` and `0x3000`) if launched from a standard user account, or at high integrity level if launched from an Administrator account. Note

that in the second case, an elevation request won't actually be displayed. For a process to set this flag, its image must also be signed and in one of several secure locations, including %SystemRoot% and %ProgramFiles%.

After the integrity check is complete, and assuming the mandatory policy allows access to the object based on the caller's integrity, one of two algorithms is used for the discretionary check to an object, which will determine the outcome of the access check:

- Determine the maximum access allowed to the object, a form of which is exported to user mode using the AuthZ API (described in the section “[The AuthZ API](#)” later in this chapter) or the older `GetEffectiveRightsFromAcl` function. This is also used when a program specifies a desired access of `MAXIMUM_ALLOWED`, which is what the legacy APIs that don't have a desired access parameter use.
- Determine whether a specific desired access is allowed, which can be done with the Windows `AccessCheck` function or the `AccessCheckByType` function.

The first algorithm examines the entries in the DACL as follows:

1. If the object has no DACL (a null DACL), the object has no protection and the security system grants all access, unless the access is from an AppContainer process (discussed in the “[AppContainers](#)” section later in this chapter), which means access is denied.
2. If the caller has the take-ownership privilege, the security system grants write-owner access before examining the DACL. (Take-ownership privilege and write-owner access are explained in a moment.)
3. If the caller is the owner of the object, the system looks for an `OWNER_RIGHTS` SID and uses that SID as the SID for the next steps. Otherwise, read-control and write-DACL access rights are granted.

4. For each access-denied ACE that contains a SID that matches one in the caller's access token, the ACE's access mask is removed from the granted-access mask.

5. For each access-allowed ACE that contains a SID that matches one in the caller's access token, the ACE's access mask is added to the granted-access mask being computed, unless that access has already been denied.

When all the entries in the DACL have been examined, the computed granted-access mask is returned to the caller as the maximum allowed access to the object. This mask represents the total set of access types that the caller will be able to successfully request when opening the object.

The preceding description applies only to the kernel-mode form of the algorithm. The Windows version implemented by

`GetEffectiveRightsFromAcl` differs in that it doesn't perform step 2, and it considers a single user or group SID rather than an access token.

OWNER RIGHTS

Because owners of an object can normally override the security of an object by always being granted read-control and write-DACL rights, a specialized method of controlling this behavior is exposed by Windows: the Owner Rights SID.

The Owner Rights SID exists for two main reasons:

- **To improve service hardening in the operating system** Whenever a service creates an object at run time, the Owner SID associated with that object is the account the service is running in (such as local system or local service) and not the actual service SID. This means that any other service in the same account would have access to the object by being an owner. The Owner Rights SID prevents that unwanted behavior.

- **To allow more flexibility for specific usage scenarios** For example, suppose an administrator wants to allow users to create files and folders but not to modify the ACLs on those objects. (Users could inadvertently or maliciously grant access to those files or folders to unwanted accounts.) By using an inheritable Owner Rights SID, the users can be prevented from editing or even viewing the ACL on the objects they create. A second usage scenario relates to group changes. Suppose an employee has been part of some confidential or sensitive group, has created several files while a member of that group, and has now been removed from the group for business reasons. Because that employee is still a user, he could continue accessing the sensitive files.

The second algorithm is used to determine whether a specific access request can be granted based on the caller's access token. Each open function in the Windows API that deals with securable objects has a parameter that specifies the desired access mask, which is the last component of the security equation. To determine whether the caller has access, the following steps are performed:

1. If the object has no DACL (a null DACL), the object has no protection and the security system grants the desired access.
2. If the caller has the take-ownership privilege, the security system grants write-owner access if requested and then examines the DACL. However, if write-owner access was the only access requested by a caller with take-ownership privilege, the security system grants that access and never examines the DACL.
3. If the caller is the owner of the object, the system looks for an `OWNER_RIGHTS SID` and uses that SID as the SID for the next steps. Otherwise, read-control and write-DACL access rights are granted. If these rights were the only access rights that the caller requested, access is granted without examining the DACL.
4. Each ACE in the DACL is examined from first to last. An ACE is processed if one of the following conditions is satisfied:
 - The ACE is an access-deny ACE, and the SID in the ACE matches an enabled SID (SIDs can be enabled or disabled) or a deny-only SID in the caller's access token.
 - The ACE is an access-allowed ACE, and the SID in the ACE matches an enabled SID in the caller's token that isn't of type deny-only.
 - It is the second pass through the descriptor for restricted-SID checks, and the SID in the ACE matches a restricted SID in the caller's access token.
 - The ACE isn't marked as inherit-only.
5. If it is an access-allowed ACE, the rights in the access mask in the ACE that were requested are granted. If all the requested access rights have been granted, the access check succeeds. If it is an access-denied ACE and any of the requested access rights are in the denied-access rights, access is denied to the object.

6. If the end of the DACL is reached and some of the requested access rights still haven't been granted, access is denied.

7. If all accesses are granted but the caller's access token has at least one restricted SID, the system rescans the DACL's ACEs looking for ACEs with access-mask matches for the accesses the user is requesting and a match of the ACE's SID with any of the caller's restricted SIDs. Only if both scans of the DACL grant the requested access rights is the user granted access to the object.

The behavior of both access-validation algorithms depends on the relative ordering of allow and deny ACEs. Consider an object with only two ACEs: one that specifies that a certain user is allowed full access to an object and one that denies the user access. If the allow ACE precedes the deny ACE, the user can obtain full access to the object, but if the order is reversed, the user cannot gain any access to the object.

Several Windows functions, such as `SetSecurityInfo` and `SetNamedSecurityInfo`, apply ACEs in the preferred order of explicit deny ACEs preceding explicit allow ACEs. For example, the security editor dialog boxes with which you edit permissions on NTFS files and registry keys use these functions. `SetSecurityInfo` and `SetNamedSecurityInfo` also apply ACE inheritance rules to the security descriptor on which they are applied.

Figure 7-9 shows an example of access validation demonstrating the importance of ACE ordering. In the example, access is denied to a user wanting to open a file even though an ACE in the object's DACL grants the access. This is because the ACE denying the user access (by virtue of the user's membership in the Writers group) precedes the ACE granting access.

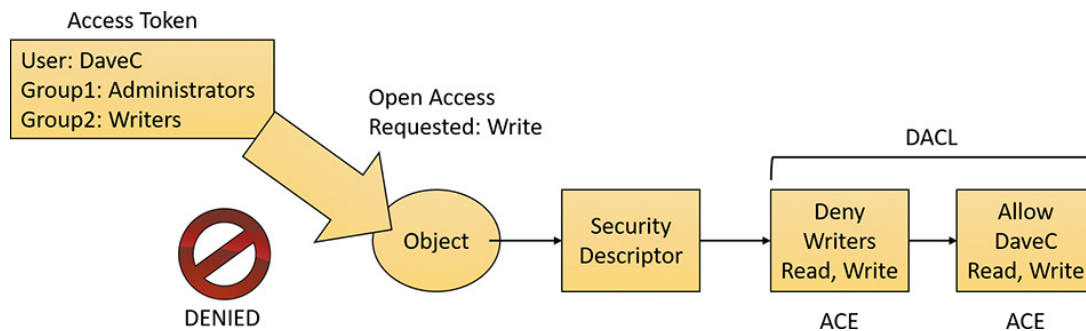


FIGURE 7-9 Access-validation example.

As stated, because it wouldn't be efficient for the security system to process the DACL every time a process uses a handle, the SRM makes this access check only when a handle is opened, not each time the handle is used. Thus, once a process successfully opens a handle, the security system can't revoke the access rights that have been granted, even if the object's DACL changes. Also keep in mind that because kernel-mode code uses pointers rather than handles to access objects, the access check isn't performed when the operating system uses objects. In other words, the Windows executive trusts itself (and all loaded drivers) in a security sense.

The fact that an object's owner is always granted write-DACL access to an object means that users can never be prevented from accessing the objects they own. If, for some reason, an object had an empty DACL (no access), the owner would still be able to open the object with write-DACL access and then apply a new DACL with the desired access permissions.

A WARNING REGARDING THE GUI SECURITY EDITORS

When you use the GUI permissions editors to modify security settings on a file, a registry, or an Active Directory object, or on another securable object, the main security dialog box shows you a potentially misleading view of the security that's applied to the object. If you allow Full Control to the Everyone group and deny the Administrator group Full Control, the list might lead you to believe that the Everyone group access-allowed ACE precedes the Administrator deny ACE because that's the order in which they appear. However, as we've said, the editors place deny ACEs before allow ACEs when they apply the ACL to the object.

The Permissions tab of the Advanced Security Settings dialog box shows the order of ACEs in the DACL. However, even this dialog box can be confusing because a complex DACL can have deny ACEs for various accesses followed by allow ACEs for other access types.

The only definitive way to know what access a particular user or group will have to an object (other than having that user or a member of the group try to access the object) is to use the **Effective Access** tab of the dialog box that is displayed when you click the **Advanced** button in the Properties dialog box. Enter the name of the user or group you want to check and the dialog box shows you what permissions they are allowed for the object.

Dynamic Access Control

The discretionary access control mechanism discussed in previous sections has existed since the first Windows NT version and is useful in many scenarios. There are scenarios, however, where this scheme is not flexible enough. For example, consider a requirement that users accessing a shared file should be allowed to do so if they are using a computer in the workplace, but should not be allowed if accessing the file from their computer at home. There is no way to specify such a condition using an ACE.

Windows 8 and Server 2012 introduced Dynamic Access Control (DAC), a flexible mechanism that can be used to define rules based on custom attributes defined in Active Directory. DAC does not replace the existing mechanism, but adds to it. This means that for an operation to be allowed, both DAC and the classic DACL must grant the permission.

Figure 7-10 shows the main aspects of Dynamic Access Control.

FIGURE 7-10 Dynamic Access Control components.

A claim is any piece of information about a user, device (computer in a domain), or resource (generic attribute) that has been published by a domain controller. Examples of valid claims are a user's title and department classification of a file. Any combination of claims can be used in expressions for building rules. These rules collectively become the central access policy.

DAC configuration is done in Active Directory and pushed through policy. The Kerberos tickets protocol has been enhanced to support authenticated transport of user and device claims (known as *Kerberos armoring*).

The AuthZ API

The AuthZ Windows API provides authorization functions and implements the same security model as the security reference monitor (SRM), but it implements the model totally in user mode in the %SystemRoot%\System32\Authz.dll library. This gives applications that want to protect their own private objects, such as database tables, the ability to leverage the Windows security model without incurring the cost of user mode-to-kernel mode transitions that they would make if they relied on the SRM.

The AuthZ API uses standard security descriptor data structures, SIDs, and privileges. Instead of using tokens to represent clients, AuthZ uses `AUTHZ_CLIENT_CONTEXT`. AuthZ includes user-mode equivalents of all access-check and Windows security functions—for example,

`AuthzAccessCheck` is the AuthZ version of the `AccessCheck` Windows API that uses the `SeAccessCheck` SRM function.

Another advantage available to applications that use AuthZ is that they can direct AuthZ to cache the results of security checks to improve subsequent checks that use the same client context and security descriptor. AuthZ is fully documented in the Windows SDK.

This type of access checking, using a SID and security group membership in a static, controlled environment, is known as *Identity-Based Access Control (IBAC)*, and it requires that the security system know the identity of every possible accessor when the DACL is placed in an object's security descriptor.

Windows includes support for *Claims Based Access Control (CBAC)*, where access is granted not based upon the accessor's identity or group membership, but upon arbitrary attributes assigned to the accessor and stored in the accessor's access token. Attributes are supplied by an attribute provider, such as AppLocker. The CBAC mechanism provides many benefits, including the ability to create a DACL for a user whose identity is not yet known or dynamically calculated user attributes. The CBAC ACE (also known as a conditional ACE) is stored in a `*-callback` ACE structure, which is essentially private to AuthZ and is ignored by the system `SeAccessCheck` API. The kernel-mode routine `SeSrpAccessCheck` does not understand conditional ACEs, so only applications calling the AuthZ APIs can make use of CBAC. The only system component that makes use of CBAC is AppLocker, for setting attributes such as path or publisher. Third-party applications can make use of CBAC by taking advantage of the CBAC AuthZ APIs.

Using CBAC security checks allows powerful management policies, such as the following:

- Run only applications approved by the corporate IT department.
- Allow only approved applications to access your Microsoft Outlook contacts or calendar.

- Allow only people in a particular building on a specific floor to access printers on that floor.
- Allow access to an intranet website only to full-time employees (as opposed to contractors).

Attributes can be referenced in what is known as a *conditional ACE*, where the presence, absence, or value of one or more attributes is checked. An attribute name can contain any alphanumeric Unicode characters, as well as the following characters: colon (:), forward slash (/), and underscore (_). The value of an attribute can be one of the following: 64-bit integer, Unicode string, byte string, or array.

Conditional ACEs

The format of SDDL strings has been expanded to support ACEs with conditional expressions. The new format of an SDDL string is this:

```
AceType;AceFlags;Rights;ObjectGuid;InheritObjectGuid;  
AccountSid;(ConditionalExpression) .
```

The AceType for a conditional ACE is either XA (for SDDL_CALLBACK_ACCESS_ALLOWED) or XD (for SDDL_CALLBACK_ACCESS_DENIED). Note that ACEs with conditional expressions are used for claims-type authorization (specifically, the AuthZ APIs and AppLocker) and are not recognized by the object manager or file systems.

A conditional expression can include any of the elements shown in **Table 7-9**.

TABLE 7-9 Acceptable Elements for a Conditional Expression

A conditional ACE can contain any number of conditions. It is ignored if the resultant evaluation of the condition is false or applied if the result is true. A conditional ACE can be added to an object using the `AddConditionalAce` API and checked using the `AuthzAccessCheck` API.

A conditional ACE could specify that access to certain data records within a program should be granted only to a user who meets the following criteria (for example):

- Holds the `Role` attribute, with a value of `Architect`, `Program Manager`, or `Development Lead`, and the `Division` attribute with a value of `Windows`
- Whose `ManagementChain` attribute contains the value `John Smith`
- Whose `CommissionType` attribute is `Officer` and whose `PayGrade` attribute is greater than `6` (that is, the rank of General Officer in the US military)

Windows does not include tools to view or edit conditional ACEs.

Account rights and privileges

Many operations performed by processes as they execute cannot be authorized through object access protection because they do not involve interaction with a particular object. For example, the ability to bypass security checks when opening files for backup is an attribute of an account, not of a particular object. Windows uses both privileges and account rights to allow a system administrator to control what accounts can perform security-related operations.

A *privilege* is the right of an account to perform a particular system-related operation, such as shutting down the computer or changing the system time. An *account right* grants or denies the account to which it's assigned the ability to perform a particular type of logon, such as a local logon or interactive logon, to a computer.

A system administrator assigns privileges to groups and accounts using tools such as the Active Directory Users and Groups MMC snap-in for domain accounts or the Local Security Policy editor (%SystemRoot%\System32\secpol.msc). **Figure 7-11** shows the User Rights Assignment configuration in the Local Security Policy editor, which displays the complete list of privileges and account rights available on Windows. Note that the tool makes no distinction between privileges and account rights. However, you can differentiate between them: Any user right that does not contain the words *log on* is an account privilege.

FIGURE 7-11 Local Security Policy editor user rights assignment.

Account rights

Account rights are not enforced by the SRM, nor are they stored in tokens. The function responsible for logon is `LsaLogonUser`. Winlogon, for example, calls the `LogonUser` API when a user logs on interactively to a computer, and `LogonUser` calls `LsaLogonUser`. `LogonUser` takes a parameter that indicates the type of logon being performed, which includes interactive, network, batch, service, and Terminal Server client.

In response to logon requests, the Local Security Authority (LSA) retrieves account rights assigned to a user from the LSA policy database at the time that a user attempts to log on to the system. The LSA checks

the logon type against the account rights assigned to the user account logging on and denies the logon if the account does not have the right that permits the logon type or it has the right that denies the logon type. **Table 7-10** lists the user rights defined by Windows.

TABLE 7-10 Account rights

Windows applications can add and remove user rights from an account by using the `LsaAdd-AccountRights` and `LsaRemoveAccountRights` functions, and they can determine what rights are assigned to an account with `LsaEnumerateAccountRights`.

Privileges

The number of privileges defined by the operating system has grown over time. Unlike user rights, which are enforced in one place by the LSA, different privileges are defined by different components and enforced by those components. For example, the debug privilege, which allows a process to bypass security checks when opening a handle to another process with the `OpenProcess` Windows API, is checked for by the process manager.

Table 7-11 is a full list of privileges and describes how and when system components check for them. Each privilege has a macro defined in the SDK headers, in the form `SE_privilege_NAME`, where *privilege* is a privilege constant—for example, `SE_DEBUG_NAME` for the debug privilege. These macros are defined as strings that start with `Se` and end with `Privilege`, as in `SeDebugPrivilege`. This may seem to indicate that privileges are identified by strings, but in fact they are identified by LUIDs, which naturally are unique for the current boot. Every access to a privilege needs to lookup the correct LUID by calling the

`LookupPrivilegeValue` function. Note, however, that Ntdll and kernel code can identify privileges with integer constants directly without going through a LUID.

TABLE 7-11 Privileges

When a component wants to check a token to see whether a privilege is present, it uses the `Privilege-Check` or `LsaEnumerateAccountRights` APIs if running in user mode and `SeSinglePrivilegeCheck` or `SePrivilegeCheck` if running in kernel mode. The privilege-related APIs are not account-right aware, but the account-right APIs are privilege-aware.

Unlike account rights, privileges can be enabled and disabled. For a privilege check to succeed, the privilege must be in the specified token and it must be enabled. The idea behind this scheme is that privileges should be enabled only when their use is required so that a process cannot inadvertently perform a privileged security operation. Enabling or disabling privileges can be done with the `AdjustTokenPrivileges` function.

EXPERIMENT: SEEING A PRIVILEGE GET ENABLED

By following these steps, you can see that the Date and Time Control Panel applet enables the `SeTimeZonePrivilege` privilege in response to you using its interface to change the time zone of the computer (Windows 10):

1. Run Process Explorer elevated.
2. Right-click the clock in the system tray in the taskbar and choose **Adjust Date/Time**. Alternatively, open the Settings app and search for **time** to open the Date and Time settings page.
3. Right-click the **SystemSettings.exe** process in Process Explorer and choose **Properties**. Then click the **Security** tab in the Properties dialog box. You should see that the `SeTimeZonePrivilege` privilege is disabled.

4. Change the time zone, close the Properties dialog box, and then open it again. On the Security tab, you should now see that the `SeTimeZonePrivilege` privilege is enabled:

EXPERIMENT: THE BYPASS TRAVERSE CHECKING PRIVILEGE

If you are a systems administrator, you must be aware of the Bypass Traverse Checking privilege (internally called `SeNotifyPrivilege`) and its implications. This experiment demonstrates that not understanding its behavior can lead to improperly applied security.

1. Create a folder and, within that folder, a new text file with some sample text.

2. Navigate in Explorer to the new file, open its Properties dialog box, and click the **Security** tab.
3. Click the **Advanced** button.
4. Deselect the **Inheritance** check box.
5. Select **Copy** when you are prompted as to whether you want to remove or copy inherited permissions.
6. Modify the security of the new folder so that your account does not have any access to the folder. To do so, select your account and check all the **Deny** boxes in the permissions list.
7. Run Notepad. Then open the **File** menu, choose **Open**, and browse to the new directory in the dialog box that appears. You should be denied access to the directory.
8. In the File Name field of the Open dialog box, type the full path of the new file. The file should open.

If your account does not have the Bypass Traverse Checking privilege, NTFS performs an access check on each directory of the path to a file when you try to open a file, which results in you being denied access to the file in this example.

Super privileges

Several privileges are so powerful that a user to which they are assigned is effectively a “super user” who has full control over a computer. These privileges can be used in an infinite number of ways to gain unauthorized access to otherwise off-limit resources and to perform unauthorized operations. However, we’ll focus on using the privilege to execute code that grants the user privileges not assigned to the user, with the knowledge that this capability can be leveraged to perform any operation on the local machine that the user desires.

This section lists the privileges and discusses some of the ways they can be exploited. Other privileges, such as Lock Pages in Physical Memory (`SeLockMemoryPrivilege`), can be exploited for denial-of-service attacks on a system, but these are not discussed. Note that on systems with UAC enabled, these privileges will be granted only to applications running at high integrity level or higher, even if the account possesses them:

■ **Debug programs (`SeDebugPrivilege`)** A user with this privilege can open any process on the system (except for a protected process) without regard to the security descriptor present on the process. For example, the user could implement a program that opens the Lsass process, copy executable code into its address space, and then inject a thread with the `CreateRemoteThread` Windows API to execute the injected code in a more-privileged security context. The code could grant the user additional privileges and group memberships.

■ **Take ownership (`SeTakeOwnershipPrivilege`)** This privilege allows a holder to take ownership of any securable object (even protected processes and threads) by writing his own SID into the owner field of the object's security descriptor. Recall that an owner is always granted permission to read and modify the DACL of the security descriptor, so a process with this privilege could modify the DACL to grant itself full access to the object and then close and reopen the object with full access. This would allow the owner to see sensitive data and to even replace system files that execute as part of normal system operation, such as Lsass, with his own programs that grant a user elevated privileges.

■ **Restore files and directories (`SeRestorePrivilege`)** A user assigned this privilege can replace any file on the system with her own. She could exploit this power by replacing system files as described in the preceding paragraph.

■ **Load and unload device drivers (`SeLoadDriverPrivilege`)** A malicious user could use this privilege to load a device driver into the system. Device drivers are considered trusted parts of the operating sys-

tem that can execute within it with System account credentials, so a driver could launch privileged programs that assign the user other rights.

■ **Create a token object (SeCreateTokenPrivilege)** This privilege can be used in the obvious way to generate tokens that represent arbitrary user accounts with arbitrary group membership and privilege assignment.

■ **Act as part of operating system (SeTcbPrivilege)**

`LsaRegisterLogonProcess`, the function a process calls to establish a trusted connection to Lsass, checks for this privilege. A malicious user with this privilege can establish a trusted-Lsass connection and then execute `LsaLogonUser`, a function used to create new logon sessions.

`LsaLogonUser` requires a valid user name and password and accepts an optional list of SIDs that it adds to the initial token created for a new logon session. The user could therefore use her own user name and password to create a new logon session that includes the SIDs of more privileged groups or users in the resulting token.



The use of an elevated privilege does not extend past the machine boundary to the network because any interaction with another computer requires authentication with a domain controller and validation of domain passwords.

Domain passwords are not stored on a computer either in plaintext or encrypted form, so they are not accessible to malicious code.

Access tokens of processes and threads

Figure 7-12 brings together the concepts covered so far in this chapter by illustrating the basic process and thread security structures. In the figure, notice that the process object and the thread objects have ACLs, as do the access token objects themselves. Also in this figure, thread 2 and thread 3 each have an impersonation token, whereas thread 1 uses the default process access token.

FIGURE 7-12 Process and thread security structures.

Security auditing

The object manager can generate audit events as a result of an access check, and Windows functions available to user applications can generate them directly. Kernel-mode code is always allowed to generate an audit event. Two privileges, `SeSecurityPrivilege` and `SeAuditPrivilege`, relate to auditing. A process must have the `SeSecurityPrivilege` privilege to manage the security event log and to view or set an object's SACL. Processes that call audit system services, however, must have the `SeAuditPrivilege` privilege to successfully generate an audit record.

The audit policy of the local system controls the decision to audit a particular type of security event. The audit policy, also called the Local Security Policy, is one part of the security policy Lsass maintains on the local system. It is configured with the Local Security Policy editor as shown in **Figure 7-13**. The audit policy configuration (both the basic settings under Local Policies and the Advanced Audit Policy Configuration) is stored in the registry as a bitmapped value in the `HKEY_LOCAL_MACHINE\SECURITY\Policy\PolAdtEv` key.

FIGURE 7-13 Local Security Policy editor audit policy configuration.

Lsass sends messages to the SRM to inform it of the auditing policy at system-initialization time and when the policy changes. Lsass is respon-

sible for receiving audit records generated based on the audit events from the SRM, editing the records, and sending them to the event logger. Lsass (instead of the SRM) sends these records because it adds pertinent details, such as the information needed to more completely identify the process that is being audited.

The SRM sends audit records via its ALPC connection to Lsass. The event logger then writes the audit record to the security event log. In addition to audit records the SRM passes, both Lsass and the SAM generate audit records that Lsass sends directly to the event logger, and the AuthZ APIs allow for applications to generate application-defined audits. **Figure 7-14** depicts this overall flow.

FIGURE 7-14 Flow of security audit records.

Audit records are put on a queue to be sent to the LSA as they are received. They are not submitted in batches. The audit records are moved from the SRM to the security subsystem in one of two ways. If the audit record is small (less than the maximum ALPC message size), it is sent as an ALPC message. The audit records are copied from the address space of the SRM to the address space of the Lsass process. If the audit record is large, the SRM uses shared memory to make the message available to Lsass and simply passes a pointer in an ALPC message.

Object access auditing

An important use of the auditing mechanism in many environments is to maintain a log of accesses to secured objects—in particular, files. To do this, the Audit object access policy must be enabled, and there must be audit ACEs in system access control lists that enable auditing for the objects in question.

When an accessor attempts to open a handle to an object, the SRM first determines whether the attempt is allowed or denied. If object access auditing is enabled, the SRM then scans the system ACL of the object. There are two types of audit ACEs: access allowed and access denied. An audit ACE must match any of the security IDs held by the accessor, it must match any of the access methods requested, and its type (access allowed or access denied) must match the result of the access check to generate an object access audit record.

Object access audit records include not just the fact of access allowed or denied, but also the reason for the success or failure. This “reason for access” reporting generally takes the form of an access control entry, specified in Security Descriptor Definition Language (SDDL), in the audit record. This allows for a diagnosis of scenarios in which an object to which you believe access should be denied is being permitted, or vice versa, by identifying the specific access control entry that caused the attempted access to succeed or fail.

As was shown in [Figure 7-13](#), object access auditing is disabled by default (as are all other auditing policies).

You can observe object access auditing by following these steps:

1. In Explorer, navigate to a file to which you would normally have access (such as a text file), open its Properties dialog box, click the **Security** tab, and then select the **Advanced** settings.

2. Click the **Auditing** tab and click through the administrative privileges warning. The resulting dialog box allows you to add auditing of access control entries to the file's system access control list.

3. Click the **Add** button and choose **Select a Principal**.

4. In the resulting Select User or Group dialog box, enter your own user name or a group to which you belong, such as **Everyone**. Click **Check Names** and then click **OK**. This presents a dialog box for creating an Audit access control entry for this user or group for this file.

5. Click **OK** three times to close the file Properties dialog box.
6. In Explorer, double-click the file to open it with its associated program (for example, Notepad for a text file).
7. Click the **Start** menu, type **event**, and choose **Event Viewer**.
8. Navigate to the Security log. Note that there is no entry for access to the file. This is because the audit policy for object access is not yet configured.
9. In the Local Security Policy editor, navigate to **Local Policies** and choose **Audit Policy**.
10. Double-click **Audit Object Access** and click **Success** to enable auditing of successful access to files.
11. In Event Viewer, click **Action** (from the menu) and **Refresh**. Note that the changes to audit policy resulted in audit records.
12. In Explorer, double-click the file to open it again.
13. In Event Viewer, click **Action** and **Refresh**. Note that several file access audit records are now present.

14. Find one of the file access audit records for event ID 4656. This shows up as “a handle to an object was requested.” (You can use the Find option to search for the file name you opened.)

15. Scroll down in the text box to find the Access Reasons section. The following example shows that two access methods, `READ_CONTROL`, `SYNCHRONIZE`, and `ReadAttributes`, `ReadEA` (extended attributes), and `ReadData` were requested. `READ_CONTROL` was granted because the accessor was the owner of the file. The others were granted because of the indicated access control entry.

Global audit policy

In addition to object-access ACEs on individual objects, a global audit policy can be defined for the system that enables object-access auditing for all file-system objects, all registry keys, or for both. A security auditor can therefore be certain that the desired auditing will be performed, without having to set or examine SACs on all the individual objects of interest.

An administrator can set or query the global audit policy via the `AuditPol` command with the `/resourceSACL` option. This can also be done programmatically by calling the `AuditSetGlobalSacl` and `AuditQueryGlobalSacl` APIs. As with changes to objects' SACLs, changing these global SACLs requires `SeSecurityPrivilege`.

You can use the `AuditPol` command to enable global audit policy.

1. If you didn't already do so in the previous experiment, open the Local Security Policy editor, navigate to the Audit Policy settings (refer to [Figure 7-13](#)), double-click **Audit Object Access**, and enable auditing for both success and failure. On most systems, SACLs specifying object access auditing are uncommon, so few if any object-access audit records will be produced at this point.

2. In an elevated command prompt window, enter the following command. This will produce a summary of the commands for setting and querying global audit policy.

```
C:\> auditpol /resourceSACL
```

3. In the same elevated command prompt window, enter the following commands. On a typical system, each of these commands will report that no global SACL exists for the respective resource type. (Note that the `File` and `Key` keywords are case-sensitive.)

[Click here to view code image](#)

```
C:\> auditpol /resourceSACL /type:File /view
```

```
C:\> auditpol /resourceSACL /type:Key /view
```

4. In the same elevated command prompt window, enter the following command. This will set a global audit policy such that all attempts to open files for write access (`FW`) by the indicated user will result in audit records, whether the open attempts succeed or fail. The user name can be a specific user name on the system, a group such as Everyone, a domain-qualified user name such as domainname\username, or a SID.

[Click here to view code image](#)

```
C:\> auditpol /resourceSACL /set /type:File /user:yourusername /success  
/failure /access:FW
```

5. While running under the user name indicated, use Explorer or other tools to open a file. Then look at the security log in the system event log to find the audit records.

6. At the end of the experiment, use the `auditpol` command to remove the global SACL you created in step 4, as follows:

[Click here to view code image](#)

```
C:\> auditpol /resourceSACL /remove /type:File /user:yourusername
```

The global audit policy is stored in the registry as a pair of system access control lists in HKLM\SECURITY\Policy\GlobalSaclNameFile and HKLM\SECURITY\Policy\GlobalSaclNameKey. You can examine these keys by running Regedit.exe under the System account, as described in the “[Security system components](#)” section earlier in this chapter. These keys will not exist until the corresponding global SACLs have been set at least once.

The global audit policy cannot be overridden by SACLs on objects, but object-specific SACLs can allow for additional auditing. For example, global audit policy could require auditing of read access by all users to all files, but SACLs on individual files could add auditing of write access to those files by specific users or by more specific user groups.

Global audit policy can also be configured via the Local Security Policy editor in the Advanced Audit Policy settings, described in the next section.

Advanced Audit Policy settings

In addition to the Audit Policy settings described previously, the Local Security Policy editor offers a much more fine-grained set of audit controls under the Advanced Audit Policy Configuration heading, shown in [Figure 7-15](#).

FIGURE 7-15 The Local Security Policy editor's Advanced Audit Policy Configuration settings.

Each of the nine audit policy settings under Local Policies (refer to [Figure 7-13](#)) maps to a group of settings here that provide more detailed control. For example, while the Audit Object Access settings under Local Policies allow access to all objects to be audited, the settings here allow auditing of access to various types of objects to be controlled individually. Enabling one of the audit policy settings under Local Policies implicitly enables all the corresponding advanced audit policy events, but if finer control over the contents of the audit log is desired, the advanced settings can be set individually. The standard settings then become a product of the advanced settings. However, this is not visible in the Local Security Policy editor. Attempts to specify audit settings by using both the basic and the advanced options can cause unexpected results.

You can use the Global Object Access Auditing option under Advanced Audit Policy Configuration to configure the global SACLs described in the previous section, using a graphical interface identical to that seen in Explorer or the Registry editor for security descriptors in the file system or the registry.

AppContainers

Windows 8 introduced a new security sandbox called an *AppContainer*. Although it was created primarily to host UWP processes, AppContainers can actually be used for “normal” processes as well (although there is no built-in tool to do that). This section will mostly cover the attributes of packaged AppContainers, which is the term that refers to AppContainers associated with UWP processes and their resulting .Appx format. A complete treatment of UWP apps is beyond the scope of this chapter. You can find more information in [Chapter 3](#) of this book, and in Chapters 8 and 9 in Part 2. Here we’ll concentrate on the security aspects of AppContainers and their typical usage as hosts of UWP apps.



NOTE

The term *Universal Windows Platform (UWP) app* is the latest used to describe processes that host the Windows Runtime. Older names include *immersive app*, *modern app*, *metro app*, and sometimes simply *Windows app*. The *Universal* part indicates the ability of such apps to be deployed and run on various Windows 10 editions and form factors, from IoT core, to mobile, to desktop, to Xbox, to HoloLens. However, they are essentially the same as the ones first introduced in Windows 8. Therefore, the concept of AppContainers discussed in this section is relevant to Windows 8 and later versions of Windows. Note that Universal Application Platform (UAP) is sometimes used instead of UWP; it's the same thing.



NOTE

The original codename for AppContainer was LowBox. You may see this term come up in many of the API names and data structures throughout this section. They refer to the same concept.

Overview of UWP apps

The mobile device revolution established new ways of obtaining and running software. Mobile devices normally get their applications from a central store, with automatic installation and updates, all with little user intervention. Once a user selects an app from the store, she can see the permissions the app requires to function correctly. These permissions are called *capabilities* and are declared as part of the package when it's submitted to the store. This way, the user can decide whether these capabilities are acceptable.

Figure 7-16 shows an example of a capabilities list for a UWP game (*Minecraft*, Windows 10 beta edition). The game requires internet access as a client and as a server and access to the local home or work network. Once the user downloads the game, she implicitly agrees the game may exercise these capabilities. Conversely, the user can be confident that the game uses *only* those capabilities. That is, there is no way the game could use other unapproved capabilities, such as accessing the camera on the device.

FIGURE 7-16 Part of an app's page in the store, showing capabilities, among other things.

To get a sense of the differences between UWP apps and desktop (classic) apps at a high level, consult **Table 7-12**. From a developer's perspective, the Windows platform can be seen as shown in **Figure 7-17**.

TABLE 7-12 High-level comparison of UWP and desktop apps

FIGURE 7-17 The Windows platform landscape.

A few items in [Figure 7-17](#) are worth elaborating on:

- UWP apps can produce normal executables, just like desktop apps. Wwahost.exe (%SystemRoot%\ System32\wwahost.exe) is used to host HTML/JavaScript-based UWP apps, as those produce a DLL, not an executable.
- The UWP is implemented by the Windows Runtime APIs, which are based on an enhanced version of COM. Language projections are provided for C++ (through proprietary language extensions known as C++/CX), .NET languages, and JavaScript. These projections make it relatively easy to access WinRT types, methods, properties, and events from developers' familiar environments.
- Several bridging technologies are available, which can transform other types of applications into UWP. See the MSDN documentation for more information on utilizing these technologies.

- The Windows Runtime is layered on top of the Windows subsystem DLLs, just like the .NET Framework. It has no kernel components and is not part of a different subsystem because it still leverages the same Win32 APIs that the system offers. However, some policies are implemented in the kernel, as well as the general support for AppContainers.

- The Windows Runtime APIs are implemented in DLLs residing in the %SystemRoot%\System32 directory, with names in the form *Windows.Xxx.Yyy...Dll*, where the file name usually indicates the Windows Runtime API namespace implemented. For example, *Windows.Globalization.Dll* implements the classes residing in the `Windows.Globalization` namespace. (See the MSDN documentation for the complete WinRT API reference.)